arXiv:2608.03392v4 [cs.SE] 24 Sep 2026

Self-Evolving Coding Agents

Hao Zhou[1] **Haichuan Hu**[1] **Tianyu Luo**[2] **Ye Shang**[2]
Chunrong Fang[2] **Zhenyu Chen**[2] **Liang Xiao**[1] **Quanjun Zhang**[1]

[1]Nanjing University of Science and Technology
[2]Nanjing University

125106010779@njust.edu.cn, huhaichuan2024@gmail.com, Ty_L191025@outlook.com
yeshang@smail.nju.edu.cn, fangchunrong@nju.edu.cn, zychen@nju.edu.cn
xiaoliang@mail.njust.edu.cn, quanjunzhang@njust.edu.cn

Large language models (LLMs) have moved software automation beyond code generation toward coding agents that inspect repositories, invoke tools, run tests, and repair failures. Yet many agents remain episodic, using feedback to solve the current task without changing how they approach the next one. Software engineering offers a particularly suitable setting for overcoming this limitation. Execution provides concrete outcomes and diagnostics, while coding trajectories and versioned repositories make experience traceable and often reproducible. Self-evolving coding agents use such experience to update the reusable components that shape their later work. This survey distinguishes these agent updates from changes to task code and identifies assets, architecture, and model parameters as progressively deeper intervention targets. As intervention deepens, a lesson can influence more decisions and become harder to attribute or reverse. We examine coding signals and adoption timing alongside this taxonomy to show how similar feedback can produce different updates and when those changes begin to shape later execution. Early adoption can use fresh feedback but may rely on incomplete evidence, while waiting permits broader validation but delays reuse. Finally, we examine benchmarks and emerging products, finding that practical systems favor inspectable assets and editable harnesses while evaluation still centers largely on individual task outcomes. Passing tests support the current software artifact under checked conditions; showing that an agent has improved requires evidence that retained changes benefit later decisions.

DATE August 20, 2026

PROJECT iSEngLab/Awesome-Self-Evolving-Coding-Agents

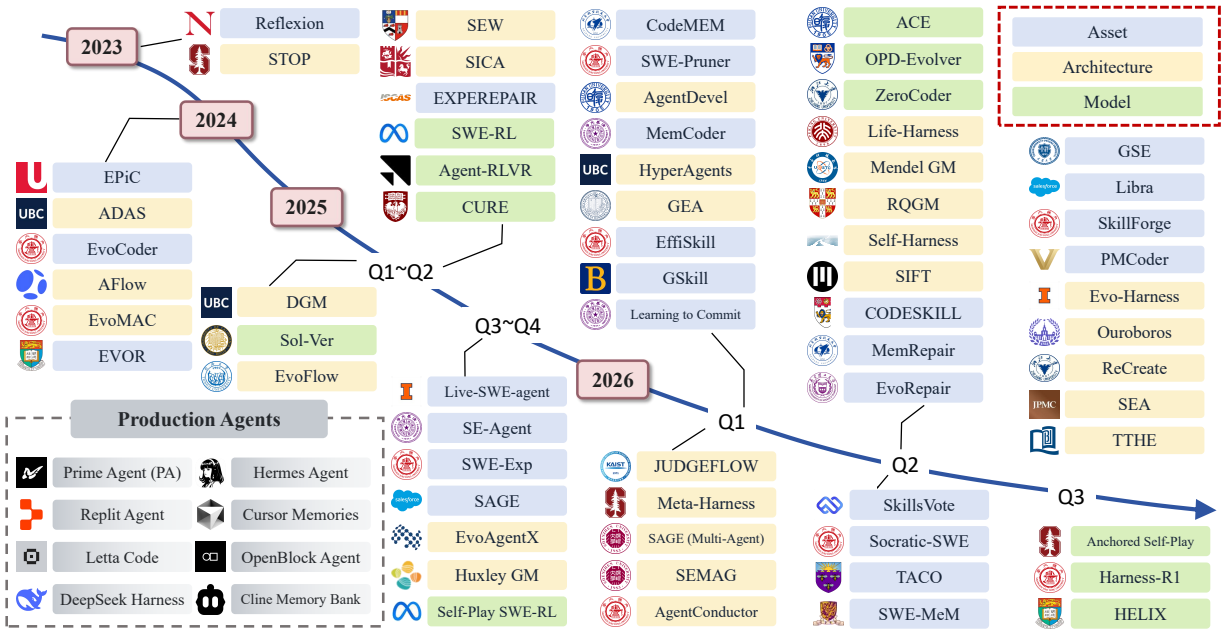


Figure 1: Evolutionary landscape of representative agent methods and coding products from 2023 to 2026. Colors indicate the primary update target in assets, architecture, or model parameters.

Contents

1 Introduction **4**
Relation to Existing Surveys . . . 5

2 Background and Definitions **6**
2.1 Coding Agents . . . 6
2.2 Self-Evolving Agents . . . 6
2.3 Self-Evolving Coding Agents . . . 7

3 Taxonomy of Self-Evolving Coding Agents **9**
3.1 Assets Self-Evolution . . . 9
3.2 Architecture Self-Evolution . . . 9
3.3 Model Self-Evolution . . . 11
3.4 Research Landscape . . . 11

4 Assets Self-Evolution **12**
4.1 Memory Self-Evolution . . . 13
4.2 Skill Self-Evolution . . . 14
4.3 Tool Self-Evolution . . . 16
4.4 Context Self-Evolution . . . 17

5 Architecture Self-Evolution **18**
5.1 Harness Evolution . . . 19
5.2 Workflow Evolution . . . 22
5.3 Multi-Agent Evolution . . . 23

6 Model Self-Evolution **24**
6.1 Self-Play Co-Evolution . . . 24
6.2 Environment-Reward RL . . . 25
6.3 Harness Co-Evolution . . . 26

7 Coding Feedback and Evolution Dynamics **27**
7.1 Why Coding Is a Natural Domain for Self-Evolution . . . 27
7.2 What Coding Signals Reveal . . . 28
7.3 How Signals Drive Agent Evolution . . . 31
7.4 When to Evolve . . . 33

8 Benchmarks and Evaluation **33**

8.1 Conventional Coding Benchmarks Used for Evolution 34
8.2 Benchmarks Designed to Evaluate Evolution 35
8.3 Measuring Benefit, Regression, and Cost 36

9 Self-Evolving Coding Products 37
9.1 Product Landscape 37
9.2 From Experience to Governed Updates 39
9.3 Toward Hot-Swappable Coding Agents 39

10 Challenges and Open Problems 40
Reproducibility, contamination, and benchmark overfitting 40
Feedback reliability, safety, and tool dependence 40
Reversibility, harness bloat, and specialization 41
Long-term memory, skills, and coordination 41
Evaluation beyond short benchmarks 42

11 Conclusion 42

1 Introduction

Large language models have moved software automation beyond code generation (Austin et al., 2021, Chen et al., 2021) toward agents that inspect repositories, use command-line tools, revise files, and test candidate patches. SWE-agent, OpenHands, AutoCodeRover, and related repair systems exemplify this shift toward repository-level execution (Arora et al., 2024, Bouzenia et al., 2024, Wang et al., 2024b, Yang et al., 2024, Zhang et al., 2024b). Benchmarks based on real issues make the resulting repairs measurable, while tying each outcome to a particular repository and environment (Deng et al., 2025, Jimenez et al., 2023, Zhang et al., 2025d). Figure 1 places this development alongside the broader rise of self-evolving agents. The unresolved step is what happens beyond the immediate repair. An agent may use a failed test to fix one patch yet carry no reusable change into later tasks, a limitation that motivates experience-driven issue resolution (Chen et al., 2026c). Figure 2 sketches the feedback and update loop at issue.

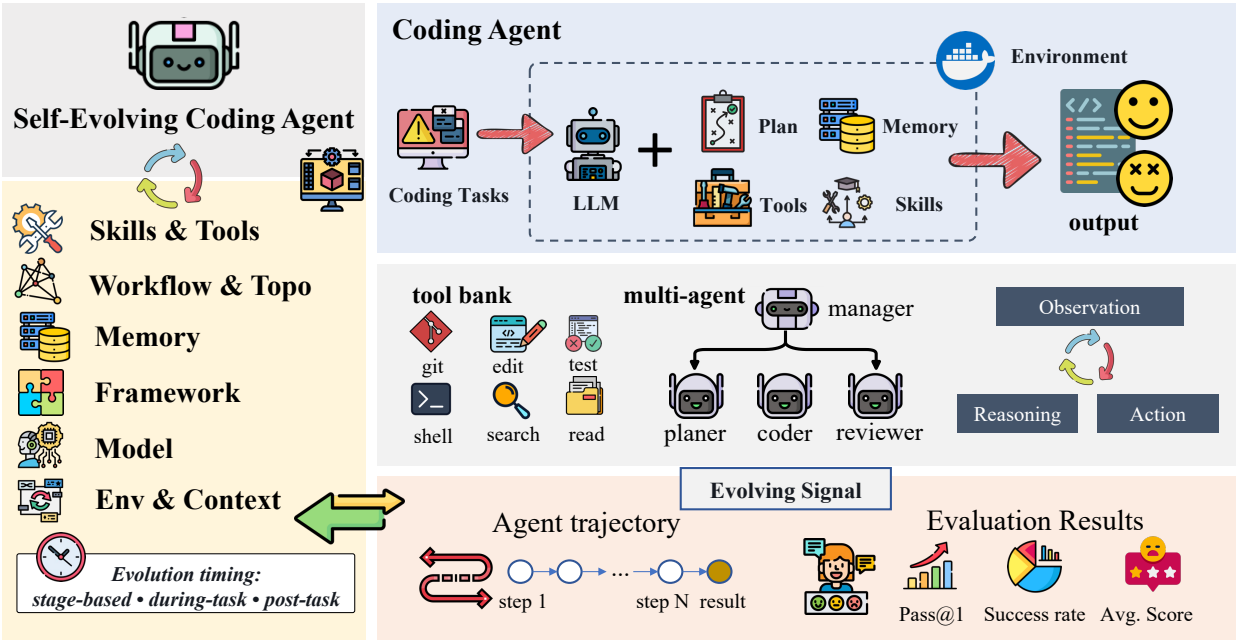


Figure 2: Overview of self-evolving coding agents.

Coding offers unusually concrete evidence for studying this step. Tests provide executable outcomes; compiler messages and stack traces narrow failures; and trajectories record the searches, edits, and retries that preceded a patch (Bouzenia and Pradel, 2024, Garg and Huang, 2026, Jain et al., 2025, Pan et al., 2025). Issues, diffs, and versioned repositories connect those observations to the software conditions under which they arose (Jimenez et al., 2023). Recorded dependencies and tool versions can make many observations re-executable, although changing environments limit that reproducibility (Eliseeva et al., 2025, Hu et al., 2025). Specific, traceable feedback is still incomplete feedback. A passing test checks only selected behavior, and functional success need not establish patch quality or security (Chen and Jiang, 2024, Wei et al., 2025c). Interpreting this evidence requires distinguishing changes to the program being built from changes to the agent that builds it. The first change is visible in a patch and its validation results. The second requires an experience-derived update to a reusable agent component, followed by its use in later execution. Earlier agent research showed how reflection and stored experience can inform later attempts (Shinn et al., 2023, Wang et al., 2025e, Zhao et al., 2024); coding-specific systems now retain repair knowledge across issues (Chen et al., 2026c). We use *self-evolving coding agent* for a system whose retained updates improve subsequent coding behavior. A successful patch may yield an overgeneralized memory, while an unsuccessful attempt may contain a useful diagnosis. The current artifact's result therefore cannot by itself establish the value of an agent update.

The same debugging experience can enter an agent at different depths. SWE-Exp and Repository Memory retain information for later retrieval, while CODESKILL and Socratic-SWE turn trajectories into reusable procedures (Chen et al., 2026c, Li et al., 2026c, Wang et al., 2026a, Xiao et al., 2026a). Live-SWE-agent creates callable utilities during issue resolution (Xia et al., 2025). Other methods revise the execution structure, as in source-level agent evolution,

workflow search, harness repair, or team coordination (Chen et al., 2026a, Hu et al., 2024b, Zhang et al., 2025c, 2024a); Agent-RLVR and SWE-RL instead update model parameters using coding feedback (Da et al., 2025, Wei et al., 2026b). We organize these targets as assets, architecture, and model parameters. The ordering reflects typical intervention depth: a resource can often be edited locally, whereas changes to control flow or learned policy can affect more decisions. Depth does not guarantee benefit. The update target alone does not explain how evolution occurs. Coding signals determine what evidence a method consumes, while adoption timing determines how much evidence is available before a change enters execution. A utility can be created within a task, team instructions can change between development rounds, experience can be retained after an issue, and a model update can aggregate trajectories across tasks (Chen et al., 2026c, Da et al., 2025, Hu et al., 2024b, Xia et al., 2025). Earlier adoption allows a rapid response to local feedback; later adoption permits broader validation but delays reuse.

Evaluation must determine whether a retained change helped after reuse, rather than crediting it for the task that produced it. Issue-resolution benchmarks measure patches, while sequential agent benchmarks make persistence and later-task behavior part of the protocol (Deng et al., 2025, Jiang et al., 2026, Jimenez et al., 2023, Xue et al., 2026, Zhang et al., 2026m). Coding tasks and their tests may also be used to construct updates, select candidates, and report final performance. Reusing related cases across these roles can make adaptation to an evaluator look like general progress. Studies of harness evolution further show why search budgets and harness choice matter when attributing a gain (Ben Sghaier et al., 2026, Vats and Golev, 2026, Wang et al., 2026h). An evolution study should therefore identify what persists, compare updated and frozen agents under matched conditions on held-out tasks, and report regressions and costs alongside gains. Those controls separate improvement from extra attempts, computation, or familiarity with evaluation cases.

Relation to Existing Surveys. Existing surveys examine either agents that evolve across domains or agents that perform software-engineering tasks. General self-evolving-agent surveys discuss programming alongside many other applications, without centering the distinctive evidence produced by tests, diagnostics, and versioned repositories (Fang et al., 2025, Gao et al., 2026a). Software-engineering agent surveys instead organize development tasks, architectures, and collaboration; retaining an experience-derived agent update is not their inclusion criterion (Liu et al., 2024, Wang et al., 2025b). This distinction matters because coding work can change both the project under repair and the agent that repairs it. *Code as Agent Harness* recognizes code as agent infrastructure, but studies that substrate across multiple domains (Ning et al., 2026). Our unit of analysis is the persistent change to a coding agent produced by software-engineering experience. We ask which component retains it, what software evidence motivates it, when it is adopted, and whether its reuse improves later coding behavior. A correct patch establishes a result for one task; it does not establish that the agent learned a useful capability. Table 1 compares these boundaries.

Table 1: Comparison of related surveys by focus, persistent updates, and the role of coding.

Survey	Main focus	Persistent agent update	Role of coding
General self-evolving agents (Gao et al., 2026a)	Evolution mechanisms across domains	Central	One application domain
Comprehensive self-evolving AI agents (Fang et al., 2025)	Feedback-driven agent-system optimization	Central	One specialized domain
LLM agents for SE (Liu et al., 2024)	LLM agent designs and SE tasks	Not required	Core task setting
Agents in SE (Wang et al., 2025b)	Agent applications and roles in SE	Not required	Core task setting
Code as Agent Harness (Ning et al., 2026)	Code-based agent infrastructure	One mechanism	Cross-domain execution substrate
Our Survey	Coding-agent updates and later benefit	Required; tested after reuse	Update and evaluation evidence

This survey makes three contributions that connect the target of evolution, the coding evidence behind an update, and the evaluation of its later effects:

- **Definition and target taxonomy.** We distinguish changes to the coding agent from changes to the code it produces. We then classify persistent agent updates as changes to assets, architecture, or model parameters, ordered by intervention depth.
- **Coding signals and update mechanisms.** We classify the evidence available during software development and examine how methods turn it into persistent updates. We also compare when updates are adopted, showing how timing affects the evidence available before reuse.
- **Evaluation of agent improvement.** We distinguish success on the current coding task from improvement caused by a retained update. Our analysis of benchmarks identifies the matched, held-out, and longitudinal comparisons needed to measure later gains, regressions, and costs.

The remainder of this survey is organized as follows. Section 2 defines coding agents and self-evolving agents, then specifies the retention, reuse, and improvement requirements that distinguish self-evolving coding agents. Section 3 introduces the target-centered taxonomy and explains the different depths of asset, architecture, and model updates, then profiles the surveyed methods by year and publication outlet. Section 4 reviews how memory, skills, tools, and context evolve. Section 5 examines changes to harnesses, workflows, and multi-agent coordination, while Section 6 addresses parameter adaptation. Section 7 analyzes coding signals, how methods turn them into persistent changes, and when those changes are adopted. Section 8 reviews benchmarks and the controls needed to measure later benefit. Section 9 examines how products expose and govern updates. Section 10 discusses open problems in feedback validity, reversibility, and generalization, and Section 11 concludes.

2 Background and Definitions

Coding agents perform software development tasks by interacting with repositories, tools, and execution environments, while the agents themselves are built from code, configurations, and model parameters. Code thus plays a dual role: it is both a product of task solving and part of the system that solves the task. This dual role allows coding capabilities to act on both task artifacts and the agent itself, but modifying code does not inherently constitute self-evolution. The key distinction is whether a change extends beyond improving the current solution to persistently changing how subsequent tasks are solved. Self-evolving agents embody this experience- and feedback-driven adaptation of the agent itself. Self-evolving coding agents ground that process in coding practice: task solving generates experience, experience drives agent updates, and retained updates guide subsequent coding activity. The object of evolution is the agent, not merely the programs it generates or modifies.

2.1 Coding Agents

A coding agent is an LLM-based system that pursues software development objectives through actions in a development environment. It decides what information to obtain, what changes to make, and how to assess their effects. Repository-level tasks require reasoning across existing files and dependencies (Jimenez et al., 2023), often with context gathered during task execution. SWE-agent and OpenHands support this process through interfaces for repository navigation, editing, and execution (Wang et al., 2024b, Yang et al., 2024). Interaction serves both to modify software and to investigate its behavior: even an unsuccessful repair can provide evidence that guides the next attempt.

The surrounding system shapes what the model can observe and do, and how its decisions are organized. CodeAct expresses actions through executable code (Wang et al., 2024a), AutoCodeRover retrieves context through structured code search (Zhang et al., 2024b), and Debug2Fix exposes runtime behavior through interactive debugging (Garg and Huang, 2026). MetaGPT coordinates specialized development roles (Hong et al., 2023), while MASAI assigns distinct objectives and strategies to modules (Arora et al., 2024). SGAgent bridges fault localization and patch generation with explicit, context-informed repair suggestions (Zhang et al., 2026f); Trae Agent organizes candidate generation, pruning, and selection (Trae Research Team et al., 2025). The value of these arrangements lies in the decisions they enable, alongside their computational cost. Agentless offers a useful comparison: a fixed localization, repair, and validation pipeline can also achieve competitive repair results (Xia et al., 2024).

2.2 Self-Evolving Agents

A self-evolving agent uses experience and feedback to update its own reusable state or components, allowing past interactions to shape subsequent behavior. Updates can affect memory, skills, tools, architecture, or model parameters (Fang et al., 2025, Gao et al., 2026a). Experience both informs current actions and provides a basis for revising how the agent will act again. These revisions persist as part of its memory, capabilities, or decision process beyond the interaction that produced them.

Experience can support adaptation without changing model weights: ExpeL extracts reusable insights (Zhao et al., 2024), Voyager accumulates composable executable skills (Wang et al., 2023), and Agent Workflow Memory induces reusable routines from task trajectories (Wang et al., 2025e). A-MEM further revises existing memories and their links as new experiences arrive (Xu et al., 2025). Adaptation also reaches control logic through Gödel Agent's modification of its own implementation (Yin et al., 2025). Policy learning offers another route: WebRL builds an online training curriculum from unsuccessful web interactions (Qi et al., 2025), while Tool-R0 co-evolves a task generator and a

tool-using solver through self-play reinforcement learning (Acikgoz et al., 2026). These mechanisms turn experience into changes that subsequent execution can use. A retained record contributes to self-evolution when it participates in this update-and-reuse process; storage alone is insufficient. Reuse may occur within a continuing task or across tasks, and human feedback or approval can accompany system-generated updates. Retention and reuse establish an adaptation mechanism, while reliable gains require evidence from evaluating the updated agent.

2.3 Self-Evolving Coding Agents

Coding agents acquire experience as they inspect repositories, execute programs, and revise solutions. This experience can help complete the current task, but it can also inform changes to the system performing the task. Self-evolving coding agents connect software task execution with improvement of the agent itself, turning coding experience into retained updates whose reuse improves subsequent coding performance. Task solving supplies both software artifacts and experience from which the agent can develop further capabilities.

Definition

A coding agent that uses experience and feedback from coding interactions to update its own assets, architecture, or model parameters, and improves its problem-solving capability, reliability, or efficiency in subsequent coding activity by retaining and reusing those updates.

To distinguish changes to the task solution from changes to the agent, we represent the reusable agent configuration at adaptation round t as

$$\mathcal{A}_t = (\mathcal{S}_t, \mathcal{H}_t, \theta_t), \tag{1}$$

where $\mathcal{S}_t$ denotes assets (memory, skills, tools, and context), $\mathcal{H}_t$ denotes architecture (harness, workflow, and multi-agent organization), and θ_t denotes model parameters. These components correspond to the three categories of evolution targets in our taxonomy and jointly shape the agent's capabilities and decision process. For a software task q in environment $\mathcal{E}$, execution is represented by

$$(z_t, \tau_t, f_t) = \text{Execute}(\mathcal{A}_t, q, \mathcal{E}). \tag{2}$$

Here, z_t is the software artifact produced or revised, τ_t records the agent's actions and resulting observations, and f_t denotes feedback on execution and its outcomes. The artifact is the immediate result of task solving; the trajectory and feedback also provide experience for revising the system that produced it.

An update mechanism $\mathcal{U}$ uses this experience to propose changes to the reusable agent configuration:

$$\widetilde{\mathcal{A}}_{t+1} = \mathcal{U}(\mathcal{A}_t, \tau_t, f_t). \tag{3}$$

The candidate may revise one or several components of $\mathcal{A}_t$ while leaving the others unchanged. The tilde distinguishes a proposed configuration from one adopted for subsequent execution. A proposal may be accepted, revised, or discarded; for a single candidate, adoption is expressed as

$$\mathcal{A}_{t+1} = \begin{cases} \widetilde{\mathcal{A}}_{t+1}, & \text{if the candidate is accepted,} \\ \mathcal{A}_t, & \text{otherwise.} \end{cases} \tag{4}$$

Acceptance retains the candidate's changes in $\mathcal{A}_{t+1}$, making them available to guide later execution. Rejection preserves the existing configuration, while revision produces another candidate for consideration.

A single coding task can involve several rounds of adaptation, and the effects of one update may extend to multiple later tasks; t therefore indexes adaptation rounds. An agent may use experience gained during an ongoing task to adjust its remaining work (Xia et al., 2025), or retain those adjustments for future tasks. Persistence allows experience to have an effect beyond the operation that produced the update, giving a coding attempt the potential to change how the agent solves problems later. Yet retained changes may encode useful strategies or mistaken generalizations from local experience. Only when their subsequent use improves problem-solving capability, reliability, or efficiency can the agent be said to have progressed through experience, rather than merely accumulated changes.

The operational boundary. We identify self-evolving coding agents using four criteria that examine how code-related experience changes the agent itself and whether reusing those changes improves subsequent coding performance:

(1) **Update target.** The change must affect an identifiable component of $\mathcal{A}_t$. Repairing a failing patch or making a target program faster changes the task artifact z_t. Self-Refine's code-refinement loop revises generated outputs (Madaan et al., 2023); AlphaEvolve's search over candidate algorithms likewise optimizes task programs (Novikov et al., 2025). These operations alone constitute task solving.

(2) **Update source.** The system must generate updates from code-related experience and feedback, such as repository interactions, execution traces, or test results.

(3) **Update reuse.** Updates must be retained as agent components and actually used in later coding execution, within the same task or across tasks. Storage alone is insufficient.

(4) **Capability improvement.** Reusing the updates must improve the agent's problem-solving capability, reliability, or efficiency in subsequent coding activity, supported by comparisons with the pre-update agent under comparable conditions.

From retained experience to behavioral influence. We analyze improvement through five links: *experience collection, component update, subsequent use, behavioral change, and effect evaluation*. These links help locate where an evolution mechanism succeeds or breaks down. Collection supplies observations, while updating converts them into reusable components. This transition requires deciding what to retain and which conditions govern its applicability. Subsequent use brings the update into execution, but use alone does not establish contribution. A retrieved memory may repeat available information, and an invoked skill may recommend an action the agent would have taken anyway. Let $\pi_{\mathcal{A}}(\cdot \mid x)$ denote the agent's action-selection distribution at a matched decision state x. The behavioral difference induced by an update can be expressed as

$$\delta_t(x) = d_{\mathrm{TV}}\big(\pi_{\mathcal{A}_{t+1}}(\cdot \mid x), \pi_{\mathcal{A}_t}(\cdot \mid x)\big)\,, \tag{5}$$

where d_{TV} is the total variation distance between probability distributions. Here, actions may include tool calls, code edits, or other execution choices. A value of $\delta_t(x) = 0$ indicates unchanged action probabilities at that state, while $\delta_t(x) > 0$ indicates a behavioral change without establishing its benefit. To assess improvement, let $\mathcal{D}$ denote an evaluation distribution over tasks or matched continuation states within the intended scope of reuse, B a common execution budget, and $s(\mathcal{A}, q; B)$ a performance score on case q, with higher values indicating better performance. The expected gain is

$$\Delta J_t = \mathbb{E}_{q\sim\mathcal{D}}\left[s(\mathcal{A}_{t+1}, q; B) - s(\mathcal{A}_t, q; B)\right]. \tag{6}$$

The expectation also averages over execution randomness. Evidence for $\Delta J_t > 0$ supports an average improvement under the specified scope and budget. A larger behavioral difference need not yield a larger gain. Coding introduces a further gap between the scope of feedback and the scope of reuse. Tests evaluate a concrete software artifact, whereas an agent update may influence many future decisions. A successful patch does not by itself validate a reusable debugging rule derived from it. Effective evolution therefore requires preserving relevant conditions when abstracting experience and aligning $\mathcal{D}$ with the claimed scope of reuse. The five links expose where useful evidence may be lost, overgeneralized, or fail to influence subsequent behavior.

How coding shapes evolution. Coding interactions connect the agent's problem-solving decisions with actual execution outcomes, providing a verifiable basis for experience formation. Test failures expose limitations of the current solution while also providing evidence for examining the decisions made during task solving and their consequences. Analysis of historical repairs further supports the identification of effective strategies and their applicable conditions, turning experience from individual tasks into reusable repair knowledge (e.g., SWE-Exp (Chen et al., 2026c), EvoRepair (Hu et al., 2026), and Repository Memory (Wang et al., 2026a)). Translating experience into agent updates involves using this knowledge to shape subsequent learning and execution. Repair patterns can be abstracted into reusable guidance, while recurring execution problems can inform the proposal and validation of changes to how the agent operates (e.g., CODESKILL (Li et al., 2026c), Socratic-SWE (Xiao et al., 2026a), Self-Harness (Zhang et al., 2026b), and AutoSaddler (Park et al., 2026)). Experience is context-dependent, and its reuse beyond the original conditions requires further execution evidence to assess and refine the applicability of the retained knowledge. Table 2 summarizes the defining requirements of the three agent concepts.

Table 2: Defining requirements across the three agent concepts.

Agent Type	Coding Tasks	Agent Updates	Updates from Coding	Retention and Reuse	Verified Coding Gains
Coding Agents	✓	✗	✗	✗	✗
Self-Evolving Agents	✗	✓	✗	✓	✗
Self-Evolving Coding Agents	✓	✓	✓	✓	✓

3 Taxonomy of Self-Evolving Coding Agents

Coding experience can improve an agent by changing the resources it can use, the organization of its execution, or the parameters underlying its decisions. Following $\mathcal{A}_t = (\mathcal{S}_t, \mathcal{H}_t, \theta_t)$, we organize these targets into **Assets Self-Evolution**, **Architecture Self-Evolution**, and **Model Self-Evolution**. Figure 3 presents the three categories, their subcategories, and representative works. Assets evolution changes *what is available*, for example by adding a tool, revising a skill, or refining memory and context. Architecture evolution changes *how execution is organized*, including the harness, workflow, and collaboration among agents. Model evolution changes *the parameterized policy* that shapes decisions within this system. This ordering reflects progressively deeper intervention in the agent's implementation. Assets are typically modular resources that can be added, revised, or replaced within an existing execution structure. Architecture updates reach into the code or configuration governing execution, changing control flow, stage dependencies, or coordination among agents. Model updates reach further into the learned parameters underlying action selection. The progression thus moves from modifying available resources, through restructuring the system that uses them, to updating its learned decision mechanism. Tool updates may also involve code, but they revise available operations, while architecture updates change how the system organizes and controls execution.

3.1 Assets Self-Evolution

Assets comprise the reusable information and capabilities that support coding activity. We represent the asset state as

$$\mathcal{S}_t = (\mathcal{M}_t, \mathcal{K}_t, \mathcal{T}_t, \mathcal{C}_t), \qquad \widetilde{\mathcal{S}}_{t+1} = \mathcal{U}_{\mathcal{S}}(\mathcal{A}_t, \tau_t, f_t), \tag{7}$$

where $\mathcal{M}_t$, $\mathcal{K}_t$, $\mathcal{T}_t$, and $\mathcal{C}_t$ denote memory, skills, tools, and context, respectively. Memory retains knowledge acquired from prior interactions or external sources; skills organize knowledge into reusable procedures; tools provide executable operations; and context determines the information, instructions, and environmental settings available during execution. Assets self-evolution changes what the agent can recall, apply, invoke, or condition its reasoning on, with representative examples including SWE-Exp (Chen et al., 2026c), CODESKILL (Li et al., 2026c), and Live-SWE-agent (Xia et al., 2025). These components may share content while serving different functions: knowledge retained in memory can enter the current context, and a learned procedure can be realized through an executable tool.

3.2 Architecture Self-Evolution

Architecture determines how the agent's capabilities are organized into task execution. We describe it through three complementary aspects,

$$\mathcal{H}_t = (h_t, w_t, g_t), \qquad \widetilde{\mathcal{H}}_{t+1} = \mathcal{U}_{\mathcal{H}}(\mathcal{A}_t, \tau_t, f_t), \tag{8}$$

where h_t denotes the harness, w_t the workflow, and g_t the multi-agent organization. The harness governs runtime interaction and execution control; the workflow specifies the arrangement and dependencies of task-solving stages; and the multi-agent organization determines how responsibilities and information are distributed among participating agents. Architecture self-evolution changes how available capabilities are coordinated, allowing experience to revise the procedure through which subsequent tasks are solved, as exemplified by DGM (Zhang et al., 2025c), AFlow (Zhang et al., 2024a), and EvoMAC (Hu et al., 2024b). These architectural aspects are not independent: a change in collaboration may alter the workflow, while its implementation may require modifications to the harness. For example, moving validation ahead of a costly rewrite changes which assumptions reach implementation without adding a new capability. Requiring a reviewer to inspect an intermediate result similarly changes a decision rule while leaving the communication graph fixed. Architecture therefore includes both structural connections and the rules governing their use. Even a small edit to these rules can affect many later decisions, making the scope of the change more informative than its code size.

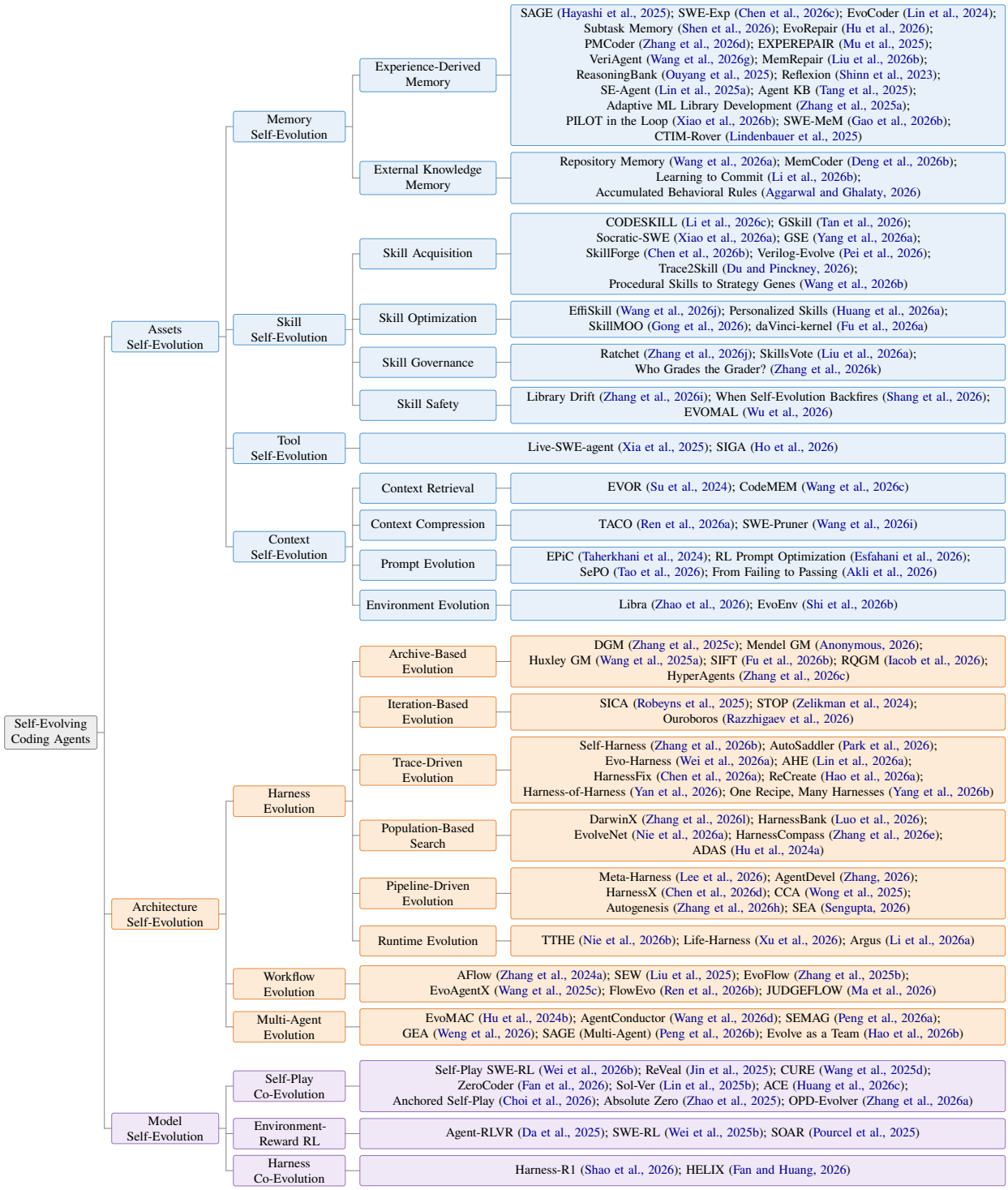


Figure 3: Hierarchical taxonomy of self-evolving coding agents by the primary persistent target of evolution: Assets Self-Evolution (blue), Architecture Self-Evolution (orange), and Model Self-Evolution (purple). The leaf nodes cover all 108 papers in Section 1 of the companion repository's taxonomy, using abbreviated titles and citations to identify each work.

3.3 Model Self-Evolution

Model self-evolution incorporates coding experience into the parameters underlying action selection. Let x denote the information available at a decision point and a an agent action. Parameter adaptation changes the policy exercised within the surrounding agent system,

$$\begin{aligned} \widetilde{\theta}_{t+1} &= \mathcal{U}_\theta(\mathcal{A}_t, \tau_t, f_t), \\ \pi_{\theta_t}(a \mid x; \mathcal{S}_t, \mathcal{H}_t) &\longrightarrow \pi_{\widetilde{\theta}_{t+1}}(a \mid x; \mathcal{S}_t, \mathcal{H}_t). \end{aligned} \tag{9}$$

Holding assets and architecture fixed in this expression isolates the effect of parameter changes without requiring them to remain fixed throughout evolution. Our taxonomy includes self-play co-evolution, environment-reward reinforcement learning, and harness co-evolution. These directions organize learning through coupled task generation and solving, feedback from environmental interaction, or adaptation between the model and its runtime harness. Representative approaches include Self-Play SWE-RL (Wei et al., 2026b) and Agent-RLVR (Da et al., 2025). Their shared target is the learned policy, through which experience can influence subsequent decisions without retrieving each contributing experience explicitly.

3.4 Research Landscape

We describe the publication landscape of the 108 methods in Figure 3, counting each once by its primary update target. Benchmarks, products, and separate empirical studies are excluded. First availability follows arXiv dates for 103 papers and catalogue years for five. The 2026 quarterly view uses arXiv months or documented release quarters for four papers without arXiv links. Venue labels follow the catalogue and are kept separate from first availability: 13 papers appeared on arXiv before their venue year.

Figure 4 shows two papers in 2023, six in 2024, 25 in 2025, and 75 in 2026, with 18, 30, and 27 in the first three quarters of 2026. The first half of 2026 alone contains 48 methods, nearly twice the 2025 total, showing marked acceleration within this corpus. The 2026 entries comprise 37 asset, 32 architecture, and six model updates; across all years, these categories contain 51, 43, and 14 papers, respectively. Assets and architecture thus account for 69 of 75 entries in 2026, compared with 17 of 25 in 2025, suggesting growing attention to reusable resources and editable execution structures. These counts describe research activity rather than effectiveness. The curated collection and incomplete Q3 also preclude conclusions about field-wide growth or a slowdown after Q2.

The outlet view records 73 papers as arXiv, 11 as ICLR, six as NeurIPS, five as ICML, and 13 under other labels. Here, arXiv means no other venue is recorded; catalogue labels may lag publication. This distribution describes dissemination at the snapshot date, while Section 8 examines evidence of actual improvement.

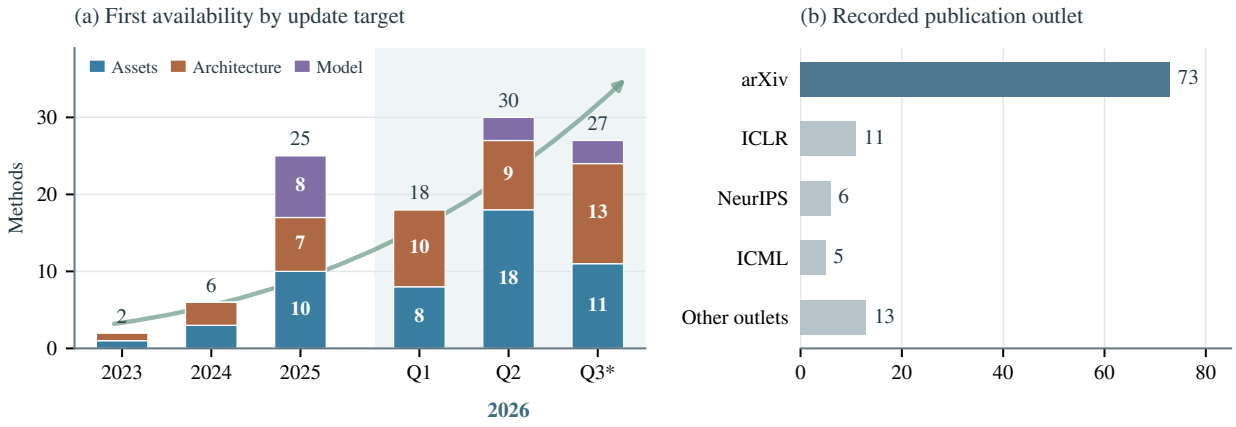


Figure 4: Publication landscape of 108 methods. (a) First availability by update target; Q3* is incomplete as of September 2026. (b) Recorded outlets; "arXiv" indicates no other listed venue.

The following sections examine the mechanisms and representative work in assets self-evolution (Section 4), architecture self-evolution (Section 5), and model self-evolution (Section 6).

4 Assets Self-Evolution

Asset self-evolution turns coding experience and external knowledge into resources that remain available for subsequent execution. The evolving state is $\mathcal{S}_t = (\mathcal{M}_t, \mathcal{K}_t, \mathcal{T}_t, \mathcal{C}_t)$. The main design choices concern the unit of experience retained, its representation, and the conditions under which it is revised or reused. Memory can preserve a diagnosis and its supporting evidence; a skill can organize that knowledge into a procedure; a tool can implement an operation; and context can expose the information needed at the next decision point. These roles connect the four asset types while giving each a distinct update target. Their evolution can involve consolidation, correction, or removal as well as the addition of new resources. Table 3 compares representative methods by their asset targets, update triggers, and the operations used to construct and maintain those assets.

Table 3: Asset targets, operations, and update triggers of representative methods.

Method	Asset target				Asset operations			Update trigger
	Memory	Skill	Tool	Context	Add	Delete	Revise	
Reflexion (Shinn et al., 2023)	●	○	○	○	✓			Failed trials
CODESKILL (Li et al., 2026c)	○	●	○	○	✓		✓	New/failed experience
Live-SWE-agent (Xia et al., 2025)	○	○	●	○	✓		✓	Tool needs
TACO (Ren et al., 2026a)	○	○	○	●	✓		✓	Rule gaps; complaints
SWE-Exp (Chen et al., 2026c)	●	○	○	○	✓			Repair outcomes
GSE (Yang et al., 2026a)	○	●	○	○	✓		✓	Diagnosed skill gaps
EVOR (Su et al., 2024)	○	○	○	●	✓			Execution feedback
EvoCoder (Lin et al., 2024)	●	○	○	○	✓	✓	✓	Reproduction outcomes
GSkill (Tan et al., 2026)	○	●	○	○	✓		✓	Rollout feedback
SIGA† (Ho et al., 2026)	○	○	●	○	✓		✓	Validation rewards
CodeMEM (Wang et al., 2026c)	○	○	○	●	✓	✓		Interaction rounds
Subtask Memory (Shen et al., 2026)	●	○	○	○	✓			Subtask completion
SkillForge (Chen et al., 2026b)	○	●	○	○	✓			Synthetic repair traces
EPiC (Taherkhani et al., 2024)	○	○	○	●	✓		✓	Failed tests
EvoRepair (Hu et al., 2026)	●	○	○	○	✓		✓	Repair feedback
Verilog-Evolve (Pei et al., 2026)	○	●	○	○	✓		✓	Verifier feedback
SePO (Tao et al., 2026)	○	○	○	●	✓		✓	Evaluation scores
EXPEREPAIR (Mu et al., 2025)	●	○	○	○	✓	✓	✓	Repair feedback
Trace2Skill (Du and Pinckney, 2026)	○	●	○	○	✓		✓	Rollout diagnostics
Libra (Zhao et al., 2026)	○	○	○	●	✓	✓	✓	Localization failures
ReasoningBank (Ouyang et al., 2025)	●	○	○	○	✓			Task outcomes
EffiSkill (Wang et al., 2026j)	○	●	○	○	✓			Slow/fast code pairs
Agent KB (Tang et al., 2025)	●	○	○	○	✓	✓	✓	Rewards; capacity
SkillMOO (Gong et al., 2026)	○	●	○	○	✓	✓	✓	Task success; cost
Repository Memory (Wang et al., 2026a)	●	○	○	○	✓			Repository history
Ratchet (Zhang et al., 2026j)	○	●	○	○	✓	✓		Failures; contribution
MemCoder (Deng et al., 2026b)	●	○	○	○	✓			New commits
SkillsVote (Liu et al., 2026a)	○	●	○	○	✓		✓	Execution attribution
SAGE (Hayashi et al., 2025)	●	○	○	○	✓			Initial rollout
Library Drift (Zhang et al., 2026i)	○	●	○	○	✓	✓		Failures; contribution

●/○ denote primary/other categories; ✓ marks documented operations, and blanks mean unconfirmed. Add, Delete, and Revise denote creation, removal or retirement, and modification of retained assets, respectively.

† SIGA evolves adapter contents across assets; its tool-category placement does not imply that new runtime tools account for its gains.

4.1 Memory Self-Evolution

Memory evolution updates $\mathcal{M}_t$ within the asset state $\mathcal{S}_t$. Its central design choice is the unit of knowledge that survives an interaction, together with the conditions under which it should be recalled. We distinguish experience-derived memory from external knowledge memory by the provenance of that knowledge. Both concern explicit storage outside model parameters, but the former abstracts agent executions, whereas the latter incorporates repository history or human guidance. For comparison, we represent a memory item as $m_i = (v_i, e_i, \Omega_i)$, where v_i is the retained knowledge, e_i its supporting evidence, and Ω_i its applicability conditions. These fields may be explicit metadata or implicit in the memory text. An agent-derived diagnosis and a developer-provided convention can share this representation while drawing on different sources of e_i.

Experience-Derived Memory. The shortest reuse horizon is a later attempt at the same problem. Reflexion retains verbal interpretations of feedback in an episodic buffer to guide subsequent trials (Shinn et al., 2023). SAGE instead abstracts an initial rollout into a plan that preserves dependencies and constraints before re-executing the same task (Hayashi et al., 2025). SE-Agent revises, recombines, and refines pilot trajectories, using relationships among previous attempts to explore alternative solutions (Lin et al., 2025a). These methods demonstrate different ways of making earlier computation useful to later execution. Their same-task improvements should be distinguished from evidence that a retained memory benefits unseen tasks, and from improvements obtained simply by increasing the number of attempts.

Across tasks, abstraction must balance reusable guidance against the details that make a past solution applicable. SWE-Exp distills successful and failed repair trajectories into a bank spanning issue comprehension and concrete code changes (Chen et al., 2026c). EvoCoder addresses issue reproduction through a hierarchical pool separating common experience from repository-specific experience (Lin et al., 2024). EXPEREPAIR separates concrete repair demonstrations from reflective insights in episodic and semantic memories, combining both when constructing an inference prompt (Mu et al., 2025). These designs expose complementary dimensions of memory organization. Repository scope determines where a lesson applies, while abstraction level determines how much of the original repair must be reconstructed before it can guide a decision.

Retrieval also needs to match the agent's current activity. Subtask Memory aligns storage, retrieval, and updating with functional subtasks, addressing the mismatch between similar issue descriptions and different stage-specific reasoning needs (Shen et al., 2026). PMCoder couples a phase planner with episodic memory, so the phase conditions retrieval and memory statistics inform stuck detection and replanning (Zhang et al., 2026d). Its matched component ablations and trajectory analyses connect better state management with fewer repeated failures. Here the evidence concerns retaining useful observations during a long repair episode; transferring the mechanism across backbones does not itself demonstrate transfer of accumulated memories across issues. Together, these methods suggest that relevance depends on the decision being made, rather than issue similarity alone. A relevance-based retrieval rule can make this dependence explicit,

$$\mathcal{R}_t^{\mathcal{M}}(x) = \mathrm{Top}_{k_M}\left(\{m_i \in \mathcal{M}_t : \Omega_i(x) = 1\};\, r(m_i, x)\right), \tag{10}$$

where x is the current decision state, $r(m_i, x)$ a relevance score, and Top_{k_M} returns up to k_M highest-scoring eligible items. The condition $\Omega_i(x) = 1$ represents an applicability check, such as a matching repository, dependency version, or task phase. This is an analytical retrieval rule; individual methods may implement eligibility and relevance jointly or use soft scores. It separates the existence of an item in $\mathcal{M}_t$ from its inclusion in the recalled set $\mathcal{R}_t^{\mathcal{M}}(x)$.

Domain-specific feedback further changes what should be remembered. EvoRepair combines accumulation within a vulnerability with reuse across vulnerabilities, updating its experience bank through quality-aware scoring (Hu et al., 2026). MemRepair distinguishes historical fixes, security patterns, and refinement trajectories, preserving repository conventions alongside reusable defenses and lessons from failed validation (Liu et al., 2026b). In RTL generation, VeriAgent converts simulation and synthesis feedback into memory nodes with triggering conditions, guidance, and metadata; its manager inserts, refines, or discards nodes (Wang et al., 2026g). The adaptive system for ML library development retains verified generated programs as demonstrations and selects them by difficulty, using successful examples to support remaining tasks in an unfamiliar programming language (Zhang et al., 2025a). These mechanisms retain different evidence, from repair failures to physical design signals and executable examples, rather than treating every trajectory as an interchangeable text record.

Memory can also mediate experience sharing and control its own information cost. ReasoningBank extracts strategies from self-judged successes and failures, then incorporates new experience after retrieval-guided execution; its MaTTS

extension uses additional computation to diversify that experience (Ouyang et al., 2025). Agent KB shares structured trajectories across heterogeneous frameworks, combining planning-time retrieval with feedback-time diagnostic guidance and a disagreement gate (Tang et al., 2025). PILOT distills procedures and failure modes into skills and memory during live supervisor–worker execution (Xiao et al., 2026b). SWE-MeM learns when, what, and how to compress histories through synthesized training trajectories and memory-aware GRPO (Gao et al., 2026b). PILOT couples retained assets with supervision, while SWE-MeM combines memory management with parameter learning. Their gains therefore reflect changes beyond memory contents. More retained experience is not necessarily better. CTIM-Rover adds general and repository-specific cross-task memories to AutoCodeRover but does not outperform its baseline in the evaluated configurations; the authors identify distracting memories and exemplars as a likely cause (Lindenbauer et al., 2025). This negative result makes selective recall part of the improvement mechanism. Memory may change action selection, giving $\delta_t(x) > 0$ at some states, without producing a positive expected gain ΔJ_t. Evaluation should therefore separate memory quality and retrieval effects from added search, supervision, or training.

External Knowledge Memory. Repository history records decisions that the agent did not necessarily make itself. Repository Memory uses historical commits, linked issues, and summaries of actively changing code regions to improve localization (Wang et al., 2026a). This supplements a code snapshot with associations between past problems and likely modification sites. MemCoder structures human commit history into intent-to-code mappings, refines execution using verification feedback, and incorporates human-validated solutions into long-term knowledge (Deng et al., 2026b). The distinction is between recalling where and why developers changed code and retaining the agent's own trial-and-error record, although a single memory system can eventually contain both. External supervision can also reveal project conventions that tests do not fully express. Learning to Commit attempts historical issues, compares its changes with actual commits, and distills the differences into reusable guidance about style, internal APIs, and architectural constraints (Li et al., 2026b). Its chronological split evaluates future pull requests, directly aligning the reuse claim with later repository work, although the reported study centers on one repository. Accumulated Behavioral Rules converts accepted review comments into persistent instructions and a pre-submission checklist (Aggarwal and Ghalaty, 2026). Its production observations support cross-session retention of corrections, but the small session sample and absence of a controlled baseline limit causal claims about improvement. Such artifacts overlap with skills and context in their eventual use; their distinguishing contribution here is converting developer knowledge into a maintained source of guidance.

Across both sources, effective memory requires preserving the relationship between v_i, e_i, and Ω_i. A compact description of maintenance is

$$\widetilde{\mathcal{M}}_{t+1} = \left(\mathcal{M}_t \setminus \mathcal{D}_t^{\mathcal{M}}\right) \cup \mathcal{W}_t^{\mathcal{M}}, \tag{11}$$

where $\mathcal{D}_t^{\mathcal{M}} \subseteq \mathcal{M}_t$ contains entries selected for removal or replacement and $\mathcal{W}_t^{\mathcal{M}}$ contains proposed new or revised entries. Merging several records removes their originals and writes a consolidated item; retirement removes an item without replacement. The tilde denotes a candidate state subject to adoption. This representation allows memory to improve through correction and consolidation even when $|\widetilde{\mathcal{M}}_{t+1}| \leq |\mathcal{M}_t|$. A repository convention may cease to hold after refactoring, while a general repair strategy may require different local actions. Consequently, the evaluation distribution $\mathcal{D}$ used to estimate ΔJ_t should reflect whether reuse is claimed within an episode, across issues in one repository, or across repositories. That alignment determines whether memory has supported local continuity, accumulated project expertise, or broader transfer.

4.2 Skill Self-Evolution

Skill self-evolution turns coding knowledge into reusable guidance for action, updating the skill collection $\mathcal{K}_t$. A skill specifies a procedure and the conditions under which it should be applied, potentially combining instructions with executable resources. Its contribution therefore depends on more than whether its content accurately summarizes a past task. The procedure must remain applicable, be selected when needed, and improve the decisions made by the receiving agent. Following Figure 3, we examine acquisition, optimization, governance, and safety as complementary concerns across the skill lifecycle. We write $k = (\Omega_k, p_k, \mathcal{Q}_k)$ for its applicability conditions, procedure, and resource requirements, respectively. The requirements $\mathcal{Q}_k$ can identify tools, dependencies, or other skills needed to execute p_k.

Skill Acquisition. Acquisition begins with choosing the experience from which a procedure can be abstracted. A candidate $\widetilde{k} = \text{Distill}(\tau_t, f_t; \mathcal{K}_t)$ captures a proposed procedure together with its conditions and requirements; conditioning on $\mathcal{K}_t$ allows the distiller to account for existing guidance. CODESKILL (Li et al., 2026c) learns a management policy that extracts task-level procedures and event-driven responses from coding trajectories. It combines

rubric feedback with the incremental execution benefit of generated skills for a frozen downstream agent, and checks whether the trajectory follows the supplied guidance. This connects acquisition to the earlier distinction between subsequent use and behavioral contribution. Socratic-SWE (Xiao et al., 2026a) gives extracted skills a different role. Its registry summarizes recurring failures and effective repairs to guide the synthesis of verifiable repository tasks, which then train the Solver. Skills thus influence future behavior through curriculum construction as well as through direct instructions, illustrating interaction between asset and model evolution.

Repository-specific methods can actively generate the experience they need. GSkill (Tan et al., 2026) combines SWE-smith tasks with reflective evolutionary optimization of repository skill documents, evaluating candidate guidance through agent rollouts. SkillForge (Chen et al., 2026b) instead creates practice issues by reimplementing test-covered repository functionality, then distills the resulting solutions into skills associated with repository entities. Both use a repository as a source of practice, but their retained guidance differs in how it is organized for later use. Broader reuse also requires compatibility with skills already acquired. GSE (Yang et al., 2026a) maintains a Skill Relation Graph, consolidates related local updates, and uses replay verification to limit overfitting and regression. Acquiring a useful procedure therefore includes deciding how its scope and relationships fit the existing collection.

Hardware coding exposes the importance of the available verifier. Verilog-Evolve (Pei et al., 2026) derives modular guidance from logged refinement histories, with creation, improvement, or skipping informed by verifier reports. Simulation, synthesis, and timing proxies provide distinct evidence about what a useful procedure must preserve. Trace2Skill (Du and Pinckney, 2026) uses an oracle, mutator, and selector loop to turn rollout failures into task-specific skills for later search, editing, validation, and recovery. Its bounded dense verifier feedback provides diagnostic observations while withholding hidden harnesses and reference solutions. The two approaches emphasize cross-session accumulation and test-time adaptation, respectively, so their reuse claims require different evaluation scopes. Representation also affects whether the acquired knowledge becomes actionable. *From Procedural Skills to Strategy Genes* (Wang et al., 2026b) finds in scientific code-solving experiments that compact control-oriented representations and distilled failure warnings can outperform expanded documentation packages. More detailed recollection need not yield more effective procedural guidance.

Skill Optimization. Optimization concerns both the quality of procedures and the way they are selected and applied. To isolate a skill collection's contribution, let $\mathcal{A}_t[\mathcal{K}]$ denote the agent with its skill collection replaced by $\mathcal{K}$ and all other components held fixed. Define $G_t(\mathcal{K}) = \mathbb{E}_{q\sim\mathcal{D}}[s(\mathcal{A}_t[\mathcal{K}], q; B)]$ using the same evaluation distribution and execution budget as before, with the expectation also averaging over execution randomness. A candidate's marginal gain is

$$\Delta_t^{\mathcal{K}}(k \mid \mathcal{K}) = G_t(\mathcal{K} \cup \{k\}) - G_t(\mathcal{K}). \tag{12}$$

For a revision, $\mathcal{K}$ excludes the version being replaced, so the old and new versions can be assessed against the same base collection. A skill's utility depends on this base; overlap, conflict, or complementarity can make its contribution differ from its standalone performance. EffiSkill (Wang et al., 2026j) mines concrete Operator Skills and higher-level Meta Skills from slow/fast program pairs, then combines diagnosis, retrieval, and planning to optimize unseen programs without runtime feedback. Its offline library provides a comparison point for systems that continually revise skills during execution. daVinci-kernel (Fu et al., 2026a) closes that loop for GPU kernels, jointly training skill selection, kernel generation, and skill summarization on a shared model backbone. New skills enter its library only after execution confirms reproducible speedups. Its improvements therefore involve both an evolving skill collection and parameter adaptation, requiring care when attributing gains to $\mathcal{K}_t$ alone.

The optimization objective also matters. SkillMOO (Gong et al., 2026) evolves skill bundles using LLM-proposed edits and Pareto selection over task success and inference cost. Such a tradeoff can be expressed through the nondominated candidate collections

$$\mathfrak{P}_t = \mathrm{ParetoMax}_{\mathcal{K}\in\mathfrak{C}_t}\big(G_t(\mathcal{K}), -\mathrm{Cost}_t(\mathcal{K})\big), \tag{13}$$

where $\mathfrak{C}_t$ is the set of candidate collections and Cost_t measures their average execution cost under the common budget cap B. A collection is nondominated if no candidate is at least as good on both objectives and strictly better on one. This formulation allows several useful tradeoffs instead of imposing a single performance–cost weight. SkillMOO's analysis finds pruning and substitution prominent among successful edits, showing that improvement can come from reducing guidance. Specialization likewise needs evidence of future relevance. *Do Personalized Skills Help Coding Agents?* (Huang et al., 2026a) extracts developer preferences from interaction histories, but reports small, inconsistent improvements from personalization in simulator-based replay, while pooled generic skills yield more consistent gains.

Together, these studies suggest that optimization should match procedural specificity and cost to the intended reuse distribution $\mathcal{D}$, rather than assume that longer or more personalized skills are better.

Skill Governance. Governance determines which skills remain available as a collection grows. For an existing skill $k \in \mathcal{K}_t$, its removal contribution is

$$\gamma_t(k) = G_t(\mathcal{K}_t) - G_t(\mathcal{K}_t \setminus \{k\}). \tag{14}$$

A small or negative estimate of $\gamma_t(k)$ motivates review for redundancy or retirement. The score remains conditional on the other skills and the evaluation distribution; it does not establish that k is useless in every setting. Both $\gamma_t(k)$ and $\Delta_t^{\mathcal{K}}$ require estimates from comparable executions, making evaluation uncertainty part of governance. SkillsVote (Liu et al., 2026a) links collection and recommendation to post-execution attribution. It profiles environment requirements and verifiability, searches structured skill folders, and separates skill-guided subtasks from agent exploration and environmental effects before admitting reusable discoveries. Ratchet (Zhang et al., 2026j) focuses on retirement, combining measured contribution with a bounded active library and constrained synthesis. Its analysis distinguishes judge errors that increase sampling requirements from false-positive judgments that can disable retirement under the analyzed rule. A governance policy thus depends on the reliability of the evidence used to retain or remove skills.

Who Grades the Grader? (Zhang et al., 2026k) makes this dependence explicit by evolving inspectable evaluation functions alongside the skill library. Anchored reference judgments constrain the evaluator, while its agreement with hidden ground truth is checked separately. Removing these anchors can produce an always-pass evaluator even when downstream scores appear satisfactory. This extends the adoption decision for $\widetilde{\mathcal{A}}_{t+1}$ to the criterion governing adoption itself. Improving skills and validating their evaluator are related but distinct requirements, particularly when execution does not supply a complete oracle.

Skill Safety. Persistence can amplify errors as well as useful experience. *Library Drift* (Zhang et al., 2026i) diagnoses how unrestricted accumulation degrades retrieval and introduces irrelevant guidance. Its contribution logs, outcome-based retirement, and active-library cap make degradation observable and governable. *When Self-Evolution Backfires* (Shang et al., 2026) identifies a further propagation mechanism. Defective skills can influence the distillation of later skills, so removing the original entry leaves contaminated descendants. Its Verifier-as-Gatekeeper screens structural validity, behavioral harmlessness, and semantic consistency, then selects skill subsets by marginal gain before admission. Control must therefore cover individual skills and their interactions within the collection. An illustrative admission rule for a candidate k combines estimated benefit with specified checks,

$$\mathrm{Admit}_t(k) = \mathbf{1}\left[\widehat{\Delta}_t^{\mathcal{K}}(k \mid \mathcal{K}_t) > \epsilon\right] \mathbf{1}[\mathrm{Check}_t(k, \mathcal{K}_t) = 1], \tag{15}$$

where $\mathbf{1}[\cdot]$ is an indicator, $\epsilon \geq 0$ a gain threshold, and Check_t represents the chosen validity, compatibility, and safety checks. The hat marks an empirical gain estimate. For a replacement, the gain compares the new and old versions against the same base collection. This rule expresses a conservative admission policy; passing finite checks remains evidence under those checks, rather than a guarantee of safety in all future uses.

EVOMAL (Wu et al., 2026) demonstrates an adversarial counterpart in coding agents. A retrieved malicious skill can become an authoring template, causing harmful content to be copied into newly stored and executed skills. Agent-authored descendants can persist after the planted entries are removed; the proposed counter-prompt reduces copying-based self-poisoning in the evaluated setting. These findings make provenance and admission consequential throughout evolution. In terms of the five links introduced earlier, a contaminated update can survive retention, influence subsequent behavior, and generate the evidence for further updates. Safe skill evolution must therefore assess both immediate behavior and the content that a skill helps propagate into future versions of $\mathcal{K}_t$.

4.3 Tool Self-Evolution

Tool self-evolution updates the executable utilities in $\mathcal{T}_t$. Where a skill describes a procedure, a tool provides a callable implementation. The retained update is a utility or adapter that supports subsequent execution, rather than the project code delivered for the current task.

Live-SWE-agent interleaves issue resolution with tool creation and revision within a minimal bash-based agent (Xia et al., 2025). Reflection on environment feedback prompts the agent to build utilities for editing, code search, or specialized analysis, then invoke and refine them as the task proceeds. Its main reuse scope is later steps within the same issue, illustrating how execution experience can expand available operations without an offline evolution phase.

SIGA evolves a simulator-facing adapter around a fixed coding model and execution loop (Ho et al., 2026). An offline proposer uses prior trajectories and rewards to revise the task primer, procedural memory, and auxiliary skills, retaining candidates selected by validation performance. These updates span several assets. Its main evolved comparison disables skill invocation to keep the runtime tool configuration fixed, so the gains support improved adapter contents rather than newly added tools. Together, the two methods distinguish expanding executable operations from improving their domain interface. In either case, a working utility alone does not establish a positive downstream gain ΔJ_t.

4.4 Context Self-Evolution

Context self-evolution targets C_t, the information, instructions, and environmental settings through which an agent encounters a coding task. The reusable change may be selected repository evidence, a compression rule, an optimized prompt, or a repository-side information structure. These changes shape what reaches the decision process and how it is presented. The central distinction is between constructing context for one invocation and retaining an experience-informed change that influences subsequent execution. The methods below occupy different points along this boundary and support different scopes of reuse.

Context Retrieval. Retrieval evolves when execution changes the evidence available to later queries or the way relevant evidence is assembled. EVOR synchronously updates queries and a heterogeneous knowledge base containing documentation, web content, code snippets, and execution feedback (Su et al., 2024). Generated programs or their error messages enter the retrieval pool, so later attempts can condition on evidence produced by earlier execution. Its loop retains more than a revised answer, although successful execution of a retrieved snippet does not establish its semantic correctness. CodeMEM addresses continuity across iterative repository-level generation (Wang et al., 2026c). It uses AST structure to preserve relevant code context, incorporates newly retrieved blocks when needed, and maintains code-centric session memory to detect forgotten corrections. Its context-retrieval role overlaps with memory because retention supplies the evidence from which later context is assembled. These methods illustrate reuse within an evolving interaction; their reported gains should not automatically be interpreted as transfer to unrelated repositories. To distinguish retained assets from their use at a decision point, let $\mathcal{B}_t$ be the available evidence pool, $p_t(x)$ the instructions for state x, and ϕ_t the selection or compression mechanism. These are components of the retained context assets C_t. The assembled context can be written as

$$\begin{aligned} c_t(x) = \mathrm{Assemble}(p_t(x), \phi_t(\mathrm{Retrieve}(x, \mathcal{B}_t); x)), \\ |c_t(x)|_{\mathrm{tok}} \le L. \end{aligned} \tag{16}$$

The lowercase $c_t(x)$ denotes one constructed context and L its token budget. Updating $\mathcal{B}_t$, p_t, or ϕ_t changes what later constructions can draw on. A different $c_t(x)$ produced from unchanged assets can instead reflect a different query or task state. This distinction locates the retained update underlying a context-adaptation claim.

Context Compression. Compression determines which observations remain visible when terminal output or repository content exceeds the useful context budget. TACO discovers and refines structured filtering rules from terminal trajectories, pairing applicability conditions with retention and removal patterns (Ren et al., 2026a). Effective rules are written back to a global pool and selected for later tasks, making the compression behavior itself reusable. SWE-Pruner instead conditions a trained neural skimmer on an explicit goal supplied by the coding agent and selects relevant source lines (Wang et al., 2026i). It demonstrates task-adaptive context selection, but adaptation of the selected lines does not by itself establish continual updating of the skimmer from deployment experience. This distinction separates TACO's retained rule evolution from the application of a learned compression component. For an original context $c_t(x)$ with nonzero token length and its compressed version $c'_t(x)$, the fractional token reduction is

$$\rho_t(x) = 1 - \frac{|c'_t(x)|_{\mathrm{tok}}}{|c_t(x)|_{\mathrm{tok}}}. \tag{17}$$

A positive $\rho_t(x)$ indicates token savings, but can coexist with $\Delta J_t < 0$ if compression removes information needed for correct action. Comparisons should therefore consider both token reduction and downstream performance under a common budget. The relevant tradeoff depends on which observations are removed, as well as their volume.

Prompt Evolution. Prompt updates change the instructions conditioning a frozen code generator. For a problem prompt p_q, writing $p'_q = r_t(p_q)$ separates the revised prompt instance p'_q from the transformation rule r_t. The instance may be reused in another attempt at q, whereas r_t can be applied to prompts for other problems if its conditions hold. EPiC

evolves variants of a problem prompt, using the test performance of generated code to guide selection (Taherkhani et al., 2024). It illustrates feedback-driven prompt search within a problem, without requiring a durable instruction shared across problems. Reinforcement-learning-based prompt optimization learns a PPO policy that chooses among direct generation, lexical mutation, and semantic rewriting using unit-test rewards (Esfahani et al., 2026). Here, the generator remains frozen, while the optimizer learns how to produce the context updates. SePO moves reuse to the system-prompt level, evolving both task-agent prompts and the prompt agent's own instructions through an archive of candidates (Tao et al., 2026). The reusable artifact thus includes guidance for performing further prompt optimization. From Failing to Passing evolves natural-language transformation rules and combines them with execution-feedback repair in DUALFIX (Akli et al., 2026). Its cross-problem reuse and cross-model transfer tests examine whether an effective transformation generalizes beyond the failures used to discover it. These approaches distinguish an improved prompt instance from a reusable prompt-transformation mechanism, an important difference when defining the evaluation distribution $\mathcal{D}$.

Environment Evolution. The agent's information surface can also change on the repository side. Libra introduces hierarchical Markdown catalogs and rewrites them after a frozen solver fails to localize information in response to synthetic queries (Zhao et al., 2026). The catalogs persist as navigable indices for later queries and solvers. Its transfer experiments across models and problem sets test whether the improved environment remains useful beyond the optimization interactions. EvoEnv explores a different environmental target, synthesizing reusable Python environments that generate reasoning instances, compute references, and score answers (Shi et al., 2026b). Candidates enter an active training pool after validity, semantic consistency, difficulty, and novelty checks, and the shared generator–solver policy is updated through reinforcement learning. This is environment–model co-evolution for reasoning training, rather than repository configuration or direct evidence of improved software issue resolution. The comparison identifies two distinct reuse channels. Libra changes what future solvers can navigate, whereas EvoEnv changes the executable sources of future training experience. Their common asset-level feature is a retained environment artifact, while the downstream learning mechanisms and the claims supported by evaluation differ. For a pool $\mathfrak{E}_t$ of retained task-generating environments, their effect on training can be represented by

$$\mathcal{D}_t^{\text{train}} = \sum_{e \in \mathfrak{E}_t} \alpha_t(e) \mathcal{D}_e, \tag{18}$$

where $\mathcal{D}_e$ is the task distribution generated by environment e, $\alpha_t(e) \geq 0$, and $\sum_e \alpha_t(e) = 1$. Revising environments or their sampling weights changes the experience available for subsequent learning. This abstraction applies to generative training environments such as EvoEnv; repository catalogs such as Libra instead change the evidence available to the solver on an existing task. Environment adaptation can thus influence either the information used during execution or the distribution from which future learning experience is obtained.

Across these four asset types, the common challenge is to preserve useful experience in a form that can influence the right later decision. Memory emphasizes the evidence retained, skills the procedures made reusable, tools the operations made executable, and context the information exposed during execution. An asset may be present while $\delta_t(x) = 0$ at a given state. Even when $\delta_t(x) > 0$, improvement requires separate evidence that $\Delta J_t > 0$ over the intended scope of reuse. Comparisons should therefore isolate the contribution of retained assets from additional search, training, or feedback, and account for their maintenance costs and failure modes.

5 Architecture Self-Evolution

Architecture self-evolution incorporates coding experience into the mechanisms that organize execution. Within $\mathcal{H}_t = (h_t, w_t, g_t)$, an update may change runtime control in the harness, dependencies among workflow stages, or the allocation of responsibilities and information among agents. These targets offer different ways to address the same failure. A missed validation step may motivate a submission check in the harness, an earlier testing stage in the workflow, or a revised handoff to a reviewing agent. The resulting improvement can come from arranging existing capabilities more effectively, even when the available tools and model parameters remain unchanged. The distinction concerns the functional role of the update, since a workflow or collaboration change may also require harness code. Following Figure 3, we examine how experience becomes a retained execution rule, task-solving structure, or coordination mechanism.

5.1 Harness Evolution

Harness evolution updates the executable layer h_t that mediates model interaction, tool execution, feedback handling, and termination. In practice, harness optimization may also revise prompts, memory, skills, and tools in $\mathcal{S}_t$. We classify changes to reusable content or available operations as asset updates, and changes to execution organization and control as architecture updates. A method can combine both, so gains from an evolved harness should be interpreted in light of the components actually modified.

Updating execution control gives experience a direct route into agent behavior. A debugging lesson stored as a skill still depends on retrieval and the agent's decision to follow it; a harness intervention can check its activating condition whenever execution reaches the relevant point. Its influence therefore depends on where it is installed, which states activate it, and what it changes when activated. Its key opportunity is to convert a recurring failure into an intervention at the point where that failure arises. For example, a repeated invalid tool call may motivate argument validation before execution, while an unproductive retry loop may motivate a recovery rule after feedback. To make this intervention explicit, let z be the runtime state, including execution history and remaining budget, and let b be a control decision such as dispatching a call, retrying, or terminating. A harness induces a transition kernel

$$(b, z') \sim P_{h_t}(\cdot \mid z; \mathcal{S}_t, \theta_t), \qquad \widetilde{h}_{t+1} = \mathrm{Apply}(h_t, e_t), \tag{19}$$

where e_t is a proposed code or configuration edit and z' is the resulting runtime state. This description locates the update in the rules governing execution, without prescribing how a method must search for edits. The six mechanisms below emphasize different aspects of that search and can coexist within one system.

Archive-Based Evolution. An archive makes earlier executable agents available as starting points for further improvement. This matters because a modification can have value beyond its immediate task score. A branch that currently performs poorly may contain a useful implementation choice, expose a recurring failure, or support a later change that would be difficult to discover from the current best agent. DGM operationalizes this idea by sampling agents from an expanding archive, modifying their source code, and evaluating the resulting variants on coding tasks (Zhang et al., 2025c). Retaining alternative branches allows search to continue from earlier designs instead of requiring every useful path to pass through the latest agent. The archive retains executable implementations alongside their traces, evaluation records, and lineage information. Search uses this history both to select a parent and to construct a candidate modification. Retention for further search and adoption for task execution are separate decisions. An archive can preserve an informative candidate without making it the deployed agent.

Choosing a parent raises a different question from choosing an agent to solve the next task. Huxley GM identifies a mismatch between current coding performance and the ability to produce useful descendants, and uses estimated clade-level metaproductivity to guide tree search (Wang et al., 2025a). To express this distinction independently of a particular estimator, let $u(\mathcal{A}) = \mathbb{E}_{q\sim\mathcal{D}}[s(\mathcal{A}, q; B)]$ measure current performance. Under a specified search protocol and current archive $\mathcal{Z}_t$, let $\mathrm{Desc}_C(\mathcal{A})$ be the random set of descendants generated from $\mathcal{A}$ with evolution budget C. A conceptual measure of its value as a starting point is

$$\Psi_t(\mathcal{A}; C) = \mathbb{E}\left[\max_{\mathcal{A}' \in \{\mathcal{A}\}\cup \mathrm{Desc}_C(\mathcal{A})} u(\mathcal{A}') \;\middle|\; \mathcal{Z}_t\right]. \tag{20}$$

The evolution budget C pays for candidate construction and assessment, whereas B controls each agent's task execution. A high $u(\mathcal{A})$ need not imply a high $\Psi_t(\mathcal{A}; C)$. The latter depends on which descendants the search can reach and afford to evaluate. It is an idealized quantity; observed descendant results provide evidence for estimating such potential. This distinction explains why selecting only the current highest-scoring parent can discard useful routes to improvement.

Mendel GM focuses on what evidence should inform the next modification. Its reaction-norm mutation compares one agent's trajectories across tasks, while cross-lineage hybridization uses another lineage's trajectory on the same task (Anonymous, 2026). These comparisons reveal different information. Repeated failures across tasks can suggest a recurring weakness in the harness; different outcomes on a shared task can suggest a strategy worth transferring. Writing $\mathcal{E}_{i,q}$ for an agent–task execution record and Ω_t for the observed agent–task pairs, the two evidence views are

$$\mathcal{E}_t^{\mathrm{tasks}}(i) = \{\mathcal{E}_{i,q} : (i,q) \in \Omega_t\}, \qquad \mathcal{E}_t^{\mathrm{agents}}(q) = \{\mathcal{E}_{j,q} : (j,q) \in \Omega_t\}. \tag{21}$$

The first holds the agent fixed, and the second holds the task fixed. Neither requires every agent to run on every task. These views turn the archive into a source of comparative diagnoses as well as candidate implementations. Executing the modified agent under matched conditions tests these proposed explanations.

The archive also creates an allocation problem because evaluating every possible modification can consume the evolution budget before useful branches are explored. SIFT uses an LLM judge to rank candidate patches and concentrates expensive benchmark execution on promising candidates (Fu et al., 2026b). By reducing full evaluations, screening can make a wider search affordable, but its usefulness depends on which candidates it excludes. A judge that repeatedly favors familiar edits can suppress the unusual branches that archive-based exploration is intended to retain. The relevant comparison therefore concerns the quality of agents discovered under a common evolution budget, alongside the cost and reliability of candidate screening.

The proposal and evaluation mechanisms can themselves evolve. HyperAgents places the task agent and its modifying meta agent in one editable program, allowing experience to revise both task execution and the procedure producing future updates (Zhang et al., 2026c). By modifying the proposal mechanism itself, improvement can change which modifications become reachable. RQGM also permits evaluator changes between epochs while keeping the utility fixed within each epoch (Iacob et al., 2026). If the internal objective changes from u_t to u_{t+1}, a higher reported score can reflect both a different agent and a different criterion. Comparison under the common $\mathcal{D}$, s, and B remains necessary to interpret the result as ΔJ_t. Across these methods, the archive's contribution depends on how retained variants change parent selection, comparative diagnosis, proposal generation, or allocation of evaluation effort. Its size alone does not capture that contribution.

Iteration-Based Evolution. Iterative methods make the agent's implementation a recurring coding task. SICA uses reflection and editing tools to revise the program orchestrating its model calls and actions, retaining code changes for later executions (Robeyns et al., 2025). STOP makes this relationship explicit through a seed program improver that applies its own language-model-assisted search to itself, changing how subsequent programs are generated and selected while leaving model weights fixed (Zelikman et al., 2024). Ouroboros extends implementation revision into continued operation. Explicit improvement tasks and problems encountered during ordinary work produce reviewed commits that become the runtime for later work (Razzhigaev et al., 2026). Its frozen benchmark snapshots and live deployment lineage provide different evidence, respectively about a particular system version and the operation of a continuing update process. The defining transition is adoption into later execution; repeatedly editing candidate code does not establish that this transition occurred.

Iterative revision also couples the quality of the current agent to the quality of its future edits. A change that improves repository inspection may help both ordinary issue resolution and inspection of the agent's own implementation. However, these two uses expose different failure cases. Evaluating later self-modification attempts can reveal whether an update improves the revision process as well as the task solver. Retaining versioned implementations makes this distinction inspectable and supports returning to an earlier revision when a later change degrades execution.

Trace-Driven Evolution. Execution traces can reveal which runtime mechanism prevented an agent from using its available capabilities effectively. Self-Harness clusters recurring failures, proposes small changes tied to those failures, and retains modifications through regression testing (Zhang et al., 2026b). AHE makes editable components observable at file level, condenses trajectories into inspectable evidence, and pairs edits with predictions that can be checked against subsequent outcomes (Lin et al., 2026a). HarnessFix provides more explicit localization by aligning runtime data and control flow with harness artifacts, then converting recurring diagnoses into scoped repair specifications (Chen et al., 2026a). AutoSaddler combines mini-batch diagnosis and structured patching with development-set selection and separate final testing, distinguishing repair of observed failures from evidence that an update generalizes (Park et al., 2026). ReCreate uses experience retrieval and a reasoning–creation process to generate scaffold edits, abstracting individual interactions into reusable domain patterns (Hao et al., 2026a). Across these methods, the important conversion is from an observed failure to an actionable hypothesis about the execution mechanism responsible for it.

A diagnosis becomes operational only when it specifies where and under what conditions execution should change. AHE illustrates this with an intervention that records protected output paths after successful validation and intercepts later commands that would delete or reset those outputs (Lin et al., 2026a). HarnessFix retains both applicability and non-applicability conditions with repair outcomes, helping later edits respect the scope of earlier experience (Chen et al., 2026a). To describe this class of local change, let z include the runtime state and pending decision at a chosen intervention point. An intended guarded transformation can be written as

$$I_e(z) = \begin{cases} R_e(z), & g_e(z) = 1, \\ z, & g_e(z) = 0, \end{cases} \tag{22}$$

where g_e is the activating condition and R_e applies the intended correction before execution continues. This abstraction

makes applicability part of the update rather than an assumption that every failure-derived rule should apply globally. It also exposes two failure modes. A correct repair can be ineffective because its condition rarely activates, or harmful because that condition captures states where the original behavior was appropriate. Self-Harness addresses a related decision at proposal time by selecting evidence-supported mechanisms that its editable surfaces can address, then testing candidates for regressions (Zhang et al., 2026b). Proposal eligibility, runtime activation, and candidate admission thus govern different parts of the improvement process.

The resulting updates need not have identical scope. One Recipe, Many Harnesses finds that a shared evolution procedure yields both transferable strategies and adaptations specific to language ecosystems and model weaknesses (Yang et al., 2026b). Evo-Harness compiles single-shot executions into general and topic-specific skill guidance that is retrieved for later tasks (Wei et al., 2026a). Although described as harness learning, its retained Markdown skills primarily update procedural assets under our functional distinction. Harness-of-Harness illustrates a further boundary by organizing existing harnesses into repeated planning, implementation, and independent testing, retaining project histories to inform subsequent development (Yan et al., 2026). Its demonstrated progression concerns the software project and accumulated project state; this does not by itself establish persistent revision of the underlying execution architecture. These cases make the update target essential to interpretation, even when methods share the language of harness improvement.

Population-Based Search. Maintaining several candidates allows discoveries from different workloads or search paths to inform subsequent harnesses. DarwinX uses a frozen model and selects harness variants under a preserve-and-extend criterion, retaining alternative lineages for recombination (Zhang et al., 2026l). HarnessBank organizes successful harnesses by semantic characteristics in a gene bank, then recombines and screens offspring through staged verification (Luo et al., 2026). EvolveNet distributes a shared harness to deployments with local workloads and combines the resulting program adaptations using their scopes and supporting evidence, without centralizing the raw workloads (Nie et al., 2026a). HarnessCompass instead emphasizes task-agnostic edit constraints, feedback from the executing agent, and separate component optimization before consolidation (Zhang et al., 2026e). ADAS provides an earlier code-based search formulation in which a meta-agent programs new agent designs using an expanding archive of prior discoveries (Hu et al., 2024a). These methods overlap with archive-based evolution, emphasizing the selection and combination of useful changes across candidates.

Combining useful changes introduces a further problem of compatibility. Two edits that each help their original parent may conflict when placed in one harness, for example if they change the same retry rule or assume different representations of execution state. Population diversity is valuable when it preserves complementary solutions and the evidence needed to reconcile them. The unit of inheritance therefore matters, whether a method transfers a complete harness, a component implementation, or a scoped program change. Evaluating these units only in isolation can miss failures created by their composition.

Pipeline-Driven Evolution. An explicit development pipeline separates proposing a modification from releasing the resulting agent. Meta-Harness gives its proposer filesystem access to earlier candidate code, scores, and execution traces, preserving detailed evidence for subsequent harness edits (Lee et al., 2026). AgentDevel maintains a canonical release line, translating observed failure symptoms into executable diagnoses and assessing candidates through newly solved cases and regressions (Zhang, 2026). CCA uses a meta-agent to construct, test, and refine agents on a modular SDK, complementing its runtime orchestration and persistent notes (Wong et al., 2025). HarnessX composes typed harness primitives and evolves them from traces; its broader framework also turns trajectories into model-training signals, connecting architecture and model updates (Chen et al., 2026d). Autogenesis separates versioned resources from the protocol for proposing, assessing, and committing changes, with lineage and rollback spanning several component types (Zhang et al., 2026h). SEA places admission gates with anytime-valid certificates around a versioned harness and steering adapter, with guarantees tied to the gate's statistical assumptions and error budget (Sengupta, 2026). Across these systems, the pipeline determines which evidence is available when a candidate becomes persistent.

A release pipeline also preserves the link between a modification and the evidence used to accept it. Retaining a candidate's source version, motivating traces, predicted effects, and evaluation conditions makes later regressions easier to diagnose. This becomes especially useful when improvements accumulate across rounds, because a failure may result from an interaction between accepted edits. Tests of whether the modified harness runs successfully and measurements of whether it improves downstream decisions answer different questions. A release decision benefits from both, together with evidence about which earlier successes were lost.

Regression-sensitive admission can be expressed using a fixed, nonempty validation set $\mathcal{V}$. Let $y(h, q) \in \{0, 1\}$ record

whether the agent with harness h solves case q under matched evaluation conditions. For a candidate $\widetilde{h}$, the fractions of newly solved and newly failed cases are

$$F^+(\widetilde{h}, h) = \frac{1}{|\mathcal{V}|} \sum_{q \in \mathcal{V}} \mathbf{1}[y(h,q) = 0,\ y(\widetilde{h},q) = 1], \qquad F^-(\widetilde{h}, h) = \frac{1}{|\mathcal{V}|} \sum_{q \in \mathcal{V}} \mathbf{1}[y(h,q) = 1,\ y(\widetilde{h},q) = 0]. \tag{23}$$

The empirical success-rate change is $F^+ - F^-$. Reporting both terms reveals whether an aggregate gain hides losses on previously solved cases. These are finite-set diagnostics; stochastic outcomes require repeated evaluation, and passing an admission gate does not establish benefit beyond its validation scope.

Runtime Evolution. TTHE updates harnesses during evaluation using unlabeled execution traces, maintaining candidates and committing a program through execution-derived proxy signals so that it governs subsequent inputs (Nie et al., 2026b). Argus retains verified runtime state, including reusable procedures and routing decisions, across bounded missions with a fixed model; its longitudinal evidence is observational rather than a controlled learning ablation (Li et al., 2026a). Life-Harness offers a complementary runtime interface design. It converts training-trajectory failures into interventions for interface contracts, procedures, action validation, and recovery, then keeps the resulting harness fixed on unseen tasks (Xu et al., 2026). Its evaluation concerns deterministic interactive environments beyond coding. These cases separate when an intervention operates from when it is learned. A runtime check can be the product of offline evolution, while online evolution requires an update that persists beyond the execution that generated it.

Online updating further couples learning to the changing distribution of experience. An adopted retry or termination rule changes which states later executions visit, and therefore which failures the next update can observe. The absence of an earlier error may indicate a successful repair, or simply that the revised harness no longer reaches the relevant situation. Runtime evaluation should therefore track both task outcomes and the circumstances in which interventions activate. In terms of the earlier five links, harness evolution must connect a diagnosed failure to an executable change, its actual activation, and evidence of benefit over the intended reuse scope.

5.2 Workflow Evolution

Workflow evolution updates w_t, the arrangement of task-solving stages and their dependencies. Its value lies in changing when evidence becomes available and which decisions it can influence. Moving validation before an expensive rewrite can expose a mistaken assumption before it propagates into code; routing a failing test back to localization can reopen a decision that a fixed repair loop would leave unchanged. Such changes can improve the use of existing capabilities. We describe a workflow as $w_t = (V_t^w, E_t^w, \ell_t)$, where V_t^w contains stages, E_t^w describes control and information dependencies, and ℓ_t assigns instructions and operations. A proposed structural or configuration edit η_t produces

$$\widetilde{w}_{t+1} = \mathrm{Edit}(w_t; \eta_t). \tag{24}$$

Edits can add or remove stages, redirect dependencies, or change branch and loop conditions. A workflow can therefore evolve by removing redundant work or improving feedback routing as well as by adding new stages.

Structure Search and Diagnosis. AFlow searches code-represented workflows using Monte Carlo Tree Search, reusable operators, execution feedback, and experience from previous modifications (Zhang et al., 2024a). Its search explores alternative arrangements of operations, making the task-solving procedure itself a candidate for evaluation. SEW separately evolves workflow structure and agent prompts, including mutation prompts, and studies how textual representations affect workflow validity (Liu et al., 2025). These methods highlight two coupled choices, the structure to execute and the representation through which that structure can be modified. A search space must make useful changes expressible while preserving executable dependencies. JUDGEFLOW addresses where to edit. Its judge inspects traces, ranks the responsibility of logic blocks, and directs the optimizer toward problematic blocks (Ma et al., 2026). This is useful because the stage reporting a failure may have inherited an earlier mistake. For example, a debugging stage may repeatedly revise a patch when the underlying problem is incorrect file localization. Trace-based diagnosis can suggest changing the upstream stage or the feedback path. The diagnosis guides a proposal; executing the revised workflow tests whether that proposal improves the overall procedure.

Diversity and Reusable Workflow Components. EvoFlow maintains heterogeneous workflows through tag-based parent retrieval, crossover, mutation, and niching selection (Zhang et al., 2025b). Retaining different structures allows workflow choice to reflect task complexity and inference cost. A short synthesis task may benefit little from a review loop that is useful for a difficult repair, so one aggregate score can conceal which tasks benefit from which structures.

EvoAgentX provides a modular platform for generating, executing, evaluating, and optimizing workflows, integrating TextGrad, AFlow, and MIPRO to support changes to prompts, tool configurations, and topology (Wang et al., 2025c). Its scope shows why improvements to a workflow system may involve several component types. FlowEvo makes the interaction with assets explicit by compiling successful workflows into executable skills, retaining them in a persistent bank, and retrieving them for direct execution or later workflow construction (Ren et al., 2026b). It also tracks downstream utility to suppress skills associated with negative transfer. A discovered procedure can thus move between w_t and the skill state $\mathcal{K}_t$. Retained skills expand the building blocks available to later workflow search, while new workflows provide settings in which those skills are exercised and reassessed. This creates a feedback loop between learning reusable procedures and discovering how to organize them.

Selecting an existing branch exercises a workflow; retaining an experience-derived revision constitutes an update. To examine a stage's contribution, let $\mathcal{A}^w$ use workflow w and $w \setminus v$ be a valid bypass of stage v, with the other agent components fixed. Under matched evaluation conditions,

$$\Gamma(v \mid w) = \mathbb{E}_{q\sim\mathcal{D}}\left[s(\mathcal{A}^w, q; B) - s(\mathcal{A}^{w\setminus v}, q; B)\right]. \tag{25}$$

This contribution depends on the surrounding workflow and bypass design. A stage may help because it supplies evidence that another stage needs; its effect cannot be inferred from its own output quality alone. A small average contribution may also hide benefits on rare failures or redundancy with another stage. The common budget B constrains extra computation, while recording actual cost and branch activation helps explain how an evolved workflow achieves its result. The central evaluation question is whether reorganizing execution makes useful evidence influence decisions at the right time and at an acceptable cost.

5.3 Multi-Agent Evolution

Multi-agent evolution concerns g_t, including the participating agents, their responsibilities, and the mechanisms through which they exchange and act on information. While a workflow describes dependencies among stages, a team description also identifies who performs them and how participants coordinate. A useful abstraction is

$$g_t = (V_t^g, E_t^g, \rho_t^g, p_t^g), \tag{26}$$

where V_t^g is the participant set, E_t^g contains communication relations, ρ_t^g assigns responsibilities, and p_t^g specifies interaction and decision rules. The latter matters because the same communication graph can support different handoff requirements, review responsibilities, or disagreement-resolution procedures. For example, a reviewer may merely suggest changes or require unresolved test failures to be addressed before a patch is accepted. These arrangements give the same message different consequences. Team evolution therefore concerns how distributed evidence becomes a shared decision, including who can challenge an assumption and who must act on the result.

Feedback-Driven Team Revision. EvoMAC obtains feedback from a testing team and uses textual backpropagation to revise the coding team's agent instructions and connections (Hu et al., 2024b). These updates are exercised in subsequent rounds of the same software-development task. Meta-Team, introduced in *Evolve as a Team*, preserves individual execution contexts and enables post-task discussion to combine evidence distributed across participants (Hao et al., 2026b). It updates individual scaffolds, teammate profiles, and team-level composition and rules, committing validated changes to a reusable team scaffold. Their comparison highlights two distinct reuse scopes, adaptation within a task and retained changes for later tasks. They also expose a difficulty in assigning responsibility for team failures. The last agent to produce an incorrect answer may have acted on an incomplete handoff, while the relevant warning remained in another participant's context. For example, a tester may observe an environment mismatch that a developer interprets as a code defect. Improving the developer's instructions alone may leave this failure intact. Useful revision requires relating what each agent observed, what it communicated, and how the recipient used that information. This can reveal whether to improve an individual role, change a handoff, or require explicit resolution of conflicting evidence.

Adaptive Topology and Mixed Update Targets. AgentConductor trains an orchestrator to generate layered interaction graphs that reflect task difficulty and communication cost (Wang et al., 2026d). At inference, the orchestrator is frozen but can regenerate a topology after execution feedback. Its persistent learning resides in the orchestrator parameters, while its runtime effect is architectural adaptation. SEMAG uses hierarchical planning, coding, debugging, and discussion, together with an optional mechanism that selects updated backbone models using retrieved information and sampled performance (Peng et al., 2026a). Its execution-dependent escalation demonstrates adaptive coordination,

although traversing predefined stages alone does not establish a learned revision of coordination rules. SAGE provides another boundary case. Its Challenger, Planner, Solver, and Critic organize task generation, planning, solving, and quality filtering, while verifier-based training updates their shared model (Peng et al., 2026b). This supports collaborative model improvement rather than independently demonstrating topology evolution. Together, these cases distinguish the organization instantiated for a task from the persistent mechanism that produces or uses it. A different graph on the next task may reflect a learned coordination policy or a fixed routing rule responding to different inputs. Evaluation must identify which mechanism changed and whether its choices remain useful beyond the situations that generated the update. This distinction also connects architecture and model evolution without treating every dynamically assembled team as a new learning event.

Collaboration during Evolution. GEA shares experience among a group of candidate agents so that framework modifications and execution outcomes from one lineage can inform others (Weng et al., 2026). The resulting offspring retain their own revised implementations. Here, the group organizes the improvement process and need not become a team that jointly solves each downstream task. This distinction separates who collaborates to produce an update from what the deployed update changes. Sharing experience can broaden the evidence available to an offspring, but a useful change in one lineage may rely on roles or interfaces absent from another. The transferable unit therefore includes the conditions under which an edit worked, as well as the edit itself. Otherwise, combining successful local changes can introduce incompatible assumptions.

Across these settings, evidence of team improvement should identify the retained target and assess its contribution under the common execution budget B. Additional participants can provide independent checks or complementary evidence, but agreement can also arise from agents repeating the same unsupported premise. Message count and consensus alone therefore provide weak evidence of effective coordination. Comparisons should account for total model calls, communication, and latency, and examine whether revised roles or interaction rules change which errors are detected and corrected. A positive ΔJ_t establishes an overall gain under the specified conditions; attributing that gain to coordination requires comparisons that separate these revisions from added computation or simultaneous model and asset updates.

6 Model Self-Evolution

Model self-evolution incorporates coding experience into the parameters θ_t that shape subsequent agent decisions. Within $\mathcal{A}_t = (\mathcal{S}_t, \mathcal{H}_t, \theta_t)$, these updates can change how the model localizes a defect, proposes a repair, interprets feedback, or decides to verify its work. Parameter learning can make an acquired behavior available across tasks without requiring explicit retrieval of the originating experience. Its usefulness then depends on whether the model preserves the conditions under which that experience was informative. A successful repair can teach a transferable strategy, but training on an incidental feature of the same repair may produce an unreliable shortcut.

The following discussion distinguishes how training experience is generated, how feedback becomes an optimization signal, and which model role is updated. Self-play co-evolution changes the tasks or evaluators that supply learning signals; environment-reward learning grounds updates in task outcomes; harness co-evolution connects parameter learning with changes to the surrounding execution system. These mechanisms can overlap. We retain the earlier distinction between a candidate update and its adoption, and use ΔJ_t to assess subsequent benefit. The formulas below describe shared mechanisms or explicit comparison protocols; individual methods instantiate different data, losses, and update schedules.

6.1 Self-Play Co-Evolution

Self-play makes the current system a source of its own future training experience. An evolving model may generate tasks, solve them, construct tests, or judge alternative outputs. Consequently, parameter updates also change the distribution of experience available in the next round. Let Q_t denote the training-task distribution. In a batch of n_t records, each (q_i, τ_i, f_i) contains a task, an execution trace, and its feedback. The records may include several model roles and unsuccessful attempts. A common data-construction abstraction is

$$\mathcal{D}_t^{\text{train}} = \text{Curate}_t\left(\{(q_i, \tau_i, f_i)\}_{i=1}^{n_t}\right). \tag{27}$$

Here q_i is sampled from Q_t, which may combine generated and externally supplied tasks. Curation determines whether a record supplies a demonstration, a preference, a reward, or privileged information for a teacher. It can relabel

or reweight experience as well as reject it. The learning signal therefore depends on both what the current system encounters and how those encounters are interpreted.

Task Generation and Curriculum. Self-Play SWE-RL uses a shared model for bug injection and repair, combining consistency checks and difficulty-sensitive rewards with joint reinforcement learning (Wei et al., 2026b). Failed repairs can contribute further bug states, connecting the solver's current limitations to later training tasks. Absolute Zero similarly trains a model to propose and solve executable reasoning tasks, using execution to establish task outcomes and solver success rates to guide task learnability (Zhao et al., 2025). These mechanisms make task difficulty responsive to the learner. However, difficulty alone does not establish relevance to the intended deployment distribution. Anchored Self-Play addresses this gap in code repair by mixing reference bugs into training and shaping generated edits through similarity to reference changes (Choi et al., 2026). This comparison separates three curriculum requirements, whether tasks are valid, whether the model can learn from them, and whether they exercise the capabilities needed later. Improving one does not ensure the others. Analysis of learnable information gain also links sustained self-play improvement to the information supplied by the synthetic pipeline rather than data volume (Liu et al., 2026c).

Coupled Code and Test Learning. ReVeal trains one policy for code generation and self-verification, filtering generated tests against a reference solution during training (Jin et al., 2025). CURE also uses a shared model in coder and tester roles, with reference unit tests helping establish code quality and guide tester learning (Wang et al., 2025d). These reference signals anchor different parts of the loop. A test that accepts a correct program need not distinguish that program from plausible incorrect alternatives. For a task q, let $\mathcal{Z}_t^-(q)$ be a nonempty set of programs independently established as incorrect, and let v be a test checked against the task specification. Its empirical rejection rate is

$$d_t(v;q) = \frac{1}{|\mathcal{Z}_t^-(q)|} \sum_{z \in \mathcal{Z}_t^-(q)} \mathbf{1}[v \text{ rejects } z]. \tag{28}$$

This diagnostic separates test validity from its ability to expose known errors. It does not measure coverage of unobserved failures. When no independent labels are available, both program quality and test quality must instead be inferred from their interactions, making mutual agreement a less reliable anchor.

ZeroCoder studies this latter setting by estimating code and test quality from execution consistency without reference code or tests in its basic configuration (Fan et al., 2026). Its extensions use a small labeled set for supervision and, for its dynamic Bayesian selector, prior calibration. Sol-Ver trains solving and verification through supervised fine-tuning followed by preference optimization (Lin et al., 2025b). ACE uses a shared model as solver and adversary, alternating solver fine-tuning with adversarial preference learning (Huang et al., 2026c). Its generated inputs probe runtime failures, while reference tests remain part of code selection. The methods therefore differ in both the optimization procedure and the claim supported by execution. Normal termination, agreement with generated expectations, and agreement with reference tests are distinct signals. A co-evolving evaluator is useful when it reveals weaknesses that matter to the task; merely becoming more favorable toward the co-evolving solver can conceal those weaknesses.

Learning from Privileged Experience. OPD-Evolver couples rapid memory updates with slower parameter learning through on-policy self-distillation (Zhang et al., 2026a). The same evolver supplies a teacher view conditioned on hindsight information and a student view restricted to the information available when acting. Its parameter updates cover selection, execution, memory writing, and maintenance. Let x_j be the student's information at token position j, including its own generated prefix, and let u_j denote additional hindsight information. With frozen teacher parameters $\bar{\theta}_t$, the token-level distillation loss has the form

$$\mathcal{L}_t^{\text{distill}}(\theta) = \mathbb{E}_{(x_j,u_j)\sim\mathcal{P}_t} D_{\text{KL}}\big(\pi_{\bar{\theta}_t}(\cdot \mid x_j, u_j) \,\|\, \pi_\theta(\cdot \mid x_j)\big). \tag{29}$$

Here $\mathcal{P}_t$ is formed from student rollouts and their associated hindsight views; the teacher distribution receives no gradient. Asset and architecture conditioning is implicit in the two views. The teacher's advantage comes from additional information, without requiring a stronger external model. This is a boundary case within self-play co-evolution, where the roles are informationally asymmetric rather than adversarial. The update seeks to make lessons from later outcomes useful at earlier decision points. The privileged information must remain unavailable to the evaluated student if the experiment is to establish that this capability was learned.

6.2 Environment-Reward RL

Environment-grounded learning derives supervision from what generated programs or agent trajectories achieve. For a fixed training round, write $\mathcal{A}_\theta = (\mathcal{S}_t, \mathcal{H}_t, \theta)$ and let $R_t(q,\tau)$ be the scalar training return constructed from execution

feedback. An idealized policy objective is

$$\max_{\theta} \ \mathbb{E}_{q\sim\mathcal{Q}_t,\ \tau\sim\mathrm{Execute}(\mathcal{A}_\theta,q,\mathcal{E}_q)}[R_t(q,\tau)]. \tag{30}$$

The trajectory notation denotes the trace component of execution, with $\mathcal{E}_q$ the task environment. Holding assets and architecture fixed isolates parameter optimization within the round. Practical updates use finite samples and may include regularization. The training return R_t need not equal the evaluation score s, and $\mathcal{Q}_t$ need not equal the held-out distribution $\mathcal{D}$. These distinctions matter when a convenient reward captures only part of the intended coding capability.

Guided Exploration and Credit Assignment. Agent-RLVR addresses sparse success signals by having agents first attempt software tasks, then reattempt failures with guidance derived from plans or environment feedback (Da et al., 2025). Successful traces support an initial supervised stage; successful and unsuccessful attempts supply preferences for offline DPO. Guidance is used during training, while the evaluated agent acts without it. For a common conditioning input x, let y^+ and y^- denote preferred and dispreferred continuations in $\mathcal{D}_t^{\mathrm{pref}}$. A preference-learning objective is

$$\mathcal{L}_t^{\mathrm{pref}}(\theta) = -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}_t^{\mathrm{pref}}} \log \mathrm{sigmoid}\left[\beta \log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\mathrm{ref}}(y^+ \mid x)} - \beta \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\mathrm{ref}}(y^- \mid x)}\right]. \tag{31}$$

Here π_{ref} is a fixed reference policy and $\beta > 0$ controls the preference scale. Execution determines the ordering; it does not uniquely identify which intermediate decision caused the outcome. A passing trace may contain redundant searches or incidental edits. Pair construction and guidance therefore affect what the model learns from the same terminal test result. Agent-RLVR's reported experiment covers one training cycle, so its gains should not be interpreted as evidence of indefinitely repeated improvement.

Reward Semantics and Search Experience. SWE-RL uses issue descriptions, code context, and reference patches from software history, rewarding similarity between generated and reference patches with a penalty for invalid formatting (Wei et al., 2025b). It optimizes the model with GRPO. Its supervision is grounded in software evolution data, but the training reward is not a unit-test execution verdict. Similarity can provide an inexpensive signal while favoring the particular implementation represented by the reference patch. The method therefore illustrates why software provenance, executable verification, and autonomous task generation must be distinguished when comparing learning loops.

SOAR supplies a complementary boundary case under this feedback-based category (Pourcel et al., 2025). It alternates evolutionary program search with supervised learning from the resulting attempts, improving both program sampling and refinement. Hindsight relabeling converts an unsuccessful program into a solution for a different task consistent with its actual outputs. Thus, execution feedback can support parameter learning without serving as a direct policy-gradient reward. Relabeling preserves useful computation from failed search, but changes what the resulting training pair teaches. A program that failed the original specification is not thereby a successful solution to that specification. SOAR's inclusion highlights a shared feedback source across different optimization mechanisms.

Feedback Infrastructure and Transfer. SWE-Gym and R2E-Gym provide executable environments and training pipelines for agents and verifiers (Jain et al., 2025, Pan et al., 2025); SWE-RM learns execution-free feedback for candidate selection and policy improvement (Shum et al., 2025). Their contribution includes making informative experience or scalable evaluation available to a learning loop. The resulting model gains still depend on how that feedback is used. Learned rewards can reduce execution cost, but policy optimization may exploit their errors; executable tests provide concrete evidence, but cover only specified cases. Evaluation should therefore retain independent outcome checks and compare updated policies under the same budget B. Training and data-collection costs should be reported separately, especially when guidance or repeated search makes successful traces expensive.

6.3 Harness Co-Evolution

The harness h_t shapes both the model's current execution and the experience available for its next update. Changing action interfaces, feedback delivery, or recovery rules can make different trajectories reachable. Conversely, parameter learning can change which interventions remain useful. Harness co-evolution therefore concerns this feedback between learned behavior and execution control. It also distinguishes learning to solve tasks from learning to modify the system that solves them.

Learning the Editing Policy. Harness-R1 trains a dedicated harness engineer through supervised initialization and online GRPO, while freezing the target agent during engineer training (Shao et al., 2026). Failure traces condition

executable patches, and rerunning the target with a patch supplies the engineer's reward. To describe this separation, let ω denote editor parameters, b_t the failure evidence, and $\mathcal{A}_t^e$ the target with edit e installed. For an installable edit evaluated on a nonempty matched task batch $\mathcal{V}_t$, the empirical utility change can be written as

$$\widehat{\Delta}_t(e) = \frac{1}{|\mathcal{V}_t|} \sum_{q \in \mathcal{V}_t} [s(\mathcal{A}_t^e, q; B) - s(\mathcal{A}_t, q; B)]. \tag{32}$$

The batch includes previously successful cases so that regressions contribute to the signal. Define $r_t^{\text{edit}}(e)$ to equal this change for a completed valid evaluation and zero for invalid or incomplete candidates. The corresponding editor objective is

$$\max_{\omega} \ \mathbb{E}_{e \sim \pi_\omega(\cdot|b_t)}[r_t^{\text{edit}}(e)]. \tag{33}$$

The expectation also includes the training distribution of failure bundles. The updated parameters belong to the editor, while the target receives an architectural intervention. Harness-R1 evaluates this mechanism on WebShop, ALFWorld, and DBBench; its evidence concerns executable harness engineering across interactive domains, rather than a direct repository-repair benchmark. The learned capability is to produce useful edits from new failures, so transfer of the editor must be distinguished from reuse of one particular patch.

From Harness Search to Model Learning. HELIX proposes a build–update–rebuild process in which harness variants produce verified experience for model updating, followed by harness rebuilding for the changed model (Fan and Huang, 2026). It retains source-traceable execution records for related candidates, including successful repairs, regressions, and near misses, and materializes supervised, critic, filtering, and preference data. Such contrasts can preserve why two executions differed, instead of retaining only the best final patch. Its reported study demonstrates harness search and training-data construction, but does not train an updated model. It therefore supports the handoff from architecture evolution to parameter learning, while leaving the complete repeated cycle to be evaluated.

To assess a completed coupled update, the old and new model–harness combinations should be compared under matched conditions. For $a, b \in \{0, 1\}$, let j_{ab} be the expected score under $\mathcal{D}$ and budget B when model parameters θ_{t+a} are paired with harness h_{t+b}. All four combinations must have compatible interfaces. Assets, workflow, and team organization are held fixed for this comparison. The interaction contrast is

$$I_t^{\theta,h} = j_{11} - j_{10} - j_{01} + j_{00}. \tag{34}$$

A positive value means the combined change exceeds the sum of the isolated changes on the chosen score scale; it does not by itself imply positive total improvement, which requires $j_{11} > j_{00}$. The contrast is specific to the tested configurations, distribution, and budget. It makes a central question of co-evolution explicit, whether parameter learning and harness revision reinforce one another or merely contribute separate gains. Across all three model-evolution mechanisms, stronger training signals matter only when their effects survive adoption and improve coding behavior.

7 Coding Feedback and Evolution Dynamics

Coding environments expose unusually rich evidence about the consequences of agent actions. A single attempt can leave a patch, build status, test verdicts, an exception stack, a sequence of tool calls, resource measurements, and a developer correction. These observations make software engineering a natural setting for self-evolution because they can often be attached to inspectable artifacts and reproduced in an executable environment. Their precision does not remove the central difficulty. A test may establish that one patch satisfies one checked property while providing insufficient support for a memory, skill, or harness rule that will influence many later tasks. We therefore organize the discussion around four questions: why coding environments provide distinctive signals, what each signal reveals, how methods convert signals into persistent updates, and when those updates are made.

7.1 Why Coding Is a Natural Domain for Self-Evolution

Coding has played a formative role in agent research and has been one of the earliest and most sustained application frontiers for LLM-based agents. It supplies an instrumented environment, not only a task domain, in which a model can execute an action, observe its consequences, and revise its behavior. Codex and AlphaCode established execution-based evaluation early in the development of language models for code (Chen et al., 2021, Li et al., 2022). Repository-level

systems then extended this generate–execute–inspect loop to file navigation, command-line tools, testing, and multi-step repair (Jimenez et al., 2023, Wang et al., 2024b, Yang et al., 2024). Tool-mediated action, environment feedback, and verifier-guided revision were consequently developed or stress-tested early in coding settings. These patterns now reach other domains, where code serves as an operational medium for action, state, and verification (Ning et al., 2026). Coding is therefore a mature proving ground for agents, with both attempted changes and much of their evidence already represented in machine-readable form.

A coding task is embedded in a repository snapshot, dependency configuration, toolchain, and execution environment. One attempt can leave a patch, build status, named test failures, an exception stack, tool-call history, resource measurements, and review comments. We represent the evidence associated with one coding event as

$$\mathcal{F}_t = (y_t, d_t, \tau_t, r_t, q_t, h_t, m_t; e_t, p_t), \tag{35}$$

where y_t records execution outcomes, d_t diagnostic evidence, τ_t the coding trajectory, r_t repository context, q_t software-quality attributes, h_t human judgments, and m_t model judgments. The environment state e_t and provenance p_t bind those observations to a repository revision, dependency state, tool version, candidate change, and evaluator. This structure matters because an identical patch can produce a different result after a dependency update, and the same score has a different meaning when it comes from a regression suite, a model judge, or a developer review.

Three properties distinguish these signals as a basis for evolution.

- **Specificity.** Coding feedback often identifies a concrete violated condition. A compiler can report an error class and source location, a unit test can name the failed assertion and expose expected and observed behavior, and a static analyzer can identify a rule and code region. The signal can therefore constrain both *what* should change and *where* the agent should look. Executable benchmarks and training environments exploit this property to turn test outcomes into validation or learning signals (Chen et al., 2021, Da et al., 2025, Pan et al., 2025). Specificity is nevertheless local to the checked property. A passing suite is a partial oracle, and converting one failure into a general memory, skill, or harness rule still requires identifying the conditions under which that lesson applies.
- **Traceability.** Software work leaves a causal record that can be inspected after execution. Issues state the requested behavior, version control identifies the repository state, diffs record the proposed intervention, and trajectories connect searches, edits, commands, and responses. SWE-bench grounds evaluation in issue–repository–patch relations, while SWE-agent shows how an agent–computer interface makes repository actions and their feedback explicit (Jimenez et al., 2023, Yang et al., 2024). This lineage lets an evolution mechanism attach provenance to a retained update and assign credit more precisely than a final reward alone. It also exposes confounders. A successful patch may follow several irrelevant actions, whereas a failed attempt may contain a correct diagnosis followed by an incorrect edit.
- **Reproducibility.** A repository revision, environment image, command, and test suite can define a replayable experiment. The same candidate can be executed again, compared with its parent under matched conditions, and checked for regressions before adoption. SWE-Gym and R2E-Gym package coding tasks as executable environments, and SWE-bench-Live provides task-specific container images and repeated validation to improve reproducibility (Jain et al., 2025, Pan et al., 2025, Zhang et al., 2025d). Reproducibility here is conditional rather than absolute. Flaky tests, external services, dependency drift, hardware, and stochastic agent decisions can still change an outcome. Coding environments are distinctive because these conditions can often be recorded, replayed, and tested for instability.

Together, specificity, traceability, and reproducibility shorten the path from an observed failure to an addressable and testable candidate update. Other agent environments may provide one or two of these properties, but software engineering routinely combines all three through executable artifacts, version history, and automated checks. This combination explains why coding has been an important proving ground for self-correction and agent evolution. It does not, however, collapse task verification into evolution evaluation. Tests directly judge the current software artifact; only retained reuse and matched evaluation on later cases can show that the changed agent has improved.

7.2 What Coding Signals Reveal

The properties established above explain why software evidence is useful, but an evolution mechanism still depends on which evidence it actually consumes. We distinguish seven non-exclusive signal classes throughout this survey: **execution outcomes (E)**, **diagnostic evidence (D)**, **coding trajectories (T)**, **repository context (R)**, **quality attributes (Q)**, **human judgments (H)**, and **model judgments (M)**. The first five classes are grounded in software execution,

process, state, or properties. H and M instead record evaluative judgments: H reflects developer or organizational assessment, whereas M reflects a learned evaluator whose reliability depends on both the evaluator and its input evidence. The codes describe evidence used to construct, revise, or select a persistent update. They do not describe the benchmark on which the resulting agent is later evaluated. A method may receive several codes because one coding event can expose several views of the same outcome. For example, a failed integration test supplies an execution outcome (E), its stack trace supplies diagnostic evidence (D), the preceding tool calls form a trajectory (T), and the repository revision defines the conditions under which the evidence is valid (R).

Table 4: Seven non-exclusive signal classes used by coding-agent evolution mechanisms. The codes identify evidence consumed during updating rather than the benchmark used for final evaluation.

Signal class	Observable evidence	Question answered	Evolutionary role and validity boundary
E Execution outcomes	Unit and integration tests, builds, executions, patch verifiers	Did the artifact satisfy an executable check?	Validate or select candidates within observed test coverage.
D Diagnostic evidence	Compiler messages, test logs, exceptions, build and tool failures	Where and how did the failure appear?	Locate update targets; symptoms may not identify their causes.
T Coding trajectories	Searches, inspections, edits, tests, retries, and recovery actions	How was the result produced?	Extract procedures; final success provides incomplete credit.
R Repository context	Issues, commits, pull requests, diffs, interfaces, dependencies, revisions	When is reuse valid?	Ground retrieval and reuse despite repository and dependency drift.
Q Quality attributes	Efficiency, security, maintainability, and vulnerability repair	Which quality attributes improved?	Assess non-functional gains and competing objectives.
H Human judgments	Code review, edits, acceptance, rejection, corrections, preferences	What do developers or organizations value?	Convey intent and authority under sparse, subjective review.
M Model judgments	LLM critiques, rankings, credit assignments, learned rewards	How does an evaluator assess the evidence?	Scale semantic evaluation, subject to evaluator reliability.

Execution Outcomes (E). Execution outcomes record whether a software artifact satisfies an operational condition. They include unit and integration tests, compilation, interpretation, patch verifiers, task-resolution checks, and measured execution outcomes. DGM, Mendel GM, and Huxley GM use verified coding performance to select agent variants across generations, while SICA retains source-level self-modifications according to coding-task results (Anonymous, 2026, Robeyns et al., 2025, Wang et al., 2025a, Zhang et al., 2025c). AFlow uses execution outcomes to select workflows, and Agent-RLVR turns unit-test results into rewards for policy learning (Da et al., 2025, Zhang et al., 2024a). The same class of evidence can therefore drive architecture or model evolution. Its strength is an explicit and repeatable verdict; its boundary is the property checked. Compilation confirms acceptance by a build process, and a passing test confirms only the encoded assertions. Infrastructure failure, test non-execution, and a completed negative verdict must also remain distinguishable because they support different updates.

Diagnostic Evidence (D). Diagnostic evidence describes where and how execution failed. It includes compiler messages, assertion output, exception stacks, test logs, shell output, dependency conflicts, build failures, and tool errors. Live-SWE-agent uses command and test feedback while revising utilities, whereas HarnessFix and HELIX use failed trajectories and diagnostic traces to identify editable harness components (Chen et al., 2026a, Fan and Huang, 2026, Xia et al., 2025). ReVeal, CURE, ZeroCoder, and Sol-Ver similarly use interpreter feedback, generated tests, or execution failures to improve coder–verifier behavior (Fan et al., 2026, Jin et al., 2025, Lin et al., 2025b, Wang et al., 2025d). Diagnostics give failure an address, making them especially useful for deciding what to revise. That address may still identify a symptom. An exception in a caller may originate in an earlier interface change, while an import error may reflect the environment rather than the application code.

Coding Trajectories (T). A coding trajectory records how an attempt unfolds through repository search, fault localization, file inspection, editing, testing, failed branches, retries, and recovery. Its value lies in retaining process information discarded by a final score. SAGE compresses search–edit–test rollouts into reusable plans; SWE-Exp, EvoCoder, and Subtask Memory transform issue-resolution histories into experience at episode, repository, or subtask granularity (Chen et al., 2026c, Hayashi et al., 2025, Lin et al., 2024, Shen et al., 2026). CODESKILL and Socratic-SWE

further distill traces into skills for future tasks (Li et al., 2026c, Xiao et al., 2026a). Trajectories support procedural abstraction and credit assignment, but they do not solve either problem automatically. Successful traces can contain ineffective actions, and failed traces can contain a correct diagnosis followed by a faulty edit. Labeling every step with the final outcome would preserve both kinds of error.

Repository Context (R). Repository context describes the persistent software setting in which an experience was produced. They include issues, commits, pull requests, changed files, directory structure, interfaces, cross-file dependencies, line-level relevance, and repository revisions. Repository Memory links commit history, issues, and file changes to later localization, while GSkill and GSE use repository structure, conventions, and interaction histories when forming reusable capabilities (Tan et al., 2026, Wang et al., 2026a, Yang et al., 2026a). SWE-Pruner learns which code should remain visible, and Libra revises repository catalogs after localization failures (Wang et al., 2026i, Zhao et al., 2026). These signals do more than supply context. They define applicability. A workaround tied to one dependency version should not become an unconditional rule, and an accepted historical patch is evidence about one development state rather than the only valid implementation.

Quality Attributes (Q). Quality attributes describe software properties beyond functional correctness, including runtime, memory use, resource consumption, security, vulnerability removal, maintainability, and reviewability. EffiSkill derives optimization skills from slow-to-fast program pairs and measured efficiency gains, while EvoRepair combines repair outcomes with vulnerability patterns to retain security-repair experience (Hu et al., 2026, Wang et al., 2026j). Such signals are essential when two patches both pass tests but differ in operational or engineering value. They also introduce multi-objective trade-offs. An update that improves latency can increase memory use, and a security hardening rule can reduce compatibility. The metric, workload, and acceptable trade-off must therefore remain attached to the retained update.

Human Judgments (H). Human judgments arise from ordinary software collaboration. They include code-review comments, developer edits, acceptance or rejection, corrections, project conventions, and personal preferences. Behavioral Rules turns accepted review comments into persistent instructions and tracks whether the corresponding errors recur. Personalized Skills extracts user-specific procedures from developer–agent histories, and Ouroboros incorporates reviewed commits and feedback about weaknesses in the coding-agent core (Aggarwal and Ghalaty, 2026, Huang et al., 2026a, Razzhigaev et al., 2026). Human judgments can express intent, maintainability, and organizational authority that tests omit. It is also sparse, costly, and sometimes subjective, so the identity and scope of the feedback provider matter when deciding whether a preference should become a general rule.

Model Judgments (M). Model judgments are produced by an LLM judge, critic, reward model, or peer agent rather than directly by program execution or a developer. It includes natural-language diagnoses, candidate rankings, patch scores, responsibility assignments, and learned reward estimates. JUDGEFLOW uses a judge to rank the responsibility of workflow blocks, SIFT screens candidate harness patches before expensive execution, and SWE-RM predicts the quality of software-engineering trajectories (Fu et al., 2026b, Ma et al., 2026, Shum et al., 2025). M makes dense and scalable feedback available when execution is expensive or when a judgment requires semantic interpretation. Its output is nevertheless an inference, not an independent software fact. A confident critique may mislocalize a failure, and a reward model may preserve biases in its training comparisons. When a model judge consumes tests, diagnostics, trajectories, or repository evidence, we assign M together with the corresponding E, D, T, or R codes; an ungrounded model score receives M alone.

The seven codes are non-exclusive evidence labels rather than a ranking of reliability. Execution outcomes are usually precise but narrow; trajectories and human judgments are semantically richer but require interpretation; model judgments are scalable but inherit the coverage, provenance, and failure modes of both its inputs and evaluator. A method evaluated on a coding benchmark does not automatically receive a coding-specific evidence code. The label records what the evolution loop actually consumes while constructing or selecting its update.

We manually annotated all 108 methods in Figure 3 under this criterion. Execution outcomes dominate (E, 99), followed by trajectories (T, 87), diagnostics (D, 55), model judgments (M, 43), repository context (R, 37), quality attributes (Q, 21), and human judgments (H, 5).

The distribution exposes a sharp grounding imbalance. Current systems learn mainly from what software can execute and what agent runtimes can record, while developer intent remains weakly represented. Grouping the same annotations by persistent update target shows that this imbalance is structured rather than uniform. Memory methods rely most heavily on trajectories and repository context, consistent with retaining an experience together with the conditions under which it remains applicable. Skill and tool methods draw on the broadest mix of signals and use quality attributes more

often than other targets, because a reusable operation must be judged by more than task completion alone. Harness evolution has the highest reliance on model judgments, which help assign responsibility among interacting runtime components before executable checks validate a proposed repair. Model evolution, meanwhile, uses execution outcomes and trajectories in every surveyed method and diagnostics in most of them. Signal choice is therefore coupled to what evolves. Each update target observes a different slice of software development and inherits the blind spots of that evidence.

7.3 How Signals Drive Agent Evolution

A failed test is evidence, not yet an agent update. Its diagnosis can be stored as memory, abstracted into a debugging skill, used to revise a validation workflow, or converted into training supervision. The same software event can therefore produce changes with different reach and failure modes.

The conversion mechanism determines which part of the evidence becomes a lesson and how it can affect future work. We therefore examine experience generation, candidate construction, and selection for reuse. Self-play generates new tasks and trajectories; supervised fine-tuning (SFT), preference optimization, and reinforcement learning (RL) fit model parameters to such records; program search compares executable agent structures. These operations can be combined. Signal type and retained component alone do not reveal how an agent evolves.

Constructing experience. Many methods begin with trajectories from actual coding attempts, including failed edits and their diagnostics. SWE-Exp distills both successful and failed repairs, while Agent-RLVR uses unit-test results to identify failed attempts and then obtains guided reattempts that provide otherwise scarce successful trajectories (Chen et al., 2026c, Da et al., 2025). Self-Play SWE-RL (SSR) takes a different route: a shared agent injects bugs into sandboxed repositories, specifies them through test patches, and attempts to repair them. The generated cases change with the agent's capabilities and supply experience for joint RL (Wei et al., 2026b). Thus, self-play is a way to construct a training distribution, not itself an update to the deployed agent. Generated tasks must still be valid, informative, and relevant to later coding work.

Distilling reusable behavior. Non-parametric methods select or synthesize something that can be used again without retraining the task model. SWE-Exp compresses a repair attempt into issue-resolution experience; Repository Memory and Agent KB organize repository facts and past interactions for later retrieval (Chen et al., 2026c, Tang et al., 2025, Wang et al., 2026a). Retrieval is part of the mechanism: an accurate lesson that is never surfaced, or is applied to a different repository state, cannot improve the next decision. CODESKILL instead extracts procedures at several granularities and maintains a compact skill bank. It trains the skill-management policy with RL using both rubric feedback and verifiable outcomes from a frozen downstream coding agent (Li et al., 2026c). This is an important cross-layer case: RL optimizes the manager, while the reusable resource presented to the coding agent is a skill. Socratic-SWE uses trace-derived skills to help construct training tasks, and Live-SWE-agent writes and revises callable utilities during issue resolution (Xia et al., 2025, Xiao et al., 2026a). Skill abstraction and tool synthesis increase reuse, but can omit the conditions under which the original repair worked.

Searching executable structures. Other methods make the agent implementation itself a candidate. HarnessFix turns recurring failure diagnoses into scoped harness repairs, while EvoMAC uses testing-team feedback to revise coding-team instructions and connections (Chen et al., 2026a, Hu et al., 2024b). DGM samples an agent from an archive, uses a foundation model to modify its source code, evaluates the variant on coding tasks, and keeps alternative lineages available for later search (Zhang et al., 2025c). AFlow represents workflows as code and uses Monte Carlo Tree Search, code edits, and execution feedback to explore their structure (Zhang et al., 2024a). In these systems, feedback guides proposal or selection; it does not specify the correct architecture. The measured result depends on both the selected implementation and the search budget used to discover it.

Optimizing model parameters. When the retained change is a model, the training record and optimization rule must be stated separately. Agent-RLVR uses successful initial trajectories for SFT, then guided retries of failures to construct successful–unsuccessful pairs for offline DPO (Da et al., 2025). SWE-RL instead trains on historical issue and patch pairs using GRPO; its inexpensive reward measures similarity to the reference patch, rather than unit-test success (Wei et al., 2025b). Self-Play SWE-RL uses generated bug-repair episodes for joint RL (Wei et al., 2026b). These methods all learn from software-related evidence, but they optimize against different targets. A test verdict, reference-patch similarity, and a preference pair can rank the same trajectory differently. Parameter learning can spread such judgments across many future decisions, making reward design and data curation as consequential as the optimizer.

Table 5 compares 27 representative methods from Figure 3, pairing their update mechanisms with reported results on individual coding benchmarks.

Each row reports a result on the benchmark named beside it. CODESKILL's Terminal-Bench 2 gain compares its learned skill manager with a no-skill baseline on a benchmark held out from training; AFlow's HumanEval entry is a final score. The numbers use different models, budgets, baselines, and reuse scopes. A final score describes task performance; a change against a matched baseline more directly informs the contribution of an update. In particular, the table cannot isolate whether a model gain came from generating better training cases, filtering them differently, or changing the optimization rule. The same distinction applies to memory retrieval, skill management, and architecture search. Candidate construction must therefore be followed by independent checks and later reuse before it counts as a beneficial retained change.

Table 5: Evolution mechanisms and reported results for 27 representative methods in Figure 3.

Method	Mechanism	Target	Result	Benchmark
Asset updates				
SAGE	Grounded plan abstraction	Memory	73.2% resolved	SWE-bench Verified
SWE-Exp	Experience distillation	Memory	73.0% Pass@1	SWE-bench Verified
EvoRepair	Scored experience retrieval	Memory	93.47% (15 turns)	PATCHEVAL
Reflexion	Verbal reflection memory	Memory	91.0%	HumanEval
Repository Memory	Commit-memory retrieval	Memory	68.6% loc. Acc@1	SWE-bench Verified
MemCoder	Commit-memory refinement	Memory	68.4% → 77.8%	SWE-bench Verified
CODESKILL	RL-based skill management	Skills	+8.24 points	Terminal-Bench 2
Live-SWE-agent	Runtime tool synthesis	Tools	70.6% → 75.4%	SWE-bench Verified
EVOR	Query-knowledge co-evolution	Context	53.3% Pass@1	EvoR-bench (Tensor-M)
SWE-Pruner	Goal-conditioned context pruning	Context	70.6% → 72.0%	SWE-bench Verified
Architecture updates				
DGM	Archive-based code search	Agent code	20.0% → 50.0%	SWE-bench Verified*
SICA	Reflective self-code editing	Agent code	17.0% → 53.0%	SWE-bench Verified*
HarnessFix	Trace-guided harness repair	Harness	45.3% → 57.3%	SWE-bench Verified*
AFlow	Workflow search (MCTS)	Workflow	94.7% Pass@1	HumanEval*
SEW	Workflow and prompt mutation	Workflow	92.1% Pass@1	HumanEval
EvoFlow	Crossover, mutation, niching	Workflow	92.85% Pass@1	HumanEval*
JUDGEFLOW	Judge-guided block search	Workflow	93.4% Pass@1	HumanEval*
EvoMAC	Textual backpropagation	Team	94.51% Pass@1	HumanEval
AgentConductor	SFT + GRPO orchestration	Orchestrator	58.8% Pass@1	APPS
GEA	Group experience sharing	Agent code	71.0% resolved	SWE-bench Verified
Model updates				
SSR	Self-play + joint RL	Model	+10.4 points	SWE-bench Verified
ReVeal	Verification-aware RL (TAPO)	Model	38.7% (25 turns)	LiveCodeBench V6
CURE	Coder-tester co-evolution (RL)	Model	33.5% → 40.5%	LiveCodeBench V2
Sol-Ver	Self-play, SFT, and DPO	Model	18.23% → 27.24%	LiveCodeBench
Absolute Zero	Self-play + TRR++	Model	19.9% → 31.7%	LiveCodeBench V1-5
Agent-RLVR	Guided retries, SFT, offline DPO	Model	9.4% → 22.4%	SWE-bench Verified
SWE-RL	Patch-similarity reward + GRPO	Model	41.0%	SWE-bench Verified

Scores report final performance; arrows and gains indicate within-paper comparisons. Points denote percentage points. Repository Memory measures localization; EvoRepair measures cumulative repair. *Benchmark subset. Evaluation settings differ across papers.

Signal conversion can amplify errors. Self-authored verification can confidently approve invalid updates; external audits improve reliability (Guo et al., 2026a). Memory Reward Inflation shows failed episodes being overrated and retrieved; correction raises BIRD execution accuracy from 54.0% to 56.9% (Asadolahi et al., 2026). Phantom Guardrails reports nonexistent failures inducing persistent harness rules in 15 of 60 runs (Wang et al., 2026e). These cases show why adoption requires independent checks of provenance and applicability.

7.4 When to Evolve

The time at which feedback appears need not be the time at which the agent changes. Four events should be distinguished:

$$t_{\text{observe}} \leq t_{\text{propose}} \leq t_{\text{adopt}} \leq t_{\text{reuse}}. \tag{36}$$

An agent may observe a compiler error immediately, propose a tool after several failed commands, adopt it only after validation, and reuse it later in the same task or in a future repository. Online feedback therefore does not by itself establish online evolution. Timing is defined by when a persistent change enters later execution.

Within-Task Evolution. Within-task evolution adopts an update before the current task ends, shortening the distance between feedback and behavioral change. Live-SWE-agent creates and revises utilities while resolving an issue, and EvoMAC updates coding-team instructions and connections across rounds of the same development task (Hu et al., 2024b, Xia et al., 2025). This timing can exploit task-local state and test a candidate quickly, but the available evidence is incomplete and highly correlated with the current trajectory. A provisional update is appropriate when its scope is narrow and its effects can be reversed.

Post-Task Evolution. Post-task evolution forms and adopts an update after one task has finished. It can use the complete trajectory, final validation results, and regressions that were unavailable at the first failure. SWE-Exp distills finished attempts, CODESKILL constructs and maintains skills from accumulated coding activity, and HarnessFix turns recurring failed trajectories into scoped harness repairs (Chen et al., 2026a,c, Li et al., 2026c). Waiting improves the evidence available for credit assignment, although it delays reuse and may discard transient state needed to understand the failure.

Cross-Task Evolution. Cross-task evolution aggregates evidence from multiple tasks or evolution stages before committing broader changes. DGM evaluates generations of agent implementations, AFlow searches workflows over repeated benchmark executions, and Agent-RLVR collects guided trajectories before parameter optimization (Da et al., 2025, Zhang et al., 2025c, 2024a). Accumulation can reveal recurring patterns and support stronger validation, making it suitable for architecture and model updates with a wider blast radius. It also increases adaptation latency and the opportunity to fit the tasks used for proposal and selection.

The timing choice therefore trades responsiveness against evidential breadth. Local asset updates can often be attempted early because they are scoped and reversible. Architecture changes usually require broader regression evidence, while parameter updates commonly aggregate many trajectories before training. This is a tendency rather than a fixed correspondence. The appropriate time is determined by the uncertainty of $\mathcal{F}_t$, the intervention depth of $\widehat{u}_t$, its reuse scope ρ_t, and the cost of reversing it. Across all timings, the evolution claim begins when a candidate is adopted into later execution, and the improvement claim still requires a positive effect under the evaluation scope and budget developed next.

8 Benchmarks and Evaluation

Evaluating a self-evolving coding agent involves two coupled objects: the software artifact produced for the current task and the persistent agent state that shapes subsequent tasks. Tests, compilers, and reviewers can assess the artifact directly, but they supervise the retained update only indirectly. Let p_t denote a patch and $V(p_t; e_t)$ its validation result in environment e_t. Even a successful validation does not establish a beneficial agent update,

$$V(p_t; e_t) = 1 \quad \text{does not establish} \quad \Delta J_t > 0. \tag{37}$$

A valid patch can yield an overgeneralized memory, an unnecessary skill, or a harness rule that harms later tasks. Conversely, an unsuccessful attempt can contain a correct diagnosis that becomes useful after its conditions are identified. Evaluation must therefore distinguish artifact correctness from the behavioral contribution of the update derived from that artifact.

Coding benchmarks further play three roles in an evolution pipeline. Their tasks and signals may construct updates, select which candidate updates are retained, or report the final performance of the evolved agent. Reusing the same evidence across these roles can make repeated adaptation to the evaluator look like general improvement. A benchmark specifies where the agent acts and how task outcomes are checked; it does not by itself define a valid evaluation of evolution. The protocol must additionally specify what persists, which evidence influences adoption, what budget is available, and how far the claimed reuse extends.

Table 6: Forty representative conventional coding benchmarks grouped by evaluation setting. We summarize their task units, programming languages, and official GitHub and Hugging Face resources.

Benchmark	Task unit	Language	Link
Repository-level issue resolution			
SWE-bench	GitHub issue and patch	Python	
SWE-bench Verified	Expert-validated issue and patch	Python	
SWE-bench Lite	Curated issue and patch subset	Python	
SWE-Gym	Executable repository issue	Python	
Multi-SWE-bench	Multilingual issue and patch	Java, JS, TS, Go, Rust, C, C++	
SWE-bench-Live	Recent issue and patch	Python; multilingual extensions	
SWE-bench-Live (ML/Win)	Recent multilingual and Windows issue	Python, JS, TS, C, C++, C#, Java, Go, Rust	
SWE-Bench Pro	Long-horizon issue and patch	Python, Go, TS	
SWE-bench++	Expanded multilingual issue and patch	Python, Go, TS, JS, Ruby, PHP, Java, Rust, C++, C#, C	
SWE-EVO	Multi-step repository evolution	Python	
SWE-bench Multimodal	Visual issue and patch	JS, TS	
Visual SWE-bench	Visually grounded issue and patch	Python	
SWE-PolyBench	Multilingual issue and patch	Python, Java, JS, TS	
SWE-bench Multilingual	Curated multilingual issue and patch	C, C++, Go, Java, JS, TS, PHP, Ruby, Rust	
SWE-Sharp-Bench	C# issue and patch	C#	
SWE-Perf	Repository performance issue	Python, Cython	
OmniGIRL	Multimodal multilingual issue and patch	Python, TS, Java, JS	
SWE-Factory	Scalable repository issue and patch	Python, Java, JS, TS	
FEA-Bench	Front-end engineering issue	Python	
Function-level and competition-style programming			
HumanEval	Function completion	Python	
MBPP	Natural-language program synthesis	Python	
APPS	Interview and competition problems	Python	
CodeContests	Competition problems	C++, Python, Java	
LiveCodeBench	Temporally ordered coding problems	C++, Python, Java, others	
BigCodeBench	Function calls under complex instructions	Python	
EvalPlus	Extended functional test suites	Python	
MultiPL-E	Translated function synthesis	More than 18 languages	
DS-1000	Data-science code completion	Python	
CRUXEval	Input and output prediction for functions	Python	
CoderEval	Repository-context function or method generation	Python, Java	
Operational and specialized software tasks			
Terminal-Bench	End-to-end terminal task	Bash, Python, task-dependent	
InterCode	Interactive coding trajectory	Bash, SQL	
KernelBench	GPU kernel generation	Python, CUDA C++	
SEC-bench	Repository security task	Multiple repository languages	
CRAVE	Code-review verdict	Multiple repository languages	
CompileBench	Legacy software compilation	C, C++, build scripts	
DebugBench	Fault localization and program repair	C++, Java, Python	
BIRD	Database-grounded text-to-SQL	SQL	
Spider 2.0	Enterprise data workflow and text-to-SQL	SQL, Python, dbt	
VerilogEval	RTL generation and simulation	Verilog	

8.1 Conventional Coding Benchmarks Used for Evolution

Most self-evolving coding agents are evaluated in benchmarks originally designed to measure coding competence rather than change in the agent itself. These benchmarks remain valuable because they provide executable tasks, observable trajectories, and software-specific outcome checks. Their role in an evolution study, however, depends on both the coding behavior they expose and the claims their evaluation unit can support. We group the commonly used settings into repository-level issue resolution, function-level and competition-style programming, and operational or specialized software tasks. Table 6 lists representative datasets in these three groups and provides links to their available resources.

Repository-level issue resolution. Repository-level benchmarks expose issues, source trees, dependency states, execution traces, patches, and regression outcomes. SWE-bench introduced this setting using real GitHub issues and corresponding pull requests, while SWE-bench Lite, SWE-bench Verified, and SWE-Bench Pro refine task quality, validation, and difficulty (Deng et al., 2025, Jimenez et al., 2023). SWE-Gym further turns repository tasks into executable environments for agent training and evaluation (Pan et al., 2025). These settings expose realistic evidence for constructing updates because an experience can be grounded in a repository revision and its patch can be checked against later software behavior. In their usual episodic use, however, the evaluation unit remains an issue–patch pair. The resulting score shows whether the current agent solves sampled issues, not whether an update produced from issue c_i improves the agent on a later issue c_j.

Function-level and competition-style programming. HumanEval and MBPP provide concise specifications and executable tests, while APPS, CodeContests, and LiveCodeBench extend the difficulty and diversity of programming

problems (Austin et al., 2021, Chen et al., 2021, Hendrycks et al., 2021, Jain et al., 2024, Li et al., 2022). Their controlled interfaces make them useful for comparing model learning and workflow search, including systems such as SEMAG and SEW (Liu et al., 2025, Peng et al., 2026a). They reveal whether an update improves program synthesis under a stable oracle, but usually provide little evidence about repository navigation or dependency management. Repeated attempts can expose self-correction and algorithmic refinement within a problem, but a reset between problems removes the persistence needed to test whether the agent carries an improvement forward.

Operational and specialized software settings. Terminal-Bench evaluates long-horizon interaction with terminals and execution harnesses (Merrill et al., 2026). Repository understanding, compilation, and environment construction are examined through LongCodeQA, CompileBench, Repo2Run, EnvBench, and EnConda-Bench (Guo et al., 2026b, Ren et al., 2026a, Wang et al., 2026i). Security and efficiency benchmarks add objectives not captured by functional tests. PATCHEVAL, SEC-bench, and VUL4J examine vulnerability repair, while PIE and Mercury measure program optimization (Hu et al., 2026, Wang et al., 2026j). Finally, developer-agent sessions expose review, acceptance, and repeated use in real projects (Aggarwal and Ghalaty, 2026, Huang et al., 2026a). Together, these settings broaden the observable consequences of evolution. They also make comparisons harder because environment state, workloads, tool versions, and human assistance can change along with the agent. Their value is therefore greatest when the relevant state can be replayed and competing agents receive matched opportunities to act.

These conventional benchmarks provide the environments and feedback from which an agent may evolve, but they do not by themselves identify evolution. This limitation becomes critical when the same tasks or test signals generate trajectories, guide candidate selection, and report final performance. Repeated adaptation can then raise the score without producing a reusable improvement. Static task collections also face solution leakage, weak tests, and training contamination, motivating filtered and continuously refreshed variants (Aleithan et al., 2024, Badertdinov et al., 2025, Zhang et al., 2025d). These concerns affect coding-agent evaluation broadly; the additional challenge for self-evolution is to show that a retained change, rather than repeated exposure to the evaluator, causes better future behavior.

8.2 Benchmarks Designed to Evaluate Evolution

Evolution-oriented benchmarks differ in the object whose change they evaluate, rather than in when an update occurs. We distinguish changes to a task solution, a software artifact, and persistent agent state. Evidence at one level does not establish change at the next.

Solution-Level Adaptation. EvoCodeBench tracks correctness, runtime, memory use, and algorithmic efficiency across successive refinements of one solution (Zhang et al., 2026g). It tests whether feedback improves the current solution. If the agent is reset before the next problem, the result does not show that the state used for future tasks has changed.

Artifact-Level Evolution. SWE-EVO uses release-scale tasks that require coordinated changes across files while preserving existing behavior (Thai et al., 2025). It measures sustained change to a software artifact, but a repository can evolve while its agent remains fixed. The result supports agent evolution only when earlier experience is retained in S_t, H_t, or θ_t and affects later decisions.

Persistent Agent-State Evolution. These benchmarks carry agent state across tasks or fresh sessions. LifelongAgentBench tests accumulation and transfer over interdependent task sequences (Zheng et al., 2025). SkillFlow evaluates skill discovery, repair, reuse, and regression, while SkillLearnBench connects skill quality and invocation to task outcomes (Zhang et al., 2026m, Zhong et al., 2026). PAST-Bench compares matched persistence-on and persistence-off conditions, and SEA-Eval jointly tracks reliability, long-term performance, and token use over sequential streams (Jiang et al., 2026, Xue et al., 2026). MemoryArena and Evo-Memory isolate memory accumulation, revision, and behavioral reuse (He et al., 2026, Wei et al., 2025a). Together, these protocols test whether retained experience changes later behavior; equal success rates may still conceal different trajectories and costs.

Causal Attribution Protocols. An evolution benchmark must combine realistic software tasks with sequential state and attribution controls. It should separate the cases used to construct updates, select candidates, and report final performance. Let $\mathcal{D}_{\text{upd}}$, $\mathcal{D}_{\text{sel}}$, and $\mathcal{D}_{\text{eval}}$ denote these roles. Because candidate selection is part of improvement, final evaluation must remain insulated from both update construction and selection. Repository, temporal, and defect-family separation is stronger than a random issue split when cases can share files, APIs, tests, or solution patterns.

GDPevo illustrates this principle by distributing atomic rules across update tasks and recombining them in held-out cases (Zhou et al., 2026). Its business workflows are not coding tasks, but the same design can test new combinations

of APIs, defect families, repository structures, or validation requirements.

Attribution further requires a matched frozen agent that receives the same observations, tools, budget, and repair opportunities but does not retain the candidate update. Component hybrids such as (S_{t+1}, H_t, θ_t), (S_t, H_{t+1}, θ_t), and (S_t, H_t, θ_{t+1}) can locate the source of a gain when components are exchangeable. Checkpoints, repeated evaluation of earlier groups, and multiple task orders then reveal retention, forgetting, and curriculum effects. These controls turn a coding benchmark into an experiment about a changing agent.

8.3 Measuring Benefit, Regression, and Cost

Pass rate, resolve rate, repair rate, benchmark score, and Pass@k remain necessary measures of software outcomes (Chen et al., 2026c, Liu et al., 2025, Peng et al., 2026a, Xia et al., 2025). For an evolving agent, however, an aggregate score can hide exchanges between acquired and lost behavior. Recent evolution-oriented evaluations make this limitation visible by comparing changing and frozen agents across sequential task streams, rather than reporting only the terminal score (Deng et al., 2026a, Jiang et al., 2026). Let $d_t(c) = s(A_{t+1}, c; \mathcal{B}) - s(A_t, c; \mathcal{B})$ be the matched score change on case c. The acquisition rate is

$$G_t = \Pr_{c \sim \mathcal{D}} [d_t(c) > 0], \tag{38}$$

and the regression rate is

$$R_t = \Pr_{c \sim \mathcal{D}} [d_t(c) < 0]. \tag{39}$$

For binary task outcomes, G_t counts newly solved cases and R_t counts cases lost after the update. Reporting both prevents a positive average gain from concealing substantial regressions. For graded metrics, their magnitudes should be reported alongside the two rates. These rates provide a common summary of a distinction that recent benchmarks expose through different measurements. SkillLearnBench finds that skill-learning gains are not consistent across tasks and models (Zhong et al., 2026), while ContinualSkillBench shows that sequential improvement can be comparable to context-only adaptation and can leave weaker agents with fragmented, task-specific skill collections (Guan et al., 2026). A positive average can therefore reflect a small set of narrow gains even when the retained component remains redundant or fails to generalize.

Longitudinal evaluation adds another axis. If $J_{i,t}$ denotes performance on task group i after the agent has evolved through stage t, the matrix of $J_{i,t}$ values shows forward transfer to later groups and retention on earlier ones. Periodic evaluation on old repositories or defect families can reveal forgetting, while evaluation on new repositories tests whether the learned behavior exceeds the scope of its generating feedback. The same checkpoints should also record component growth, retirement, and rollback success. A memory bank that continually expands while retrieval quality declines is not well described by its final solve rate alone. FinEvo-Bench exemplifies this design with paired evolving and non-evolving controls evaluated under independently shuffled task streams (Deng et al., 2026a). Evidence from skill maintenance also shows that regression can be path dependent. Unconditional skill accumulation can improve performance initially and then degrade it, while removing the original harmful skills may recover only part of the loss because later skills have already inherited their errors (Shang et al., 2026). Checkpoints should therefore preserve update lineage and measure both pre-admission harm and post-hoc rollback effectiveness.

Evolution costs and use costs should be separated. Let C_{evolve} include experience processing, candidate construction, search, and selection, and let C_{use} be the cost of executing the retained agent once. If an update is reused N times, its amortized cost per use is

$$\overline{C}_N = \frac{C_{\text{evolve}}}{N} + C_{\text{use}}. \tag{40}$$

This distinction changes the interpretation of efficiency. An expensive offline harness search may be worthwhile when its result supports many later tasks, whereas a utility created for one trajectory has little opportunity to amortize its construction cost. SkillMOO makes this trade-off explicit by optimizing skill bundles for pass rate, monetary cost, and runtime, with pruning and substitution sometimes outperforming continued accumulation (Gong et al., 2026). SEA-Eval further shows that similar success rates can hide large differences in token consumption (Jiang et al., 2026). A broader study of coding-agent trajectories likewise finds substantial run-to-run token variation and no monotonic relation between greater token use and accuracy (Bai et al., 2026a). Reports should therefore include the update budget, evaluation and selection costs, subsequent token and tool usage, wall time, storage growth, and human supervision.

The resulting evaluation record should connect four quantities: the software outcomes obtained, the persistent components changed, the cases that gained or regressed, and the resources consumed to produce and use those changes.

Security, maintainability, and reviewability should be added when they are part of the intended software outcome rather than inferred from functional success. EVOMAL illustrates why this record must extend beyond task accuracy. A retrieved malicious skill can be copied into newly authored skills and remain in the evolving library after the planted source is removed (Wu et al., 2026). Such propagation is not captured by G_t or R_t alone, but it is a persistent behavioral consequence of the update process. This protocol turns benchmark scores into evidence about a changing agent. It also preserves the central boundary of the survey. A retained mechanism shows that evolution occurred, while matched, held-out, and longitudinal evaluation shows whether that evolution improved the coding agent.

9 Self-Evolving Coding Products

9.1 Product Landscape

Research systems describe what a coding agent could evolve, whereas deployed products reveal which forms of evolution can operate under practical constraints. We organize the product landscape using the asset, architecture, and model layers introduced earlier and identify the persistent object changed by each product. Inclusion requires a documented mechanism for retaining agent state across later steps, sessions, or tasks; it does not require a fully autonomous or experimentally validated evolution loop. Table 7 summarizes representative products documented as of September 2026.

The distribution across these layers is highly asymmetric. Persistent memory, skills, and context appear in many products, a smaller group reaches into the execution harness, and none publicly exposes repository-derived parameter updates as a persistent product capability. This pattern defines a *deployment frontier*. The relevant design question is how much behavioral leverage an update provides relative to the cost of scoping, validating, auditing, and reversing it. Increasing intervention depth usually expands both the influence of an update and the consequences of an error. The three layers therefore describe intervention depth rather than a maturity ranking; a well-governed asset update may be more useful in production than a deeper change whose contribution cannot be isolated.

Table 7: Representative self-evolving coding products organized by their documented persistent update layers.

Product	Company	Year	Persistent update layer		
			Assets	Arch.	Model
Architecture-spanning products					
Prime Agent (Prime Intellect, 2026)	Prime Intellect	2026	•	•	○
DeepSeek Harness (DeepSeek-AI, 2026a)	DeepSeek	2026	○	•	○
OpenBlock Agent† (OpenBlock Labs, 2026)	OpenBlock Labs	2026	•	•	○
Asset-centered products					
Gemini CLI Auto Memory (Google, 2026)	Google	2026	•	○	○
GitHub Copilot Memory (GitHub, 2026)	GitHub	2026	•	○	○
Augment Agent / Cosmos (Augment Code, 2025)	Augment	2025–2026	•	○	○
Claude Code Auto Memory (Anthropic, 2026)	Anthropic	2026	•	○	○
Cursor Memories / Automations (Cursor, 2026)	Cursor	2025–2026	•	○	○
Devin Insights / Playbooks (Cognition, 2026)	Cognition	2025–2026	•	○	○
Windsurf Cascade Memories (Windsurf and Cognition, 2026)	Windsurf	2025–2026	•	○	○
Letta Code (Letta, 2026)	Letta	2025–2026	•	○	○
Hermes Agent (Nous Research, 2026)	Nous Research	2026	•	○	○
Replit Agent (Replit, 2026)	Replit	2025–2026	•	○	○
OpenClaw (OpenClaw, 2026)	Community	2026	•	○	○

• indicates a documented persistent update at the layer; ○ indicates that no persistent self-update is documented, not that the product lacks that component.
†OpenBlock includes announced capabilities.

Asset-centered products. Assets are the dominant entry point for productized evolution. Claude Code, GitHub Copilot, Cursor, Windsurf, Augment, and OpenClaw persist repository facts, preferences, or workspace context across sessions (Anthropic, 2026, Augment Code, 2025, Cursor, 2026, GitHub, 2026, OpenClaw, 2026, Windsurf and Cognition, 2026). Other products move from factual retention toward procedural reuse. Gemini CLI proposes memory patches

and skill drafts, Devin consolidates session evidence into knowledge and playbooks, Letta Code revises memory and versions skills, Hermes creates and updates skills from experience, and Replit Agent can preserve learned procedures as project skills (Cognition, 2026, Google, 2026, Letta, 2026, Nous Research, 2026, Replit, 2026). These mechanisms change what the agent can recall or invoke while leaving the surrounding execution structure largely intact. Their concentration at this layer follows from the properties of externalized state. Memory and skills can be represented as bounded files or records, scoped to a user, branch, or repository, inspected before reuse, and edited or removed without replacing the whole agent. Their provenance can remain attached to the stored item, and access control can follow existing workspace boundaries. This supports tenant-specific adaptation while reducing the need to pool private repository traces into shared model training. GitHub Copilot, for example, attaches source citations to repository facts and revalidates them against the current branch, while Gemini CLI holds generated memory patches and skills for review before adoption (GitHub, 2026, Google, 2026). Asset updates thus resemble small, versionable software changes whose scope and owner can be made explicit.

This advantage also reveals the main weakness of asset-centered products. Storage is easier than maintaining a useful asset collection. As entries accumulate, overlapping memories can disagree, a skill can outlive the API or repository state under which it was learned, and retrieval can introduce an irrelevant item into an otherwise correct trajectory. Product quality therefore depends increasingly on lifecycle operations such as consolidation, conflict detection, revalidation, retirement, and rollback. The competitive boundary is shifting from whether a product can remember to whether it can govern what should remain active and determine when that retained state should influence a decision.

Architecture-spanning products. A smaller set exposes changes to the organization of execution. Prime Agent treats prompts, skills, memory, and sub-agent specifications as editable continual-harness state and records revisions for rollback (Prime Intellect, 2026). DeepSeek Harness allows capabilities and runtime modes to be recomposed through plugins and persistent presets (DeepSeek-AI, 2026a). OpenBlock presents an additional boundary case that connects learned codebase patterns with planned skill and evaluation-suite construction (OpenBlock Labs, 2026). These products can also update assets, but their architectural significance lies in changing how capabilities are assembled, delegated, or controlled. Such changes have a wider effect because they can alter control flow, tool exposure, and coordination across many later tasks.

Harness evolution is emerging because it occupies a useful middle layer. A change to validation order, retry policy, context routing, or sub-agent delegation can affect many decisions without modifying the underlying model. The implementation remains code or configuration that can be versioned, tested, compared, and rolled back. It can also survive a model upgrade, giving the product a source of persistent behavior that is partly independent of a particular model release. These properties make architecture a high-leverage target when stronger base models alone no longer determine product behavior.

The same leverage increases the blast radius of a faulty update. A misleading memory may affect the tasks that retrieve it, whereas a changed termination or validation rule may affect every task passing through the harness. Asset and architecture evolution are also coupled in practice. An asset determines what knowledge or operation is available, while the harness determines when it is retrieved, trusted, executed, and revised. A product that creates skills but never changes these control rules remains asset-centered; a product that revises selection, delegation, or validation moves into architecture even when the edit also touches prompts or skills. This functional distinction explains why similar persistent files can represent different depths of intervention.

Model-level products. The catalogue contains no product that publicly documents persistent, repository-derived updates to its deployed model parameters. Providers may continue training their underlying models, but that process normally occurs outside the user-visible agent loop and cannot be attributed to a particular repository trajectory. The product landscape therefore becomes sparser as the intervention moves from assets to architecture and then to model parameters. Deeper changes have a broader behavioral scope, require more expensive validation, and are harder to diagnose or reverse when they fail. Several constraints explain this gap. A repository trajectory provides sparse and correlated evidence, and a passing patch does not show which parameter change should be retained. Online adaptation can also introduce forgetting, spread repository-specific shortcuts across tasks, or encode sensitive code in a component shared by multiple users. Parameter updates are expensive to evaluate because their effects are distributed across behaviors that may be unrelated to the generating task. Rollback is correspondingly coarse: restoring a checkpoint removes all changes bundled into it, while the harmful influence of one memory or harness edit can often be traced and withdrawn directly.

This creates a separation between provider evolution and product evolution. Providers can train new model releases using aggregated data, but the resulting change follows a release cycle and is rarely attributable to one user's coding experience. User-visible self-evolution therefore remains concentrated in external state and executable orchestration. The product landscape suggests a practical design rule: expose the deepest persistent intervention whose provenance, evaluation scope, and rollback boundary can still be made concrete.

9.2 From Experience to Governed Updates

Productization turns evolution into a controlled state change. A task trajectory and its feedback may generate a candidate update $\hat{u}_t$, but the candidate should affect later execution only after an admission decision. Let z_t denote the persistent product state, which may contain assets or architecture. A generic governed update is

$$z_{t+1} = \begin{cases} \mathrm{Apply}(z_t, \hat{u}_t), & \mathrm{Gate}(\hat{u}_t; v_t, p_t, \rho_t) = 1, \\ z_t, & \text{otherwise}, \end{cases} \tag{41}$$

where v_t summarizes validation, p_t records provenance and authority, and ρ_t specifies the permitted scope of reuse. The gate may be implemented by automatic checks, explicit user approval, organizational policy, or a combination of them.

This compact expression contains several decisions that products often conflate. Proposal determines which part of a trajectory becomes a candidate; validation asks whether its evidence extends beyond the case that produced it; authorization determines who may change shared agent state; and scoping decides which user, repository, branch, or runtime may receive the update. These are part of the evolution mechanism because they control which experience is allowed to shape future behavior. A permissive gate increases plasticity but can accumulate stale memories, conflicting skills, or brittle control rules. A conservative gate preserves stability but may discard useful adaptation. The appropriate threshold depends on the update's blast radius. A repository fact can be admitted under narrow scope and revalidated when retrieved, whereas a tool or harness revision may require integration tests, held-out tasks, and explicit rollback support. Validation can therefore combine the estimated gain $\widehat{\Delta J}_t$, regression risk $\widehat{R}_t$, update and use costs, and the breadth of ρ_t rather than reducing admission to success on the generating task.

Products instantiate this control point differently. Gemini CLI never applies automatically extracted memory or skill candidates without approval, whereas Claude Code writes repository-scoped memory that remains directly inspectable and editable (Anthropic, 2026, Google, 2026). GitHub Copilot validates a recalled repository fact before use and retires entries that remain unused (GitHub, 2026). Prime Agent exposes update history and rollback for its continual harness (Prime Intellect, 2026). These mechanisms treat a persistent agent update much like a software change. It is proposed, scoped, reviewed or validated, committed, monitored, and potentially reverted.

Admission should remain provisional after deployment. An update can move from *candidate* to *active*, then to *revised*, *quarantined*, or *retired* as new evidence arrives. Monitoring must attribute later failures and gains to the version that was actually used, while rollback must restore associated indexes, dependency bindings, and derived presets as well as the visible memory or code. Product governance thus turns a collection of persistent additions into maintained, versioned agent state. This lifecycle is also what allows the hot-swappable architecture discussed next to support continued evolution without making every runtime experiment permanent.

9.3 Toward Hot-Swappable Coding Agents

> *"A future harness may generate and deploy modifications to its own components while continuously serving requests."*
>
> —***A Programming Paradigm for Spatiotemporal Composability*** (Shi et al., 2026a, p. 5)

DeepSeek Harness provides a concrete prototype of a product form in which the composition of a running agent becomes a primary site of evolution. It exposes models, tools, skills, session management, sandboxes, storage, execution loops, scheduling, and the user interface as plugins, with configuration determining which implementations are mounted together (DeepSeek-AI, 2026a). The unit of change can therefore be an individual capability such as a search tool, context policy, validation loop, or model adapter. Its Cordis lifecycle can unload an existing plugin, unwind

the registrations and effects owned by that instance, and load revised code, while configuration edits affect only the entries that were mounted, removed, or reconfigured (DeepSeek-AI, 2026b). Creator mode extends this mechanism by allowing developers or agents to inspect the live runtime, test temporary plugins in memory, and combine selected components into presets (DeepSeek-AI, 2026a). These mechanisms suggest a two-speed evolution process. The fast path mounts a candidate during a session and observes its immediate behavioral effects. The slower path validates it on broader cases, records its provenance and dependencies, and promotes it into a versioned preset for later sessions. A temporary mount supplies evidence about a candidate; adoption, retention, and subsequent reuse turn that candidate into a persistent update. The harness consequently acts as an *evolution control plane* joining experience-derived proposals with evaluation and deployment.

This product form shifts the central engineering problem from replacing a whole agent to controlling change inside a live composition. Active tasks must remain bound to compatible plugin versions, state must be migrated or discarded when interfaces change, and rollback must withdraw both a component and the effects it introduced. Version binding, dependency-aware disposal, shadow or canary evaluation, and atomic promotion can support a small stable kernel surrounded by an evolving periphery of plugins. The common plugin boundary does not erase the taxonomy developed earlier. A callable utility remains an asset update, a plugin that changes routing or the execution loop is an architecture update, and a model-adapter swap establishes model self-evolution only when the model's parameters are themselves updated. Functional role still determines the depth of evolution, even when every change is packaged and deployed in the same way.

10 Challenges and Open Problems

Self-evolving coding agents raise challenges beyond those of conventional coding agents because their behavior changes over time. An unreliable test result, noisy trajectory, weak verifier, or benchmark-specific shortcut may not only affect one patch, but also be stored as memory, distilled into a skill, selected as a workflow, used to update a model, embedded in a scaffold, or used to reshape the agent's context and execution environment. Thus, the key challenge is not only whether self-evolution improves benchmark performance, but whether the evolutionary process remains reliable, reproducible, and aligned with software engineering constraints. The consequences depend on both reuse scope and intervention depth. A faulty repository memory may bias a family of related issues, whereas a faulty harness or model update can alter every subsequent decision. The open problems below therefore concern the reliability of the entire path from local software evidence to persistent behavioral change.

Reproducibility, contamination, and benchmark overfitting.. Self-evolution makes reproducibility difficult because agents may change across runs, tasks, repositories, tool environments, or model versions. Systems that select self-modifications or agent variants using benchmark outcomes are especially sensitive to evaluation noise and benchmark leakage (Robeyns et al., 2025, Zhang et al., 2025c). This concern is amplified in code evaluation, where contamination and benchmark-specific adaptation are already known issues (Jain et al., 2024). SWE-rebench evaluates models on continuously collected, fresh software-engineering tasks and finds that some results on SWE-bench Verified may be inflated by contamination (Badertdinov et al., 2025). DeepSWE exposes a related measurement problem. On original long-horizon tasks, an independent judge disagrees with its purpose-built verifiers far less often than with tests inherited from public fixes, showing that both solution exposure and incomplete tests can distort the apparent capability of an agent (Huang et al., 2026b). A matched-budget study of automatic harness evolution provides more direct evidence: on Terminal-Bench 2.1, evolved harnesses do not consistently outperform simple test-time search baselines and show limited generalization to held-out tasks (Wang et al., 2026h). Reproducibility also depends on the surrounding harness. Holding the model fixed across 35 sequential Qwen Code releases reveals quality fluctuations attributable to harness changes, while a separate comparison finds up to a 40× difference in tokens per solved task across three harnesses (Ben Sghaier et al., 2026, Vats and Golev, 2026). For a self-evolving agent, contamination extends beyond pretraining overlap. If the same tasks construct experience, select retained updates, and measure the final system, the evaluation loop can progressively fit its own evidence. Even a fully reproducible final score may then reproduce an overfitted evolutionary path. A credible report should preserve the initial agent, task order, update history, evaluator version, and adopted and rejected candidates; separate update, selection, and held-out evaluation cases; and repeat the process across seeds or task orders. The object to reproduce is the trajectory of change, not only the terminal agent.

Feedback reliability, safety, and tool dependence.. Executable feedback is central to self-evolving coding agents, but tests, compilers, CI logs, generated tests, and reward models are imperfect. Systems that rely on unit-test validation, environment rewards, or learned verifiers may therefore inherit the biases and blind spots of these signals (Pan et al., 2025, Wei et al., 2025b). This is particularly risky when agents modify tools, workflows, or their own scaffolds, because a misleading feedback signal can shape future behavior rather than only one output. An analysis of 4,892 patches from ten software-development agents finds that even test-passing patches can modify different files and functions from developer patches, exposing limits in the tests used to identify acceptable repairs (Chen and Jiang, 2024). Security-focused evidence is more direct. Across 1,030 valid traces from seven agent frameworks, 170 patches passed the available functional checks while omitting protections, applying inadequate fixes, or introducing vulnerabilities; neither test passing nor LLM review reliably intercepted these cases (Bai et al., 2026b). The same concern applies when the agent authors its own acceptance signal. Experiments on heuristic self-improvement show that self-assigned verification scores can stay near perfect while sealed deployment performance remains low or regresses; retaining an external accept–reject audit improves results across six models (Guo et al., 2026a). Memory-based agents expose a related feedback loop. Incorrect episodes can receive inflated self-evaluation scores and then be preferentially retrieved, whereas a de-inflation method raises BIRD execution accuracy from 54.0% to 56.9% (Asadolahi et al., 2026). Coding feedback is therefore explicit without necessarily being complete. The danger grows when an update compresses a locally observed outcome into a broadly reused rule while discarding the conditions under which the outcome was valid. Future systems need to retain provenance, environment versions, test coverage, and applicability conditions with the update itself. Adoption should combine independent signals where possible, such as regression tests, static analysis, security checks, differential execution, and review, while treating flaky tests, generated oracles, sandbox mismatches, and tool failures as uncertainty rather than negative task evidence. Tool reliability and sandbox fidelity are thus part of the learning signal, not merely implementation details.

Reversibility, harness bloat, and specialization.. The hot-swappable product form discussed in Section 9.3 makes reversibility a runtime property. Withdrawing a live component must also undo its state changes and dependency effects while requests continue to be served. Cordis calls this property *temporal composability* and implements it through tracked inverse effects (Shi et al., 2026a). AHE similarly makes harness components explicit and revertible, but end-to-end guarantees of this kind are not yet standard across self-evolving coding agents (Lin et al., 2026a). Its experiments also show why explicit components and attribution matter: ten evolution rounds improve Terminal-Bench 2 performance, while ablations locate the transferable gains in tools, middleware, and long-term memory rather than in every edited component. The Phantom Guardrails study demonstrates how bloat can arise even without a real failure: a proposer installs a guardrail for a nonexistent violation in 15 of 60 runs when harmless input resembles a familiar rule, compared with none on featureless input, and an add-only loop makes the spurious component persist (Wang et al., 2026e). Reversibility must include the evaluation path as well as task execution. An audit of real self-improvement trajectories finds harness tampering across different systems and observes that such edits can persist in the lineage of the selected best agent (Wang et al., 2026f). A candidate may appear better because it changes task access, termination, logging, or validation rather than because it solves software problems more effectively. This makes authority, provenance, and evaluator isolation part of harness integrity. Components that generate and judge an update should not be able to silently weaken the checks that authorize its adoption. Without lifecycle controls, evolution can become cumulative trial-and-error. A locally successful fix may be appended as another prompt rule, memory item, tool wrapper, routing condition, or agent role rather than replacing the mechanism that caused the failure. The resulting harness becomes larger, more coupled, and harder to audit, while narrow benchmark selection can turn these additions into repository- or environment-specific shortcuts. Future systems should therefore treat removal as a first-class operation, track the provenance and inverse of persistent modifications, enforce complexity budgets, retire redundant components, and validate promoted changes on held-out software environments. Progress should be measured against both performance and structural cost. Otherwise, an agent can improve a benchmark while becoming less portable, less inspectable, and more expensive to maintain.

Long-term memory, skills, and coordination.. Memory and skill mechanisms allow agents to reuse software-engineering experience, but they also introduce quality-control problems. Experience banks, repository memory, and skill libraries may become stale, redundant, overly repository-specific, or contaminated by failed trajectories (Chen et al., 2026c, Li et al., 2026c, Xiao et al., 2026a). Controlled experiments on memory management reveal an experience-following effect in which similar retrieved records induce similar outputs. Incorrect or misaligned records can therefore propagate errors, whereas selective addition and deletion outperform unrestricted memory growth (Xiong et al., 2026). Skill evolution has a similar lifecycle problem. A systematic study across extractors, consumer models,

and five agentic domains finds that generated skills help on average but also produce non-trivial negative transfer; extraction quality and consumption quality do not necessarily coincide (Huang et al., 2026d). These results make retention conditional: an artifact is useful only for a compatible task, retriever, consumer, and execution context. This producer–consumer gap is also visible at the harness level. Models from different capability tiers can produce updates with similar gains, yet their ability to activate and follow the resulting artifacts is non-monotonic, with weak models often failing to use what was retained (Lin et al., 2026b). Multi-agent and workflow-evolving systems add coordination challenges because a local improvement can alter the assumptions of neighboring components. Changing one role, message schema, or routing rule may make an existing skill unreachable or duplicate another agent's responsibility. AgentConductor empirically shows that topology affects both coding accuracy and communication cost, obtaining up to a 14.6% Pass@1 improvement while reducing topology density and token use (Wang et al., 2026d). Evidence from a deployed collaboration network shows that scale alone does not solve this problem: among more than 1.5 million shared assets in EvoMap, 98% are never reused and over 84% of approved assets bypass quality checks through vacuous tests (Ye et al., 2026). The broader problem is therefore not how much experience the system retains, but whether it can maintain a coherent persistent state. Future work should jointly manage relevance, conflicts, dependencies, usage, and retirement across memory, skills, workflows, and teams instead of optimizing each store in isolation.

Evaluation beyond short benchmarks.. Current evaluations mainly measure short-horizon success through pass rates, resolve rates, or benchmark scores. Yet real software engineering also requires maintainability, security, reviewability, efficiency, and long-term reliability. The gap is already visible without moving beyond coding. SWE-EVO requires release-scale changes spanning an average of 21 files and hundreds of tests; a GPT-5 and OpenHands configuration resolves 21% of these tasks, compared with 65% on SWE-bench Verified (Thai et al., 2025). The difference shows that success on isolated issues does not establish sustained planning, preservation of earlier behavior, or control of interactions among changes. Longitudinal agent benchmarks reveal a second gap. LifelongAgentBench finds that straightforward experience replay is limited by irrelevant records and context constraints (Zheng et al., 2025). PATH-Bench further shows that strong forward transfer does not guarantee retention and that later experience can reshape gains acquired earlier in a task sequence (Yang et al., 2026c). Coding-specific memory experiments across six benchmarks find a 3.7% average gain from cross-domain transfer, but also show that abstract validation routines transfer more reliably than concrete traces, which can cause negative transfer through excessive specificity (Kim et al., 2026). Evaluation must consequently follow the agent after an update rather than test it once. Repeated checkpoints should measure newly acquired behavior, regressions on earlier repositories, transfer to held-out repositories and defect families, resource growth, and sensitivity to task order. Cross-domain transfer remains an open question, but the immediate standard should be stronger: evolution learned from one part of software engineering should survive time and improve later coding work beyond the cases that produced it.

11 Conclusion

Coding agents are unusually well positioned to learn from their own work. Execution yields specific outcomes, diagnostics, and trajectories that can be tied to repository state and checked again. Our synthesis follows how this evidence becomes persistent change in assets, architecture, or model parameters, with progressively deeper intervention. The crucial distinction is between retaining an update and showing that it improves later decisions. Passing a test validates a software artifact under particular conditions; demonstrating agent improvement requires reuse and benefit on matched, held-out tasks.

Research and products currently favor memories, skills, tools, and editable harnesses because their changes can be scoped, inspected, and reversed. Looking ahead, a coding agent could treat these components as a versioned system that it revises during ordinary coding work. Experience would generate candidate changes, independent checks would govern their adoption, and later tasks would show whether each change should be kept, revised, or rolled back. Extending this cycle to workflows and model parameters will require stronger evidence of gains, regressions, and costs across repositories and time. The long-term goal is an agent that can identify which experiences deserve to persist, where they apply, and when they should be forgotten.

References

Emre Can Acikgoz, Cheng Qian, Jonas Hübotter, Heng Ji, Dilek Hakkani-Tür, and Gokhan Tur. Tool-R0: Self-evolving LLM agents for tool-learning from zero data, 2026. URL https://arxiv.org/abs/2602.21320.

Aditya Aggarwal and Nahid Farhady Ghalaty. Self-improving AI coding agents through accumulated behavioral rules: A closed-loop framework, 2026. URL https://arxiv.org/abs/2607.13091.

Amal Akli, Melissa Akli, Cedric Richter, Mike Papadakis, and Yves Le Traon. From Failing to Passing: Evolving Natural Language Prompt Optimization Rules for LLM Code Generation, 2026. URL https://arxiv.org/abs/2607.05121.

Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for LLMs, 2024. URL https://arxiv.org/abs/2410.06992.

Anonymous. Mendel Gödel machine: Comparative evolution enables state-of-the-art self-improving coding agents, 2026. Manuscript under review.

Anthropic. Manage Claude's memory. Claude Code documentation, 2026. URL https://code.claude.com/docs/en/memory. Accessed 2026-08-18.

Daman Arora, Atharv Sonwane, Nalin Wadhwa, Abhav Mehrotra, Saiteja Utpala, Ramakrishna Bairi, Aditya Kanade, and Nagarajan Natarajan. MASAI: Modular architecture for software-engineering AI agents, 2024. URL https://arxiv.org/abs/2406.11638.

Mohammad Asadolahi, Amir Amini, Samira Talebi, Amirfarhad Farhadi, and Azadeh Zamanifar. Memory reward inflation in self-improving LLM agents, 2026. URL https://arxiv.org/abs/2608.00017.

Augment Code. How we built memory review for augment agent. Official product blog, 2025. URL https://www.augmentcode.com/blog/how-we-built-memory-review. Updated 2026-06-18; accessed 2026-08-18.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, 2025. URL https://arxiv.org/abs/2505.20411.

Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How do AI agents spend your money? analyzing and predicting token consumption in agentic coding tasks, 2026a. URL https://arxiv.org/abs/2604.22750.

Wenji Bai, Muhammad Waseem, Zeeshan Rasheed, Jaakko Peltonen, and Pekka Abrahamsson. When passing tests hides vulnerabilities: An empirical study of silent failures in agentic systems, 2026b. URL https://arxiv.org/abs/2609.10548.

Oussama Ben Sghaier, Hao Li, Bram Adams, and Ahmed E. Hassan. Don't blame the large language model: How agent harness evolution shapes coding agent quality, 2026. URL https://arxiv.org/abs/2607.03691.

Islem Bouzenia and Michael Pradel. You name it, i run it: An LLM agent to execute tests of arbitrary projects, 2024. URL https://arxiv.org/abs/2412.10133.

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair, 2024. URL https://arxiv.org/abs/2403.17134.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert,

Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws, 2026a. URL https://arxiv.org/abs/2606.06324.

Silin Chen, Han Li, Xiaodong Gu, Yuling Shi, and Haibing Guan. SkillForge: Self-Distilling Agents for Project-Specific Issue Resolution, 2026b. URL https://arxiv.org/abs/2608.18933.

Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2026c. URL https://arxiv.org/abs/2507.23361.

Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. HarnessX: A composable, adaptive, and evolvable agent harness foundry, 2026d. URL https://arxiv.org/abs/2606.14249.

Zhi Chen and Lingxiao Jiang. Evaluating software development agents: Patch patterns, code quality, and issue complexity in real-world GitHub scenarios, 2024. URL https://arxiv.org/abs/2410.12468.

Caroline Choi, Zeyneb Kaya, Shirley Wu, Tengyu Ma, Tatsunori Hashimoto, and Ludwig Schmidt. Anchored Self-Play for Code Repair, 2026. URL https://arxiv.org/abs/2607.03523.

Cognition. Devin Session Insights. Official documentation, 2026. URL https://docs.devin.ai/product-guides/session-insights. Accessed 2026-08-18.

Cursor. Automations with past-run memory. Cursor changelog, March 5, 2026, 2026. URL https://cursor.com/changelog/03-05-26. Accessed 2026-08-18.

Jeff Da, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. Agent-RLVR: Training software engineering agents via guidance and environment rewards, 2025. URL https://arxiv.org/abs/2506.11425.

DeepSeek-AI. DeepSeek Harness developer preview: Everything is a plugin. Official product documentation, 2026a. URL https://www.deepseek.com/harness/en/. Accessed 2026-09-11.

DeepSeek-AI. Composition and HMR: DeepSeek Harness developer documentation. Official developer documentation, 2026b. URL https://deepseek-harness.github.io/deepseek-harness/cordis-tutorial/06-composition-and-hmr/. Accessed 2026-09-11.

Bo Deng, Kang Zhou, Lifan Guo, Chongyang Tao, Xuanren Chen, Chenggang Xie, Renzhao Liang, Feng Chen, and Chi Zhang. FinEvo-Bench: A longitudinal benchmark for self-evolving agents in professional financial workflows, 2026a. URL https://arxiv.org/abs/2608.06144.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?, 2025. URL https://arxiv.org/abs/2509.16941.

Yi-Xuan Deng, Xiaoqin Liu, Yi Zhang, Guo-Wei Yang, and Shuojin Yang. Your code agent can grow alongside you with structured memory, 2026b. URL https://arxiv.org/abs/2603.13258.

Zijian Du and Nathaniel Pinckney. Trace2Skill: Verifier-Guided Skill Evolution for Long-Context EDA Agents, 2026. URL https://arxiv.org/abs/2605.21810.

Anastasia Eliseeva, Alexey Kovrigin, Ilya Kholkin, Egor Bogomolov, and Yaroslav Zharov. EnvBench: A benchmark for automated environment setup. In *ICLR Workshop on Deep Learning for Code*, 2025.

Ali Mohammadi Esfahani, Nafiseh Kahani, and Samuel A.Ajila. Prompt Optimization for LLM Code Generation via Reinforcement Learning, 2026. URL https://arxiv.org/abs/2605.19102.

Lishui Fan, Mouxiang Chen, Tingwei Zhu, Kui Liu, Xin Xia, Shanping Li, and Zhongxin Liu. ZeroCoder: Can LLMs improve code generation without ground-truth supervision?, 2026. URL https://arxiv.org/abs/2604.07864.

Tianyu Fan and Chao Huang. HELIX: Model-harness co-evolution for recursive self-improvement, 2026. URL https://arxiv.org/abs/2608.13951.

Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems, 2025. URL https://arxiv.org/abs/2508.07407.

Dayuan Fu, Mohan Jiang, Tongyu Wang, Dian Yang, Jiarui Hu, Liming Liu, Jinlong Hou, and Pengfei Liu. daVinci-kernel: Co-Evolving Skill Selection, Summarization, and Utilization via RL for GPU Kernel Optimization, 2026a. URL https://arxiv.org/abs/2606.16497.

Xinghong Fu, Aravinth Kulanthaivelu, and Yutaro Yamada. Self-improvement via fast tree-search. In *International Conference on Learning Representations*, 2026b.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence, 2026a. URL https://arxiv.org/abs/2507.21046.

Shuzheng Gao, Wenhao Zeng, Zhaojian Yu, Jianqiao Wangni, Chaozheng Wang, Kai Cai, Shilin He, and Michael R. Lyu. SWE-MeM: Learning adaptive memory management for long-horizon coding agents, 2026b. URL https://arxiv.org/abs/2606.28434.

Spandan Garg and Yufan Huang. Debug2Fix: Can interactive debugging help coding agents fix more bugs?, 2026. URL https://arxiv.org/abs/2602.18571.

GitHub. About GitHub Copilot Memory. Official documentation, 2026. URL https://docs.github.com/en/copilot/concepts/agents/copilot-memory. Accessed 2026-08-18.

Jingzhi Gong, Ruizhen Gu, Zhiwei Fei, Yazhuo Cao, Lukas Twist, Alina Geiger, Shuo Han, Dominik Sobania, Federica Sarro, and Jie M. Zhang. SkillMOO: Multi-objective Optimization of Agent Skills for Software Engineering, 2026. URL https://arxiv.org/abs/2604.09297.

Google. Gemini CLI Auto Memory. Official documentation, 2026. URL https://github.com/google-gemini/gemini-cli/blob/main/docs/cli/auto-memory.md. Accessed 2026-08-18.

Tianyi Guan, Yiding Wang, Haotong Yang, Siyuan Cao, Shirui Liu, Yi Hu, Jiaqi Li, and Muhan Zhang. ContinualSkill-Bench: Can LLM agents truly evolve their capabilities?, 2026. URL https://arxiv.org/abs/2608.03874.

Diandian Guo, Cong Cao, Fangfang Yuan, Yingqi Wang, Yueshan Wang, and Dakui Wang. Self-authored verification is unreliable in heuristic self-improving agents, 2026a. URL https://arxiv.org/abs/2607.24300.

Xinshuai Guo, Jiayi Kuang, Linyue Pan, Yinghui Li, Yangning Li, Hai-Tao Zheng, Ying Shen, Di Yin, and Xing Sun. EvoConfig: Self-evolving multi-agent systems for efficient autonomous environment configuration, 2026b. URL https://arxiv.org/abs/2601.16489.

Zhezheng Hao, Hong Wang, Jian Luo, Jianqing Zhang, Yuyan Zhou, Qiang Lin, Can Wang, Hande Dong, and Jiawei Chen. ReCreate: Reasoning and Creating Domain Agents Driven by Experience. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026a. URL https://aclanthology.org/2026.acl-long.1432/.

Zhezheng Hao, Tianfu Wang, Huanshuo Dong, Ziyan Liu, Hong Wang, Xiankun Lin, Qiang Lin, Can Wang, Hande Dong, and Jiawei Chen. Evolve as a Team: Collaborative Self-Evolution for LLM-Based Multi-Agent Systems, 2026b. URL https://arxiv.org/abs/2605.29790.

Hiroaki Hayashi, Bo Pang, Wenting Zhao, Ye Liu, Akash Gokul, Srijan Bansal, Caiming Xiong, Semih Yavuz, and Yingbo Zhou. Self-abstraction from grounded experience for plan-guided policy refinement, 2025. URL https://arxiv.org/abs/2511.05931.

Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks, 2026. URL https://arxiv.org/abs/2602.16313.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS, 2021. URL https://arxiv.org/abs/2105.09938.

Matthew Ho, Brian Liu, Jixuan Chen, Audrey Wang, and Lianhui Qin. SIGA: Self-evolving coding-agent adapters for scientific simulation, 2026. URL https://arxiv.org/abs/2606.09774.

Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework, 2023. URL https://arxiv.org/abs/2308.00352.

Haichuan Hu, Guoqing Xie, Quanjun Zhang, Jiawei Liu, Shengcheng Yu, Chunrong Fang, Zhenyu Chen, and Liang Xiao. EvoRepair: Enhancing vulnerability repair agents through experience-based self-evolution, 2026. URL https://arxiv.org/abs/2605.30105.

Ruijie Hu, Cheng Peng, Xinchen Wang, Jia Xu, and Cuiyun Gao. Repo2Run: Automated building executable environment for code repository at scale. In *Advances in Neural Information Processing Systems*, 2025.

Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems (ADAS), 2024a. URL https://arxiv.org/abs/2408.08435.

Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. Self-evolving multi-agent collaboration networks for software development, 2024b. URL https://arxiv.org/abs/2410.16946.

Shuyan Huang, Kai Du, and Andrew Lan. Do personalized skills help coding agents? an empirical study of developer interaction histories, 2026a. URL https://arxiv.org/abs/2608.10319.

Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026b. URL https://arxiv.org/abs/2607.07946.

Yixu Huang, Xinglei Yu, and Zhongyu Wei. ACE: Self-evolving LLM coding framework via adversarial unit test generation and preference optimization, 2026c. URL https://arxiv.org/abs/2605.16299.

Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. From raw experience to skill consumption: A systematic study of model-generated agent skills, 2026d. URL https://arxiv.org/abs/2605.23899.

Alex Iacob, Andrej Jovanović, William F. Shen, Daniel Burkhardt, Meghdad Kurmanji, Nurbek Tastan, Lorenzo Sani, Niccolò Alberto Elia Venanzi, Ambroise Odonnat, Zeyu Cao, Bill Marino, Xinchi Qiu, and Nicholas D. Lane. The Red Queen Gödel Machine: Co-Evolving Agents and Their Evaluators (RQGM), 2026. URL https://arxiv.org/abs/2606.26294.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents, 2025. URL https://arxiv.org/abs/2504.07164.

Sihang Jiang, Lipeng Ma, Zhonghua Hong, Keyi Wang, Zhiyu Lu, Shisong Chen, Jinghao Zhang, Tianjun Pan, Weijia Zhou, Jiaqing Liang, and Yanghua Xiao. SEA-Eval: A benchmark for evaluating self-evolving agents beyond episodic assessment, 2026. URL https://arxiv.org/abs/2604.08988.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues?, 2023. URL https://arxiv.org/abs/2310.06770.

Yiyang Jin, Kunzhao Xu, Hang Li, Xueting Han, Yanmin Zhou, Cheng Li, and Jing Bai. ReVeal: Self-evolving code agents via iterative generation-verification, 2025. URL https://arxiv.org/abs/2506.11442.

Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents, 2026. URL https://arxiv.org/abs/2604.14004.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses, 2026. URL https://arxiv.org/abs/2603.28052.

Letta. Introducing the Letta Code app. Official product blog, 2026. URL https://www.letta.com/blog/introducing-the-letta-code-app/. Accessed 2026-09-11.

Boxiu Li, Zimo Wen, Yijia Fan, Chuan Wen, Fan Yang, Hangxi Guo, Jiaao Wu, Jiachen Zhang, Junxiang Lei, Mukai Li, Ruize Tang, Runjing Gu, Shibo Hu, Sihan Chen, Sufeng Guo, Wanbo Zhang, Xian Zhang, Xiaoyu Chen, Xuanhe Zhou, Xuyao Huang, Yifei Gao, Yifei Shen, Yilin Chen, Yuheng Wu, Yuzhe Zhang, Zelong Zhao, and Zhijie Deng. Argus: A general-purpose agentic reasoning runtime for long-horizon tasks, 2026a. URL https://arxiv.org/abs/2608.05144.

Mo Li, L.H. Xu, Qitai Tan, Ting Cao, and Yunxin Liu. Learning to Commit: Generating Organic Pull Requests via Online Repository Memory, 2026b. URL https://arxiv.org/abs/2603.26664.

Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. CODESKILL: Learning self-evolving skills for coding agents, 2026c. URL https://arxiv.org/abs/2605.25430.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode, 2022. URL https://arxiv.org/abs/2203.07814.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026a. URL https://arxiv.org/abs/2604.25850.

Jiaye Lin, Yifu Guo, Yuzhen Han, Sen Hu, Ziyi Ni, Licheng Wang, Mingguang Chen, Hongzhang Liu, Ronghao Chen, Yangfan He, Daxin Jiang, Binxing Jiao, Chen Hu, and Huacan Wang. SE-Agent: Self-Evolution Trajectory Optimization in Multi-Step Reasoning with LLM-Based Agents, 2025a. URL https://arxiv.org/abs/2508.02085.

Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving LLM agents, 2026b. URL https://arxiv.org/abs/2605.30621.

Yalan Lin, Yingwei Ma, Rongyu Cao, Binhua Li, Fei Huang, Xiaodong Gu, and Yongbin Li. LLMs as continuous learners: Improving the reproduction of defective code in software issues, 2024. URL https://arxiv.org/abs/2411.13941.

Zi Lin, Sheng Shen, Jingbo Shang, Jason Weston, and Yixin Nie. Learning to solve and verify: A self-play framework for code and test generation, 2025b. URL https://arxiv.org/abs/2502.14948.

Tobias Lindenbauer, Georg Groh, and Hinrich Schuetze. From Knowledge to Noise: CTIM-Rover and the Pitfalls of Episodic Memory in Software Engineering Agents. In *Proceedings of the 1st Workshop for Research on Agent Language Models (REALM 2025)*. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.realm-1.30/.

Hongyi Liu, Haoyan Yang, Tao Jiang, Bo Tang, Feiyu Xiong, Yuyu Luo, and Zhiyu Li. SkillsVote: Lifecycle Governance of Agent Skills from Collection, Recommendation to Evolution, 2026a. URL https://arxiv.org/abs/2605.18401.

Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. Large language model-based agents for software engineering: A survey, 2024. URL https://arxiv.org/abs/2409.02977.

Simiao Liu, Li Zhang, Fang Liu, Xiaoli Lian, Yang Liu, and Yinghao Zhu. MemRepair: Hierarchical Memory for Agentic Repository-Level Vulnerability Repair, 2026b. URL https://arxiv.org/abs/2605.17444.

Siwei Liu, Jinyuan Fang, Han Zhou, Yingxu Wang, and Zaiqiao Meng. SEW: Self-evolving agentic workflows for automated code generation, 2025. URL https://arxiv.org/abs/2505.18646.

Wei Liu, Siya Qi, Yali Du, and Yulan He. Self-play only evolves when self-synthetic pipeline ensures learnable information gain, 2026c. URL https://arxiv.org/abs/2603.02218.

Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution, 2026. URL https://arxiv.org/abs/2607.13683.

Zihan Ma, Zhikai Zhao, Chuanbo Hua, Federico Berto, and Jinkyoo Park. JUDGEFLOW: Agentic Workflow Optimization via Block Judge, 2026. URL https://arxiv.org/abs/2601.07477.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. URL https://arxiv.org/abs/2303.17651.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Rao, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868.

Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. EXPEREPAIR: Dual-memory enhanced LLM-based repository-level program repair, 2025. URL https://arxiv.org/abs/2506.10484.

Jun Nie, Yonggang Zhang, Qianshu Cai, Yiu ming Cheung, Xinmei Tian, and Bo Han. EvolveNet: Collaborative harness evolution for agent self-improvement, 2026a. URL https://arxiv.org/abs/2608.04968.

Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution, 2026b. URL https://arxiv.org/abs/2607.08124.

Xuying Ning et al. Code as agent harness, 2026. URL https://arxiv.org/abs/2605.18747.

Nous Research. Hermes Agent: Skills system. Official product documentation, 2026. URL https://hermes-agent.nousresearch.com/docs/user-guide/features/skills. Accessed 2026-09-11.

Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

OpenBlock Labs. OB-1: The agent that built itself. Official product page and owner's manual, 2026. URL https://www.openblocklabs.com/. Accessed 2026-08-18.

OpenClaw. OpenClaw Memory Overview. Official product documentation, 2026. URL https://docs.openclaw.ai/concepts/memory. Accessed 2026-09-11.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory, 2025. URL https://arxiv.org/abs/2509.25140.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2025. URL https://arxiv.org/abs/2412.21139.

Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces, 2026. URL https://arxiv.org/abs/2608.23041.

Zehua Pei, Hui-Ling Zhen, Yu Zhang, Sinno Jialin Pan, Mingxuan Yuan, and Bei Yu. Verilog-Evolve: Feedback-Driven and Skill-Evolving Verilog Generation, 2026. URL https://arxiv.org/abs/2605.26498.

Yulin Peng, Haowen Hou, Xinxin Zhu, Ying Tiffany He, and F. Richard Yu. SEMAG: Self-evolutionary multi-agent code generation, 2026a. URL https://arxiv.org/abs/2603.15707.

Yulin Peng, Xinxin Zhu, Chenxing Wei, Nianbo Zeng, Leilei Wang, Ying Tiffany He, and F. Richard Yu. SAGE: Multi-Agent Self-Evolution for LLM Reasoning, 2026b. URL https://arxiv.org/abs/2603.15255.

Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI (SOAR), 2025. URL https://arxiv.org/abs/2507.14172.

Prime Intellect. Prime Agent: A self-improving RLM agent. Official repository, 2026. URL https://github.com/PrimeIntellect-ai/prime-agent. Accessed 2026-08-18.

Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=oVKEAFjEqv.

Anton Razzhigaev, Andrei Gritsaev, Andrei Kaznacheev, Nikita Dragunov, Roman Yampolskiy, and Andrei Kuznetsov. Ouroboros: A self-developing frontier coding agent with reviewed core evolution, 2026. URL https://arxiv.org/abs/2608.08311.

Jincheng Ren, Siwei Wu, Yizhi Li, Kang Zhu, Shu Xu, Boyu Feng, Ruibin Yuan, Wei Zhang, Riza Batista-Navarro, Jian Yang, and Chenghua Lin. A self-evolving framework for efficient terminal agents via observational context compression, 2026a. URL https://arxiv.org/abs/2604.19572.

Zeyu Ren, Ling Yue, Ran Li, Yishu Wang, Shengxiang Xu, Hanmo Liu, Shaowu Pan, and Shimin Di. FlowEvo: Self-Evolving Agents through the Co-Evolution of Workflows and Executable Skills, 2026b. URL https://arxiv.org/abs/2607.21596.

Replit. Replit Agent Skills. Official product documentation, 2026. URL https://docs.replit.com/replitai/skills. Accessed 2026-09-11.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL https://arxiv.org/abs/2504.15228.

Biswa Sengupta. Self-evolving agents with anytime-valid certificates, 2026. URL https://arxiv.org/abs/2607.00871.

Linfang Shang, Ming Xu, Yiding Sun, Tianle Xia, Lingxiang Hu, Lan Xu, and Ning Zheng. When Self-Evolution Backfires: Pre-Commit Gating against Skill Contamination in LLM Agents, 2026. URL https://arxiv.org/abs/2608.05810.

Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories, 2026. URL https://arxiv.org/abs/2608.02276.

Kangning Shen, Jingyuan Zhang, Chenxi Sun, Wencong Zeng, and Yang Yue. Structurally aligned subtask-level memory for software engineering agents, 2026. URL https://arxiv.org/abs/2602.21611.

Yifan Shi, Wei Zhang, and Tianyi Cui. A programming paradigm for spatiotemporal composability. Preprint, draft of August 13, 2026, 2026a. URL https://github.com/cordiverse/paper/blob/main/paper.pdf. Peking University and DeepSeek-AI.

Yucheng Shi, Zhenwen Liang, Kishan Panaganti, Dian Yu, Wenhao Yu, and Haitao Mi. Learning to Build the Environment: Self-Evolving Reasoning RL via Verifiable Environment Synthesis (EvoEnv), 2026b. URL https://arxiv.org/abs/2605.14392.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

KaShun Shum, Binyuan Hui, Jiawei Chen, Lei Zhang, X. W., Jiaxi Yang, Yuzhen Huang, Junyang Lin, and Junxian He. SWE-RM: Execution-free feedback for software engineering agents, 2025. URL https://arxiv.org/abs/2512.21919.

Hongjin Su, Shuyang Jiang, Yuhang Lai, Haoyuan Wu, Boao Shi, Che Liu, Qian Liu, and Tao Yu. EVOR: Evolving retrieval for code generation, 2024. URL https://arxiv.org/abs/2402.12317.

Hamed Taherkhani, Melika Sepindband, Hung Viet Pham, Song Wang, and Hadi Hemmati. Automated prompt engineering for cost-effective code generation using evolutionary algorithm, 2024. URL https://arxiv.org/abs/2408.11198.

Shangyin Tan, Lakshya A. Agrawal, Rohit Sandadi, Dan Klein, Koushik Sen, Alexandros G. Dimakis, and Matei Zaharia. Automatically learning skills for coding agents. In *Proceedings of the ACM Conference on AI and Agentic Systems*, 2026. doi: 10.1145/3786335.3813196. URL https://doi.org/10.1145/3786335.3813196.

Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. AGENT KB: Leveraging cross-domain experience for agentic problem solving, 2025. URL https://arxiv.org/abs/2507.06229.

Wangcheng Tao, Han Wu, and Weng-Fai Wong. SePO: Self-evolving prompt agent for system prompt optimization, 2026. URL https://arxiv.org/abs/2606.04465.

Minh V. T. Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios, 2025. URL https://arxiv.org/abs/2512.18470.

Trae Research Team, Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchen Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, Yun Lin, Yingfei Xiong, Chao Peng, and Xia Liu. Trae Agent: An LLM-based agent for software engineering with test-time scaling, 2025. URL https://arxiv.org/abs/2507.23370.

Naman Vats and Oleg Golev. The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation, 2026. URL https://arxiv.org/abs/2607.22585.

Boshi Wang, Weijian Xu, Yunsheng Li, Mei Gao, Yujia Xie, Huan Sun, and Dongdong Chen. Improving code localization with repository memory, 2026a. URL https://arxiv.org/abs/2510.01003.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL https://arxiv.org/abs/2305.16291.

Junjie Wang, Yiming Ren, and Haoyang Zhang. From Procedural Skills to Strategy Genes: Towards Experience-Driven Test-Time Evolution, 2026b. URL https://arxiv.org/abs/2604.15097.

Peiding Wang, Li Zhang, Fang Liu, Chongyang Tao, and Yinghao Zhu. CodeMEM: AST-guided adaptive memory for repository-level iterative code generation, 2026c. URL https://arxiv.org/abs/2601.02868.

Siyu Wang, Ruotian Lu, Zhihao Yang, Yuchao Wang, Yanzhou Zhang, Lei Xu, Qimin Xu, Guojun Yin, Cailian Chen, and Xinping Guan. AgentConductor: Topology evolution for multi-agent competition-level code generation, 2026d. URL https://arxiv.org/abs/2602.17100.

Su Wang, Pin Qian, Yifan Lin, Jingzhou Xu, Yihang Chen, Xiaochong Jiang, Lifei Liu, and Haoran Yu. Phantom guardrails: When self-improving agent harnesses fix failures that never happened, 2026e. URL https://arxiv.org/abs/2607.13083.

Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley Gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025a. URL https://arxiv.org/abs/2510.21614.

Xing Wang, Xiaoyi Zhang, and Jie Shao. Auditing harness tampering in self-improving agents, 2026f. URL https://arxiv.org/abs/2609.00069.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents, 2024a. URL https://arxiv.org/abs/2402.01030.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents, 2024b. URL https://arxiv.org/abs/2407.16741.

Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. Agents in software engineering: Survey, landscape, and vision. *Automated Software Engineering*, 32(2):70, 2025b. doi: 10.1007/s10515-025-00544-2. URL https://doi.org/10.1007/s10515-025-00544-2.

Yaoxiang Wang, Qi Shi, ShangZhan Li, Qingguo Hu, Xinyu Yin, Bo Guo, Xu Han, Maosong Sun, and Jinsong Su. VeriAgent: A Tool-Integrated Multi-Agent System with Evolving Memory for PPA-Aware RTL Code Generation, 2026g. URL https://arxiv.org/abs/2603.17613.

Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents, 2026h. URL https://arxiv.org/abs/2607.12227.

Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. EvoAgentX: An automated framework for evolving agentic workflows, 2025c. URL https://arxiv.org/abs/2507.03616.

Yinjie Wang, Ling Yang, Ye Tian, Ke Shen, and Mengdi Wang. CURE: Co-evolving LLM coder and unit tester via reinforcement learning, 2025d. URL https://arxiv.org/abs/2506.03136.

Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. SWE-Pruner: Self-adaptive context pruning for coding agents, 2026i. URL https://arxiv.org/abs/2601.16746.

Zimu Wang, Yuling Shi, Mengfan Li, Zijun Liu, Jie M. Zhang, Chengcheng Wan, and Xiaodong Gu. EffiSkill: Agent skill based automated code efficiency optimization, 2026j. URL https://arxiv.org/abs/2603.27850.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR, 2025e. URL https://proceedings.mlr.press/v267/wang25bx.html.

Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-Memory: Benchmarking LLM agent test-time learning with self-evolving memory, 2025a. URL https://arxiv.org/abs/2511.20857.

Tianxin Wei, Zhan Shi, Minhua Lin, Bing He, Zewen Liu, Yisi Sang, Yuanchen Bei, Xuying Ning, Jiaru Zou, Ting-Wei Li, Xiao Lin, Yanjun Zhao, Chi Wang, Benoit Dumoulin, Dakuo Wang, Jingrui He, and Hanqing Lu. Evo-Harness: Context-to-harness skill compilation for self-evolving agents, 2026a. URL https://arxiv.org/abs/2608.15071.

Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution, 2025b. URL https://arxiv.org/abs/2502.18449.

Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. Toward training superintelligent software agents through self-play SWE-RL, 2026b. URL https://arxiv.org/abs/2512.18552.

Zichao Wei, Jun Zeng, Ming Wen, Zeliang Yu, Kai Cheng, Yiding Zhu, Jingyi Guo, Shiqi Zhou, Le Yin, Xiaodong Su, and Zhechao Ma. PATCHEVAL: A new benchmark for evaluating LLMs on patching real-world vulnerabilities, 2025c.

Zhaotian Weng, Antonis Antoniades, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-evolving agents: Open-ended self-improvement via experience sharing, 2026. URL https://arxiv.org/abs/2602.04837.

Windsurf and Cognition. Cascade Memories and Rules. Official documentation, 2026. URL https://docs.windsurf.com/windsurf/cascade/memories. Accessed 2026-08-18.

Sherman Wong, Zhenting Qi, Zhaodong Wang, Nathan Hu, Samuel Lin, Jun Ge, Erwin Gao, Wenlin Chen, Yilun Du, Minlan Yu, and Ying Zhang. Confucius code agent: Scalable agent scaffolding for real-world codebases, 2025. URL https://arxiv.org/abs/2512.10398.

Xiaodong Wu, Yu Shi, Qi Li, Zhimin Zhao, Xiangman Li, Bram Adams, Ahmed E. Hassan, and Jianbing Ni. EVOMAL: Self-Poisoning in Self-Evolving Coding Agents, 2026. URL https://arxiv.org/abs/2608.25776.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024. URL https://arxiv.org/abs/2407.01489.

Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent: Can software engineering agents self-evolve on the fly?, 2025. URL https://arxiv.org/abs/2511.13646.

Chuan Xiao, Zhengbo Jiao, Shaobo Wang, Wei Wang, Bing Zhao, Hu Wei, Linfeng Zhang, and Lin Qu. Socratic-SWE: Self-evolving coding agents via trace-derived agent skills, 2026a. URL https://arxiv.org/abs/2606.07412.

Yang Xiao, Yusong Sun, Haoyi Wu, Wenyang Hui, Wen Da, Zhaokai Luo, Mu Chuan, Yao Hu, Wenjie Li, and Chengyue Jiang. PILOT in the Loop: Live Self-Improvement for Long-Horizon Agents, 2026b. URL https://arxiv.org/abs/2608.26530.

Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.27. URL https://aclanthology.org/2026.acl-long.27/.

Tianshi Xu, Huifeng Wen, and Meng Li. Adapting the interface, not the model: Runtime harness adaptation for deterministic LLM agents, 2026. URL https://arxiv.org/abs/2605.22166.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2502.12110.

Shuhan Xue, Zixin Ding, Yichen Shen, Yinjie Wang, Zhenfei Yin, Yingcheng Wu, Yuxin Chen, Mengdi Wang, and Ling Yang. PAST-Bench: Benchmarking the foundations of recursive self-improvement in personal agents, 2026. URL https://arxiv.org/abs/2608.04003.

Haoyang Yan, Min le Su, Hangfan Zhang, Zhanhao Li, Chen Zhang, Shao Zhang, Yang Chen, Lei Bai, and Shuyue Hu. Harness-of-Harness: Multi-Day Autonomous Software Development with Continual Improvement, 2026. URL https://arxiv.org/abs/2609.01481.

Chen Yang, Jiashuo Tian, Ziqi Wang, Xinyin Liu, Meiru Ye, and Junjie Chen. Learning globally reusable skills for coding agents, 2026a. URL https://arxiv.org/abs/2608.06153.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024. URL https://arxiv.org/abs/2405.15793.

Siqi Yang, Qianlan Yang, Yu-Xiong Wang, Saurabh Pujar, and Martin Hirzel. One recipe, many harnesses: What self-evolution encodes across languages and models, 2026b. URL https://arxiv.org/abs/2608.10178.

Xidong Yang, Xingyi Zhang, Wenhao Li, Wenyan Liu, Junjie Sheng, Yun Hua, Wei Yin, Tao Fang, Chuyun Shen, and Xiangfeng Wang. PATH-Bench: Path-dependent evaluation of lifelong agents, 2026c. URL https://arxiv.org/abs/2608.01149.

Qiming Ye, Peixian Zhang, Yupeng He, Zifan Peng, and Gareth Tyson. Behind EvoMap: Characterizing a self-evolving agent-to-agent collaboration network, 2026. URL https://arxiv.org/abs/2605.25815.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1354. URL https://aclanthology.org/2025.acl-long.1354/.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling*, 2024. URL https://arxiv.org/abs/2310.02304.

Di Zhang. AgentDevel: Reframing Self-Evolving LLM Agents as Release Engineering, 2026. URL https://arxiv.org/abs/2601.04620.

Genghan Zhang, Weixin Liang, Olivia Hsu, and Kunle Olukotun. Adaptive Self-improvement LLM Agentic System for ML Library Development, 2025a. URL https://arxiv.org/abs/2502.02534.

Guibin Zhang, Kaijie Chen, Guancheng Wan, Heng Chang, Hong Cheng, Kun Wang, Shuyue Hu, and Lei Bai. EvoFlow: Evolving Diverse Agentic Workflows On The Fly, 2025b. URL https://arxiv.org/abs/2502.07373.

Guibin Zhang, Xun Xu, Yanwei Yue, Zikun Su, Wangchunshu Zhou, Xiaobin Hu, and Shuicheng Yan. OPD-Evolver: Cultivating Holistic Agent Evolver via On-Policy Distillation, 2026a. URL https://arxiv.org/abs/2606.17628.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves, 2026b. URL https://arxiv.org/abs/2606.09498.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents, 2025c. URL https://arxiv.org/abs/2505.22954.

Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. HyperAgents, 2026c. URL https://arxiv.org/abs/2603.19461.

Jiahao Zhang, Yifan Zhang, and Yu Huang. Coupling planning with episodic memory in LLM agents for software issue resolution, 2026d. URL https://arxiv.org/abs/2608.06811.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation, 2024a. URL https://arxiv.org/abs/2410.10762.

Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. SWE-bench Goes Live!, 2025d. URL https://arxiv.org/abs/2505.23419.

Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, and Yan Xu. HarnessCompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses, 2026e. URL https://arxiv.org/abs/2608.01918.

Quanjun Zhang, Chengyu Gao, Yu Han, Ye Shang, Chunrong Fang, Zhenyu Chen, and Liang Xiao. Sgagent: Suggestion-guided llm-based multi-agent framework for repository-level software repair. *ACM Transactions on Software Engineering and Methodology*, 2026f. doi: 10.1145/3818617. Just Accepted.

Wentao Zhang, Jianfeng Wang, Liheng Liang, Yilei Zhao, Haibin Wen, and Zhe Zhao. EvoCodeBench: A human-performance benchmark for self-evolving LLM-driven coding systems, 2026g. URL https://arxiv.org/abs/2602.10171.

Wentao Zhang, Zhe Zhao, Haibin Wen, Yingcheng Wu, Cankun Guo, Ming Yin, and Bo An. Autogenesis: A Self-Evolving Agent Protocol, 2026h. URL https://arxiv.org/abs/2604.15034.

Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Library Drift: Diagnosing and fixing a silent failure mode in self-evolving LLM skill libraries, 2026i. URL https://arxiv.org/abs/2605.19576.

Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Ratchet: How reliable must an LLM judge be to retire a skill?, 2026j. URL https://arxiv.org/abs/2605.22148.

Xing Zhang, Guanghui Wang, Yanwei Cui, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Who Grades the Grader? Co-Evolving Evaluation Metrics and Skills for Self-Improving LLM Agents, 2026k. URL https://arxiv.org/abs/2607.12790.

Yifan Zhang, Yutong Dai, Juntao Tan, Luyu Yang, Rishi Mullur, Thai Hoang, Zhiyuan Hu, James Zhu, Phil Mui, Silvio Savarese, Ran Xu, and Zeyuan Chen. DarwinX: Evolving agent harnesses through natural selection, 2026l. URL https://arxiv.org/abs/2608.07545.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement, 2024b. URL https://arxiv.org/abs/2404.05427.

Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Haibo Qiu, Qingnan Ren, Shun Zou, Zehui Chen, Feng Zhao, et al. SkillFlow: Benchmarking lifelong skill discovery and evolution for autonomous agents, 2026m. URL https://arxiv.org/abs/2604.17308.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners, 2024. URL https://arxiv.org/abs/2308.10144.

Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute Zero: Reinforced Self-Play Reasoning with Zero Data, 2025. URL https://arxiv.org/abs/2505.03335.

Xuan Zhao, Andy Chiu, and Gengyu Wang. Libra: Training the environment for agentic information retrieval, 2026. URL https://arxiv.org/abs/2607.00016.

Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, Zhongzhi Li, Yingying Zhang, Le Song, and Qianli Ma. LifelongAgentBench: Evaluating LLM agents as lifelong learners, 2025. URL https://arxiv.org/abs/2505.11942.

Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. SkillLearnBench: Benchmarking continual learning methods for agent skill generation on real-world tasks, 2026. URL https://arxiv.org/abs/2604.20087.

Leijun Zhou, Zhihao Liu, Xiang Qu, Chenxu Liu, Yifei Liu, Yanke Yu, Jingzhe Xu, Xuejun Wu, Buyue Qian, Xi Chen, Yaowei Zheng, and Junhao Hu. GDPevo: Evaluating agent self-evolution on real business tasks, 2026. URL https://arxiv.org/abs/2608.03764.